# Nanoprocessor Design Project Report

## CS1050 Computer Organization and Digital Design
## Department of Computer Science and Engineering
## University of Moratuwa

### TEAM 5

| Student Name | Index Number |
|---|---|
| G.D.J.P.Gamaetige | 240180N |
| H.M.V.L.Herath | 240225J |
| I.S.Perera | 240497R |
| K.D.B.M.Perera | 240498V |

# Table of Contents

# 1. Introduction

This report presents the design, implementation, simulation, and FPGA testing plan of a 4-bit nanoprocessor developed for the CS1050 Computer Organization and Digital Design Lab 9-10 project. The baseline task was to design a simple processor capable of executing a small instruction set consisting of MOVI, ADD, NEG, and JZR. The processor uses a register bank, program counter, program ROM, instruction decoder, arithmetic unit, multiplexers, and output display circuitry.

In addition to the basic nanoprocessor, an extended nanoprocessor version was implemented. The extended version increases the instruction word width, expands ROM and PC addressing, and introduces a multi-function ALU that supports extra arithmetic and logical operations such as SUB, AND, OR, XOR, MUL, DIV, NOT, CMP, and JMP. This report separates common components from the files that are unique to the basic and extended designs, so repeated VHDL source code is avoided.

## 2. Objectives

- Design and implement a 4-bit nanoprocessor using VHDL.
- Implement the baseline instruction set required in the lab specification.
- Create a program ROM containing an assembly program and its corresponding machine code.
- Use a register bank with eight 4-bit registers, with R0 hardwired to zero.
- Verify individual components and the complete processor using VHDL testbenches.
- Map the final design to the Basys 3 FPGA board using the required XDC constraints.
- Extend the basic processor with a larger instruction set and a more capable ALU.

# 3. System Overview

The nanoprocessor is built around the fetch-decode-execute cycle. The program counter selects an instruction from ROM. The instruction decoder interprets the opcode and register fields and then activates the correct control signals. The register bank provides operands to the arithmetic or logical unit through multiplexers. The result is written back to the selected destination register, and the program counter either increments or jumps depending on the instruction.

## 3.1 Basic Nano Processor Overview

The basic processor follows the lab specified datapath. It uses a 3-bit program counter, an 8-entry program ROM, and 12-bit instructions. It supports MOVI, ADD, NEG, and JZR. The stored program calculates 1 + 2 + 3 and stores the final result, 6, in R7. The value in R7 is connected to LEDs and the seven-segment display.

| Feature | Basic Nano Processor |
|---|---|
| Instruction width | 12 bits |
| Program counter width | 3 bits |
| ROM size | 8 instructions |
| Main operation set | MOVI, ADD, NEG, JZR |
| Final demo result | R7 = 6 |
| Top module file | src/nanoprocessor_top.vhd |

## 3.2 Extended Nano Processor Overview

The extended processor builds on the same register-bank and datapath idea but expands the control and execution capability. It uses a 4-bit program counter, a 16-entry ROM, and 14-bit instructions. The standalone add/subtract unit is reused inside a multi-function ALU. The demonstration program calculates (3 x 2) + (8 / 2) - (5 AND 3), stores 9 in R7, then demonstrates comparison, XOR, NOT, and a halt loop using JMP.

| Feature | Extended Nano Processor |
|---|---|
| Instruction width | 14 bits |
| Program counter width | 4 bits |
| ROM size | 16 instructions |
| Main operation set | ADD, NEG, MOVI, JZR, SUB, AND, OR, XOR, MUL, CMP, NOT, DIV, JMP |
| Final demo result | R7 = 9 |
| Top module file | src/nanoprocessor_top_v2.vhd |

# 4. Instruction Set and Machine Code

## 4.1 Basic Nano Processor Instruction Set

The basic processor uses the 12-bit instruction format specified in the lab sheet. The most significant bits select the instruction, while the remaining fields select registers and immediate/jump values.

| Instruction | Operation | 12-bit Format |
|---|---|---|
| MOVI R,d | R <- d | 10RRR000dddd |
| ADD Ra,Rb | Ra <- Ra + Rb | 00RaRaRaRbRbRb0000 |
| NEG R | R <- -R | 01RRR0000000 |
| JZR R,d | If R = 0, PC <- d; otherwise PC <- PC + 1 | 11RRR0000ddd |

## 4.2 Extended Nano Processor Instruction Set

The extended processor uses the 14-bit instruction format [op:4][Ra:3][Rb:3][imm:4]. This format allows more opcodes and a 4-bit jump target.

| Opcode | Mnemonic | Action | Notes |
|---|---|---|---|
| 0000 | ADD Ra,Rb | Ra <- Ra + Rb | Addition using ALU |
| 0001 | NEG R | R <- 0 - R | Two's complement negation |
| 0010 | MOVI R,d | R <- d | Immediate load |
| 0011 | JZR R,d | If R = 0, PC <- d | Conditional jump |
| 0100 | SUB Ra,Rb | Ra <- Ra - Rb | Subtraction |
| 0101 | AND Ra,Rb | Ra <- Ra AND Rb | Bitwise AND |
| 0110 | OR Ra,Rb | Ra <- Ra OR Rb | Bitwise OR |
| 0111 | XOR Ra,Rb | Ra <- Ra XOR Rb | Bitwise XOR |
| 1000 | MUL Ra,Rb | Ra <- low 4 bits of Ra x Rb | Overflow if upper nibble is non-zero |
| 1001 | CMP Ra,Rb | Zero flag <- Ra - Rb = 0 | No register write |
| 1010 | NOT R | R <- NOT R | Bitwise complement |
| 1011 | DIV Ra,Rb | Ra <- Ra / Rb | Divide by zero handled as 1111 with overflow |
| 1100 | JMP d | PC <- d | Unconditional jump |

# 5. Assembly Program and Machine Code

## 5.1 Basic Assembly Program and Machine Code

The basic program calculates 1 + 2 + 3 = 6. The loop counter is placed in R1, R2 stores -1 for decrementing, and R7 accumulates the answer.

| PC | Assembly Instruction | Machine Code | Purpose |
|---|---|---|---|
| 0 | MOVI R1, 1 | 100010000001 | Load the first number (1) into working register R1 |
| 1 | ADD R7, R1 | 001110010000 | Add 1 to R7 - running total becomes 1 |
| 2 | MOVI R1, 2 | 100010000010 | Load the second number (2) into R1, overwriting the previous value |
| 3 | ADD R7, R1 | 001110010000 | Add 2 to R7 - running total becomes 3 |
| 4 | MOVI R1, 3 | 100010000011 | DLoad the third number (3) into R1 |
| 5 | ADD R7, R1 | 001110010000 | Add 3 to R7 - running total becomes 6 (final answer) |
| 6 | JZR R0, 6 | 110000000110 | R0 is hardwired to 0, so the zero condition is always true - jumps back to PC=6 forever, halting the program on the final result |
| 7 | JZR R0, 7 | 110000000111 | Unreachable safety net - if execution ever reaches here it also halts in place |

## 5.2 Extended Assembly Program and Machine Code

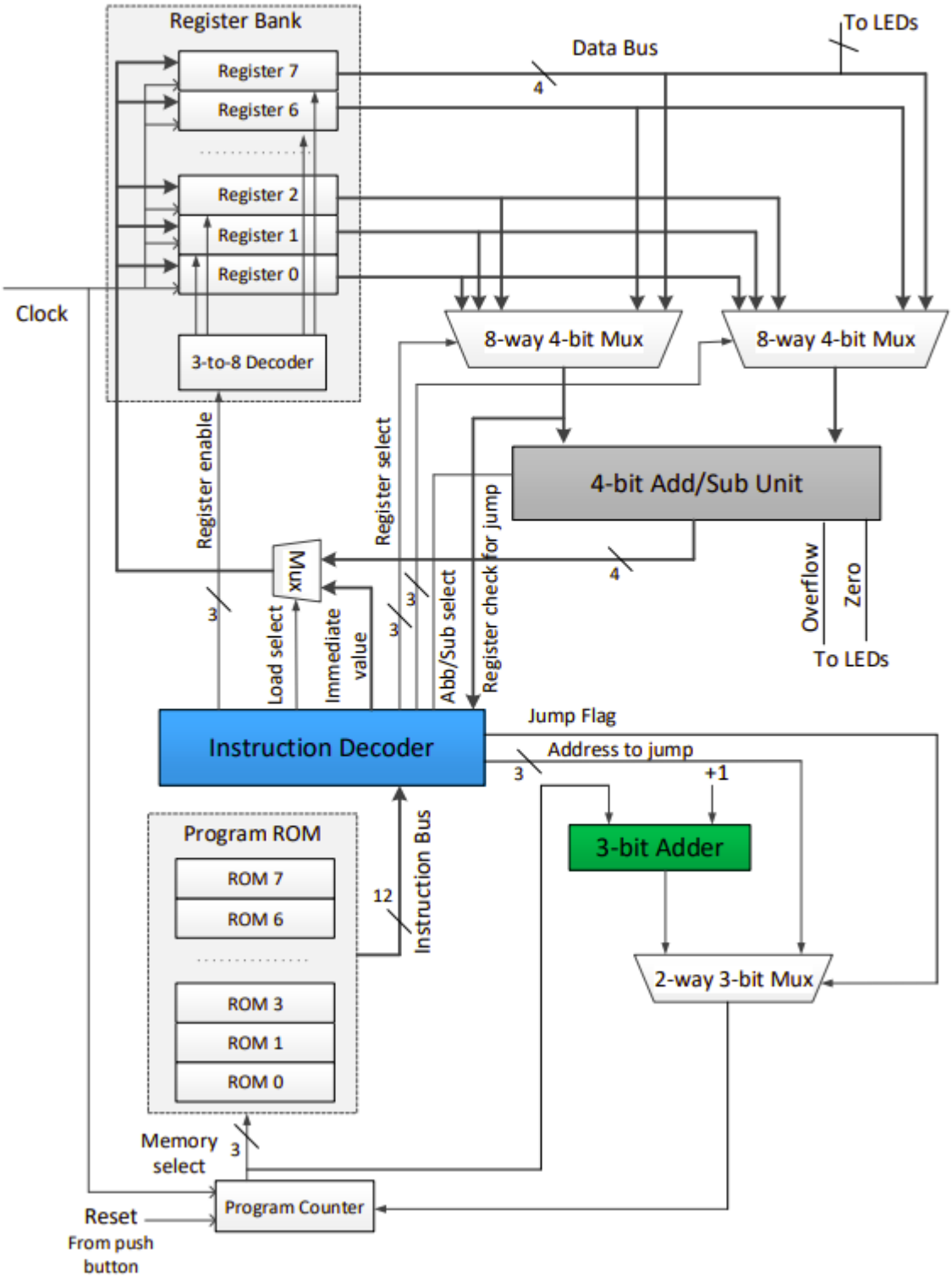
The extended program calculates (3 x 2) + (8 / 2) - (5 AND 3) = 9. It then demonstrates CMP, XOR, NOT, and JMP.

| PC | Assembly Instruction | 14-bit Machine Code | Effect |
|---|---|---|---|
| 0 | MOVI R1, 3 | 00100010000011 | R1 = 3 |
| 1 | MOVI R2, 2 | 00100100000010 | R2 = 2 |
| 2 | MUL R1, R2 | 10000010100000 | R1 = 6 |
| 3 | XOR R7, R7 | 01111111110000 | R7 = 0 |
| 4 | ADD R7, R1 | 00001110010000 | R7 = 6 |
| 5 | MOVI R3, 8 | 00100110001000 | R3 = 8 |
| 6 | MOVI R4, 2 | 00101000000010 | R4 = 2 |
| 7 | DIV R3, R4 | 10110111000000 | R3 = 4 |

| PC | Assembly Instruction | 14-bit Machine Code | Effect |
|---|---|---|---|
| 8 | ADD R7, R3 | 00001110110000 | R7 = 10 |
| 9 | MOVI R5, 5 | 00101010000101 | R5 = 5 |
| 10 | MOVI R6, 3 | 00101100000011 | R6 = 3 |
| 11 | AND R5, R6 | 01011011100000 | R5 = 1 |
| 12 | SUB R7, R5 | 01001111010000 | R7 = 9 |
| 13 | JMP 13 | 11000000001101 | — |
| 14 | JMP 15 | 11000000001111 | — |

# 6. High-Level Diagrams

## 6.1 Basic Nano Processor High-Level Diagram



# 7. Design Implementation

The final report structure separates the design into common components, basic-only components, and extended-only components. This makes the report easier to check because VHDL files reused in both versions are shown once in Appendix A.

## 7.1 Common Design Components

The following files are common to both the basic and extended processors. They are available in both source folders and are included once in Appendix A.

| Section | Component | File Name | Purpose |
|---|---|---|---|
| 7.1.1 | Add/Subtract Unit | src/add_sub_4bit.vhd | 4-bit signed add/sub unit built using full adders |
| 7.1.2 | Clock Divider / Slow Clock | src/clock_divider.vhd | Divides the 100 MHz board clock for visible execution |
| 7.1.3 | Decoder 3-to-8 | src/decoder_3to8.vhd | Generates one-hot register enable lines |
| 7.1.4 | D Flip-Flop | src/d_ff.vhd | Basic storage element with reset |
| 7.1.5 | Full Adder | src/full_adder.vhd | One-bit full adder used by ripple-carry structures |
| 7.1.6 | 2-way 4-bit Multiplexer | src/mux_2way_4bit.vhd | Selects one of two 4-bit data inputs |
| 7.1.7 | 8-way 4-bit Multiplexer | src/mux_8way_4bit.vhd | Selects one register output |
| 7.1.8 | Register Bank | src/register_bank.vhd | Eight 4-bit register structure with R0 fixed at zero |
| 7.1.9 | 4-bit Register | src/reg_4bit.vhd | Stores one 4-bit register value |
| 7.1.10 | Seven Segment Display / LUT | src/seven_seg_display.vhd | Converts a 4-bit value into seven-segment output |

## 7.2 Basic Nano Processor Unique Components

| Section | Component | File Name | Purpose |
|---|---|---|---|
| 7.2.1 | Basic Top Module | src/nanoprocessor_top.vhd | Connects the original processor datapath |
| 7.2.2 | Basic Instruction Decoder | src/instruction_decoder.vhd | Decodes 12-bit instructions and produces control signals |
| 7.2.3 | Basic Program ROM | src/program_rom.vhd | Stores the 8-instruction program for sum 1 to 3 |
| 7.2.4 | Basic Program Counter | src/program_counter.vhd | 3-bit PC built from D flip-flops |
| 7.2.5 | 3-bit PC Adder | src/adder_3bit.vhd | Computes PC + 1 |
| 7.2.6 | 2-way 3-bit Multiplexer | src/mux_2way_3bit.vhd | Selects PC + 1 or jump address |

## 7.3 Extended Nano Processor Unique Components

| Section | Component | File Name | Purpose |
|---|---|---|---|
| 7.3.1 | Extended Top Module | src/nanoprocessor_top_v2.vhd | Connects the extended datapath |
| 7.3.2 | Extended Instruction Decoder | src/instruction_decoder_v2.vhd | Decodes 14-bit instructions and extended control signals |
| 7.3.3 | Extended Program ROM | src/program_rom_v2.vhd | Stores the 16-instruction extended program |
| 7.3.4 | Extended Program Counter | src/program_counter_v2.vhd | 4-bit PC built from D flip-flops |
| 7.3.5 | 4-bit PC Adder | src/adder_4bit.vhd | Computes PC + 1 for 16 ROM locations |
| 7.3.6 | 4-bit ALU | src/alu_4bit.vhd | Supports arithmetic, logical, multiplication, division, and comparison operations |

# 8. Simulation and Verification

Simulation was used to verify both individual components and the integrated processor. The exact waveform screenshots should be added after running behavioural simulation in Vivado. The placeholders below indicate which waveform or screenshot should be inserted.

## 8.1 Common Component Simulation Results

Waveform images are in the Apendix D.1.

| Component | Testbench File |
|---|---|
| Add/Subtract Unit | sim/Add_Sub_4_TB.vhd |
| Address Selector | sim/Address_Selector_TB.vhd |
| Load Selector | sim/Load_Selector_TB.vhd |
| Register Data Multiplexer | sim/RegisterData_Multiplexer_TB.vhd |
| Register Bank | sim/Register_Bank_TB.vhd |
| Seven Segment Display / LUT | sim/LUT_16_7_TB.vhd |
| Slow Clock | sim/Slow_Clk_TB.vhd |

## 8.2 Basic Nano Processor Simulation Results

Waveform images are in the Apendix D.2.

| Simulation Target | Testbench File | Expected Result |
|---|---|---|
| Full basic processor | sim/NanoProcessor_TB.vhd | Program completes with R7 = 6 |
| Basic instruction decoder | sim/Instruction_Decoder_TB.vhd | Correct control outputs for MOVI, ADD, NEG, and JZR |
| Basic PC adder | sim/PC_Adder_TB.vhd | 3-bit PC increment is correct |
| Basic program counter | sim/Program_Counter_TB.vhd | Reset and PC update work correctly |
| Basic program ROM | sim/Program_ROM_TB.vhd | All 8 ROM instructions match the machine code table |

## 8.3 Extended Nano Processor Simulation Results

Waveform images are in the Apendix D.3.

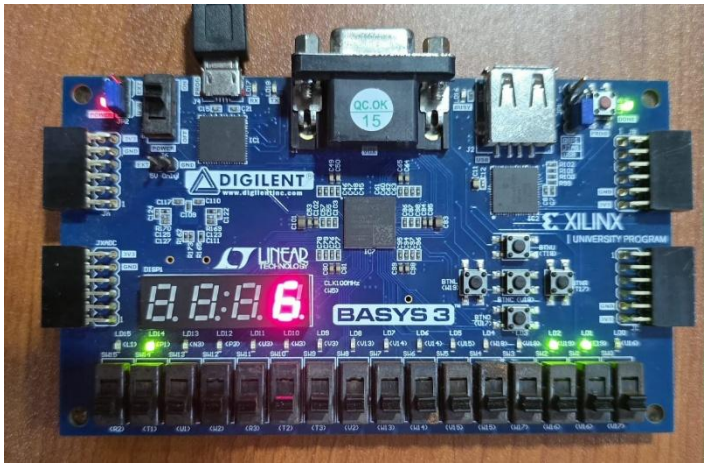
| Simulation Target | Testbench File | Expected Result |
|---|---|---|
| Full extended processor | sim/NanoProcessor_TB.vhd | Program completes with R7 = 9 |
| ALU | sim/ALU_4bit_TB.vhd | ADD, SUB, AND, OR, XOR, NOT, MUL, DIV, and divide-by-zero checks pass |
| Extended instruction decoder | sim/Instruction_Decoder_TB.vhd | Correct control outputs for extended opcodes |
| Extended PC adder | sim/PC_Adder_TB.vhd | 4-bit PC increment is correct |
| Extended program counter | sim/Program_Counter_TB.vhd | Reset and PC update work correctly |
| Extended program ROM | sim/Program_ROM_TB.vhd | All 16 ROM instructions match the machine code table |

# 9. FPGA Implementation and Constraints

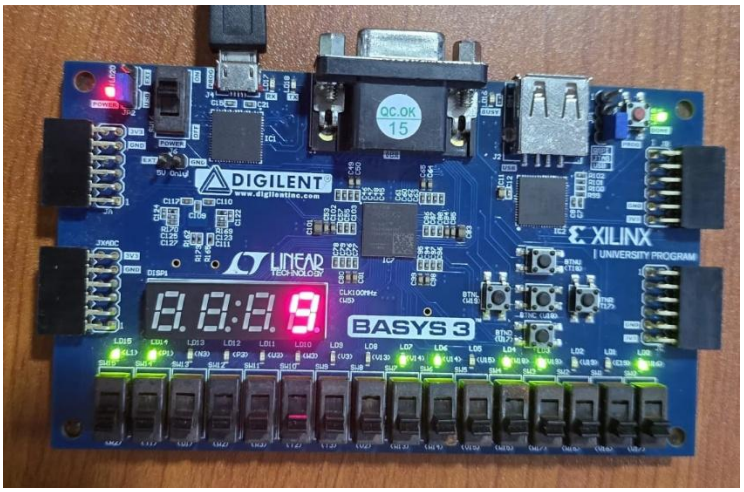
The final design is intended to be implemented on the Basys 3 FPGA board. The 100 MHz board clock is connected to the processor clock input, and the center pushbutton is used as reset. LEDs and the seven-segment display provide visible outputs.

| Signal / Output | Basic Design Mapping | Extended Design Mapping |
|---|---|---|
| Clock | W5 100 MHz clock input | W5 100 MHz clock input |
| Reset | BTNC center pushbutton | BTNC center pushbutton |
| LD0-LD3 | R7 value | R7 value |
| LD4-LD7 | Unused/off | 4-bit PC value |
| LD13 | Unused/off | Negative flag |
| LD14 | Zero flag | Zero flag |
| LD15 | Overflow flag | Jump-active flag |
| Seven-segment AN0 | Displays R7 in hexadecimal | Displays R7 in hexadecimal |

## 9.1 Basic Nano Processor Implementation




## 9.2 Extended Nano Processor Implementation



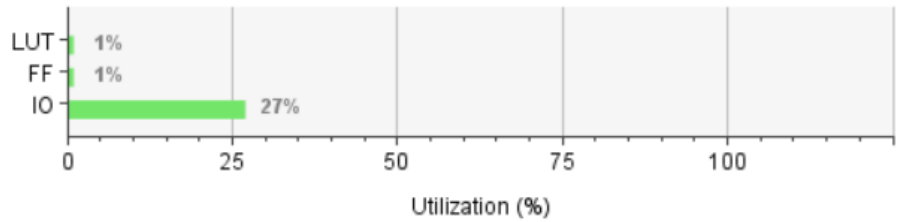
# 10. Resource Utilization

After synthesis and implementation in Vivado, the utilization summary should be inserted into the table below. Values are left blank because the Vivado report screenshots were not included in the uploaded files.

## 10.1 Basic Nano Processor

| Name | Slice LUTs (20800) | Slice Registers (41600) | Bonded IOB (106) | BUFGCTRL (32) |
|---|---|---|---|---|
| nanoprocessor_top | 37 | 39 | 29 | 1 |
| gen_div.cdiv (clock_div... | 21 | 28 | 0 | 0 |
| pc_inst (program_cou... | 6 | 3 | 0 | 0 |
| gen_ff[0].ff_i (d_ff_8) | 2 | 1 | 0 | 0 |
| gen_ff[1].ff_i (d_ff_9) | 0 | 1 | 0 | 0 |
| gen_ff[2].ff_i (d_ff_10) | 4 | 1 | 0 | 0 |
| rb (register_bank) | 10 | 8 | 0 | 0 |
| reg1 (reg_4bit) | 3 | 4 | 0 | 0 |
| gen_ff[0].ff_i (d_ff... | 0 | 1 | 0 | 0 |
| gen_ff[1].ff_i (d_ff... | 2 | 1 | 0 | 0 |
| gen_ff[2].ff_i (d_ff... | 0 | 1 | 0 | 0 |
| gen_ff[3].ff_i (d_ff... | 1 | 1 | 0 | 0 |
| reg7 (reg_4bit_0) | 7 | 4 | 0 | 0 |
| gen_ff[0].ff_i (d_ff) | 2 | 1 | 0 | 0 |
| gen_ff[1].ff_i (d_ff... | 3 | 1 | 0 | 0 |
| gen_ff[2].ff_i (d_ff... | 0 | 1 | 0 | 0 |
| gen_ff[3].ff_i (d_ff... | 2 | 1 | 0 | 0 |

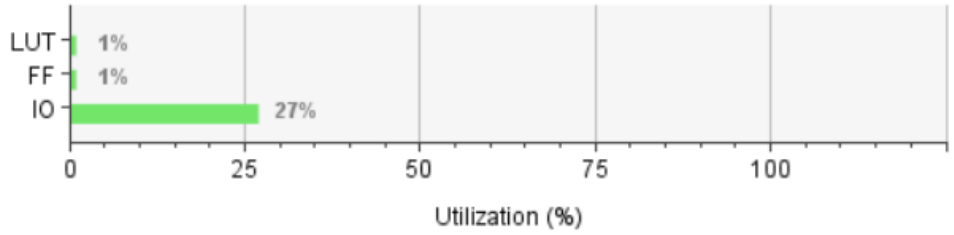
| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 37 | 20800 | 0.18 |
| FF | 39 | 41600 | 0.09 |
| IO | 29 | 106 | 27.36 |



## 10.2 Extended Nano Processor

| Name | Slice LUTs (20800) | Slice Registers (41600) | F7 Muxes (16300) | F8 Muxes (8150) | Bonded IOB (106) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|
| nanoprocessor_top_v2 | 151 | 60 | 26 | 8 | 29 | 2 |
| alu (alu_4bit) | 1 | 0 | 0 | 0 | 0 | 0 |
| gen_div.cdiv (clock_div... | 21 | 28 | 0 | 0 | 0 | 0 |
| pc_inst (program_cou... | 15 | 4 | 3 | 0 | 0 | 0 |
| gen_ff[0].ff_i (d_ff_33) | 6 | 1 | 0 | 0 | 0 | 0 |
| gen_ff[1].ff_i (d_ff_34) | 1 | 1 | 0 | 0 | 0 | 0 |
| gen_ff[2].ff_i (d_ff_35) | 6 | 1 | 0 | 0 | 0 | 0 |
| gen_ff[3].ff_i (d_ff_36) | 2 | 1 | 3 | 0 | 0 | 0 |
| rb (register_bank) | 114 | 28 | 23 | 8 | 0 | 0 |
| reg1 (reg_4bit) | 62 | 4 | 17 | 5 | 0 | 0 |
| reg2 (reg_4bit_0) | 4 | 4 | 0 | 0 | 0 | 0 |
| reg3 (reg_4bit_1) | 18 | 4 | 3 | 1 | 0 | 0 |
| reg4 (reg_4bit_2) | 5 | 4 | 1 | 1 | 0 | 0 |
| reg5 (reg_4bit_3) | 8 | 4 | 1 | 0 | 0 | 0 |
| reg6 (reg_4bit_4) | 1 | 4 | 1 | 1 | 0 | 0 |
| reg7 (reg_4bit_5) | 16 | 4 | 0 | 0 | 0 | 0 |

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 151 | 20800 | 0.73 |
| FF | 60 | 41600 | 0.14 |
| IO | 29 | 106 | 27.36 |



# 11. Discussion

The basic processor satisfies the required lab task by executing a compact four instruction set and producing the final result of 6 in R7. It demonstrates the essential processor concepts: program counter operation, instruction fetch from ROM, instruction decoding, register selection, arithmetic execution, conditional jumping, and output display.

The extended processor improves the basic design by increasing the instruction width to 14 bits and expanding the program counter to 4 bits. This allows 16 ROM instructions and a larger opcode field. The ALU also allows the processor to perform arithmetic, logical, multiplication, division, comparison, and jump related operations. As a result, the extended processor is more flexible and demonstrates a more realistic processor datapath, but it also requires more control logic and is expected to use more FPGA resources than the basic version.

A key design choice was the reuse of common building blocks. The same register bank, multiplexers, full adder, D flip-flop, and seven-segment display decoder are shared by both processor versions. This reduces duplication and makes the design easier to test and maintain.

## 12. Limitations and Future Improvements

- The datapath is only 4 bits wide, so arithmetic results are limited to small values.
- The basic processor has only 8 ROM locations and the extended processor has only 16 ROM locations.
- The program memory is ROM-based, so programs must be changed by editing VHDL and regenerating the bitstream.
- The design does not include data memory or stack support.
- The extended CMP instruction only updates the visible zero flag during its cycle; a future version could add a flag register.
- MUL and DIV are implemented as single-cycle operations; larger processors would need more efficient or multi-cycle designs.
- Future versions could support wider registers, RAM, more branch instructions, and serial or switch-based program/data input.

# 13. Conclusion

The nanoprocessor project successfully demonstrates the design of a simple processor using VHDL. The basic processor implements the required instruction set and runs the assigned program that stores the value 6 in R7. The extended processor further improves the design by adding a larger instruction format, a 4-bit program counter, a 16-location ROM, and a multi-function ALU. The extended program stores 9 in R7 and demonstrates additional arithmetic and logical instructions.

Through this project, the main processor building blocks were designed, connected, and prepared for simulation and FPGA implementation. The design also shows how small reusable digital components such as adders, registers, multiplexers, decoders, and ROM can be integrated into a complete working processor.

## 14. Team Contributions

Complete this table with the actual contribution and time spent by each team member before submission.

| Student Name | Index Number | Contribution | Hours Spent |
|---|---|---|---|
| G.D.J.P.Gamaetige | 240180N | Slow Clock ,Adders ,Decoders,Program ROM,Report | 24 Hours |
| H.M.V.L.Herath | 240225J | ALU,Multiplexers ,Constraint file,Report | 24 Hours |
| I.S.Perera | 240497R | Program Counter ,7 Segment LUT,Register Bank,4 bit Register,Report | 24 Hours |
| K.D.B.M.Perera | 240498V | Instruction Decoders,Top Files ,Report | 24 Hours |

# 15. References

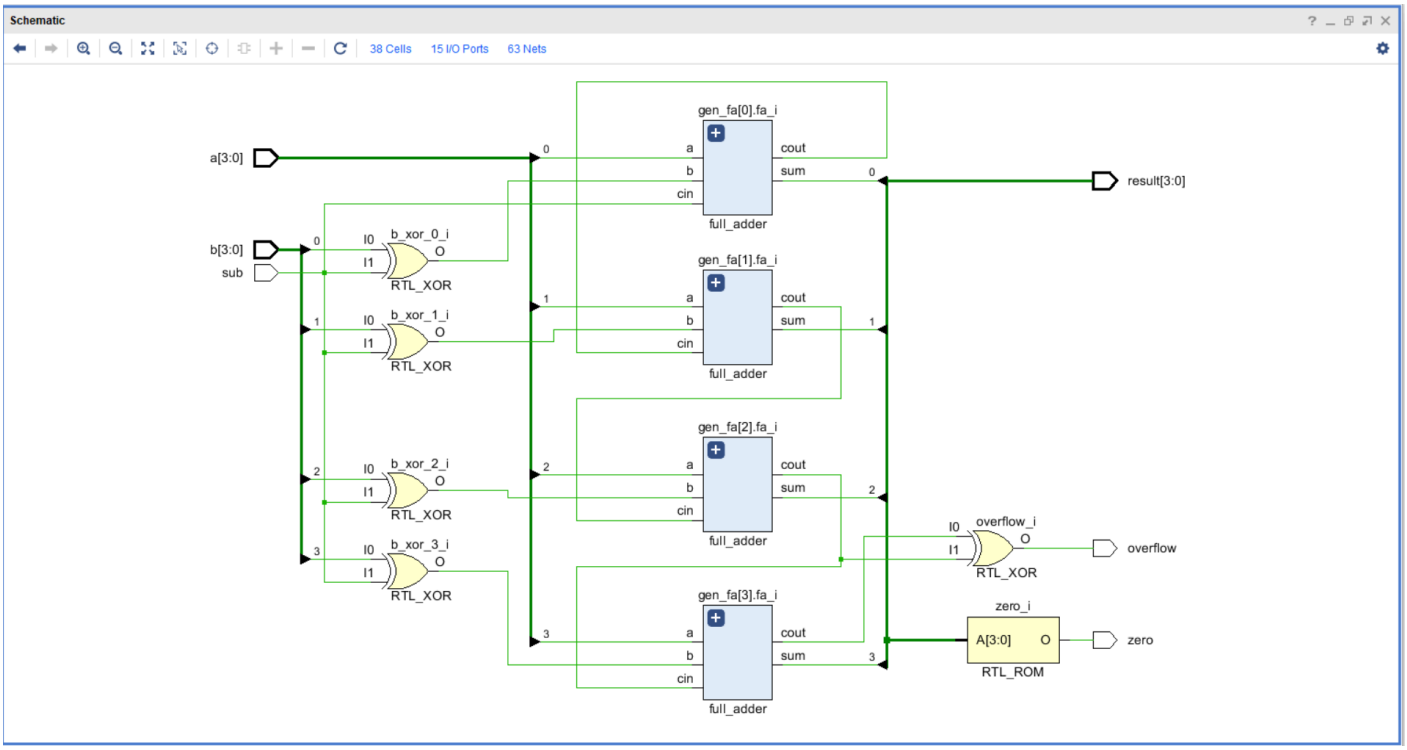

1. CS1050 Computer Organization and Digital Design, Lab 9-10 Nanoprocessor Design Competition lab sheet, Department of Computer Science and Engineering, University of Moratuwa.
2. Basys 3 FPGA board documentation and Xilinx Vivado design flow documentation.

# Appendix A - Common VHDL Codes

The following VHDL files are common to both the basic and extended nanoprocessor source folders. They are included once here to avoid repetition.

## add_sub_4bit.vhd

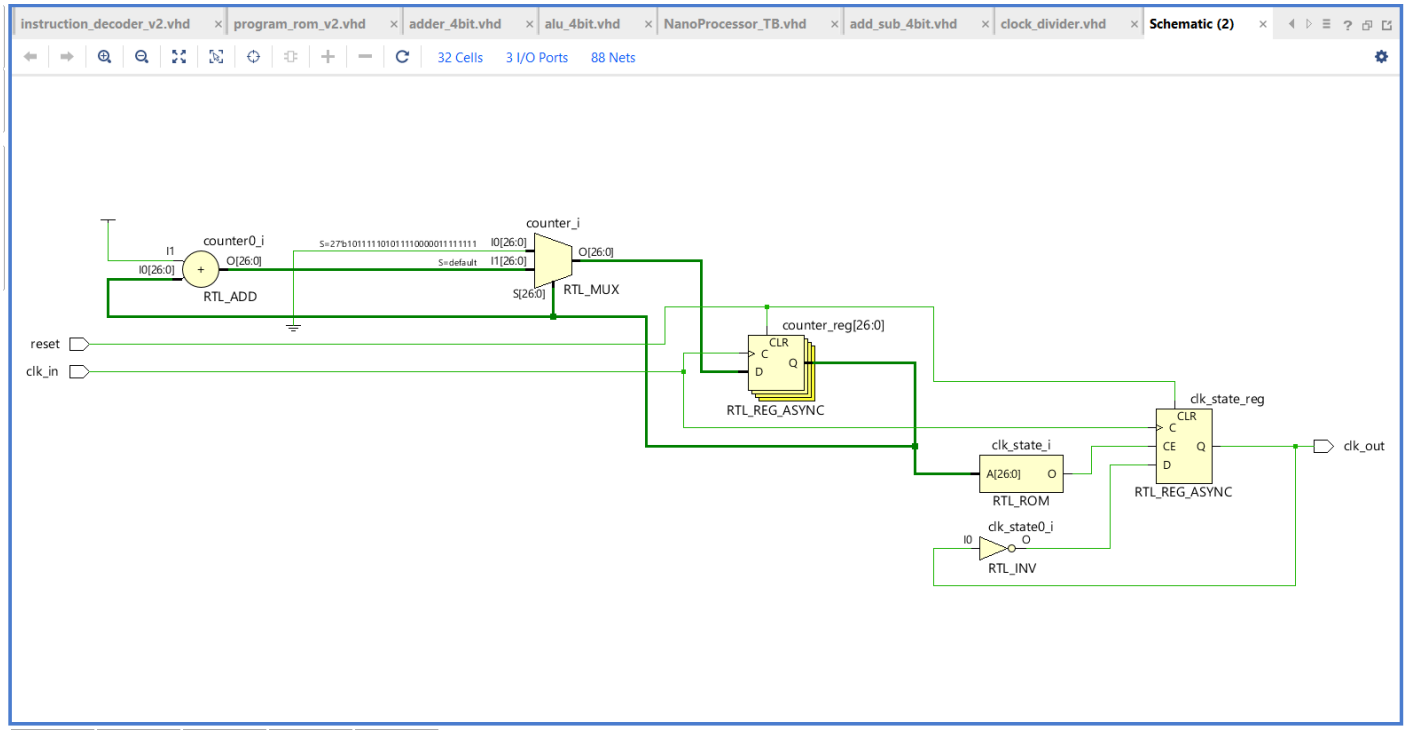
```
library ieee;
use ieee.std_logic_1164.all;

entity add_sub_4bit is
    port (
        a        : in  std_logic_vector(3 downto 0);
        b        : in  std_logic_vector(3 downto 0);
        sub      : in  std_logic;
        result   : out std_logic_vector(3 downto 0);
        overflow : out std_logic;
        zero     : out std_logic
    );
end add_sub_4bit;

architecture structural of add_sub_4bit is
    component full_adder
        port (a, b, cin : in std_logic; sum, cout : out std_logic);
    end component;

    signal b_xor : std_logic_vector(3 downto 0);
    signal carry : std_logic_vector(4 downto 0);
    signal sum   : std_logic_vector(3 downto 0);
begin
    carry(0) <= sub;                        -- cin = 1 for subtract

    gen_fa : for i in 0 to 3 generate
        b_xor(i) <= b(i) xor sub;           -- invert b for subtract
        fa_i : full_adder
            port map (a    => a(i),
                      b    => b_xor(i),
                      cin  => carry(i),
                      sum  => sum(i),
                      cout => carry(i+1));
    end generate;

    result   <= sum;
    overflow <= carry(4) xor carry(3);                -- signed overflow
    zero     <= '1' when sum = "0000" else '0';
end structural;
```




## clock_divider.vhd

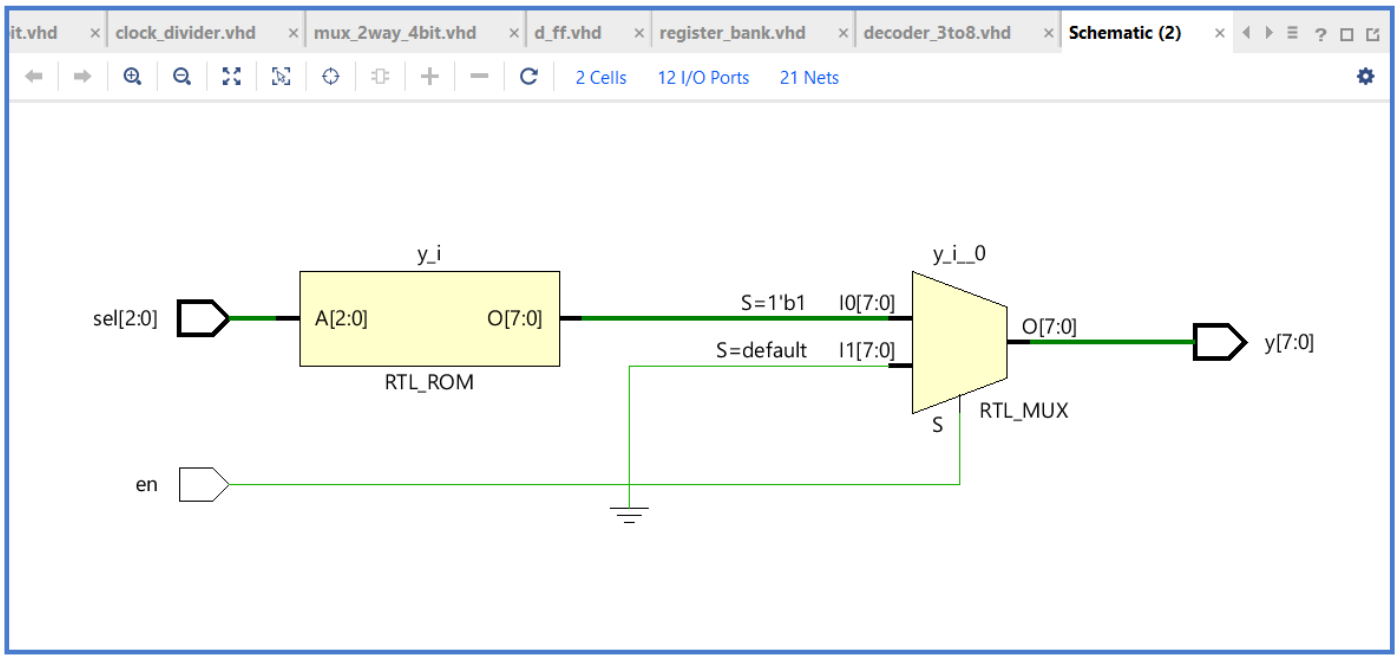
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity clock_divider is
    generic (
        DIVISOR : integer := 100_000_000
    );
    port (
        clk_in  : in  std_logic;
        reset   : in  std_logic;
        clk_out : out std_logic
    );
end clock_divider;

architecture behavioral of clock_divider is
    signal counter   : integer range 0 to DIVISOR-1 := 0;
    signal clk_state : std_logic := '0';
begin
    process(clk_in, reset)
    begin
        if reset = '1' then
            counter   <= 0;
            clk_state <= '0';
        elsif rising_edge(clk_in) then
            if counter = DIVISOR - 1 then
                counter   <= 0;
                clk_state <= not clk_state;
            else
                counter <= counter + 1;
            end if;
        end if;
    end process;

    clk_out <= clk_state;
```

```
end behavioral;
```



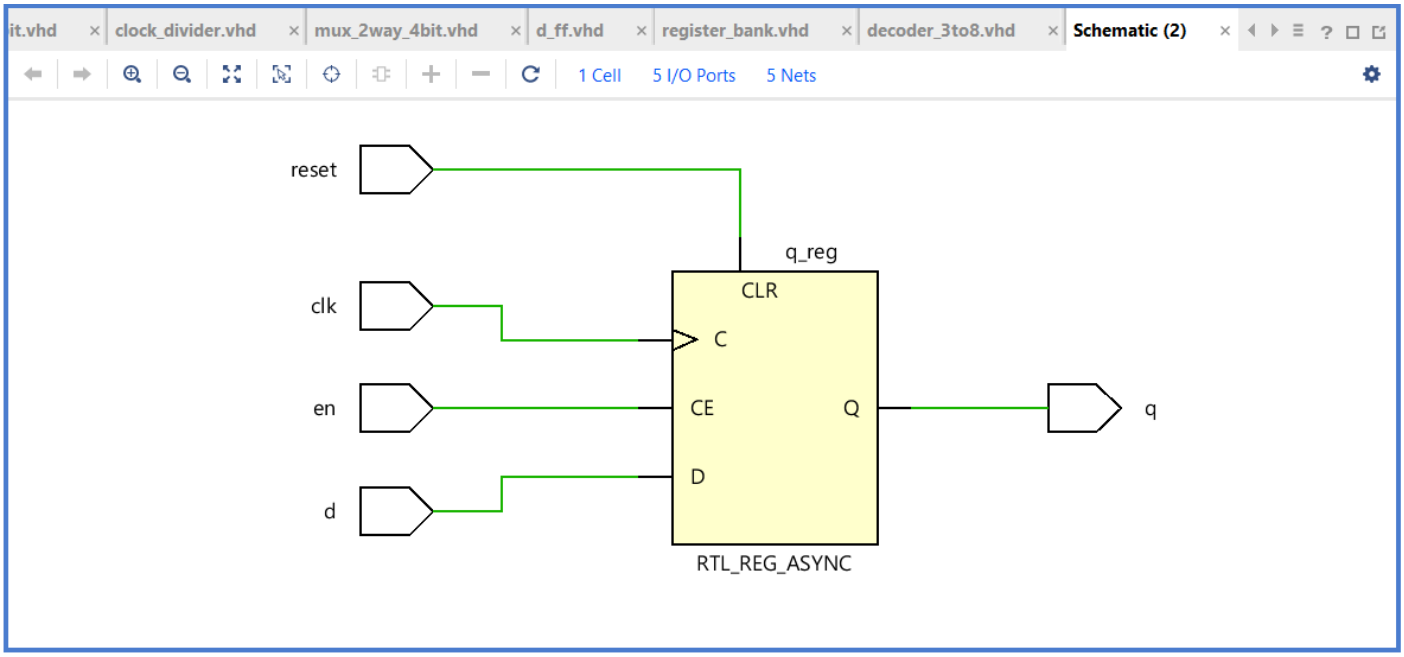

## decoder_3to8.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity decoder_3to8 is
    port (
        sel : in  std_logic_vector(2 downto 0);
        en  : in  std_logic;
        y   : out std_logic_vector(7 downto 0)
    );
end decoder_3to8;

architecture behavioral of decoder_3to8 is
begin
    process(sel, en)
    begin
        y <= (others => '0');
        if en = '1' then
            case sel is
                when "000" => y <= "00000001";
                when "001" => y <= "00000010";
                when "010" => y <= "00000100";
                when "011" => y <= "00001000";
                when "100" => y <= "00010000";
                when "101" => y <= "00100000";
                when "110" => y <= "01000000";
                when "111" => y <= "10000000";
                when others => y <= (others => '0');
            end case;
        end if;
    end process;
end behavioral;
```



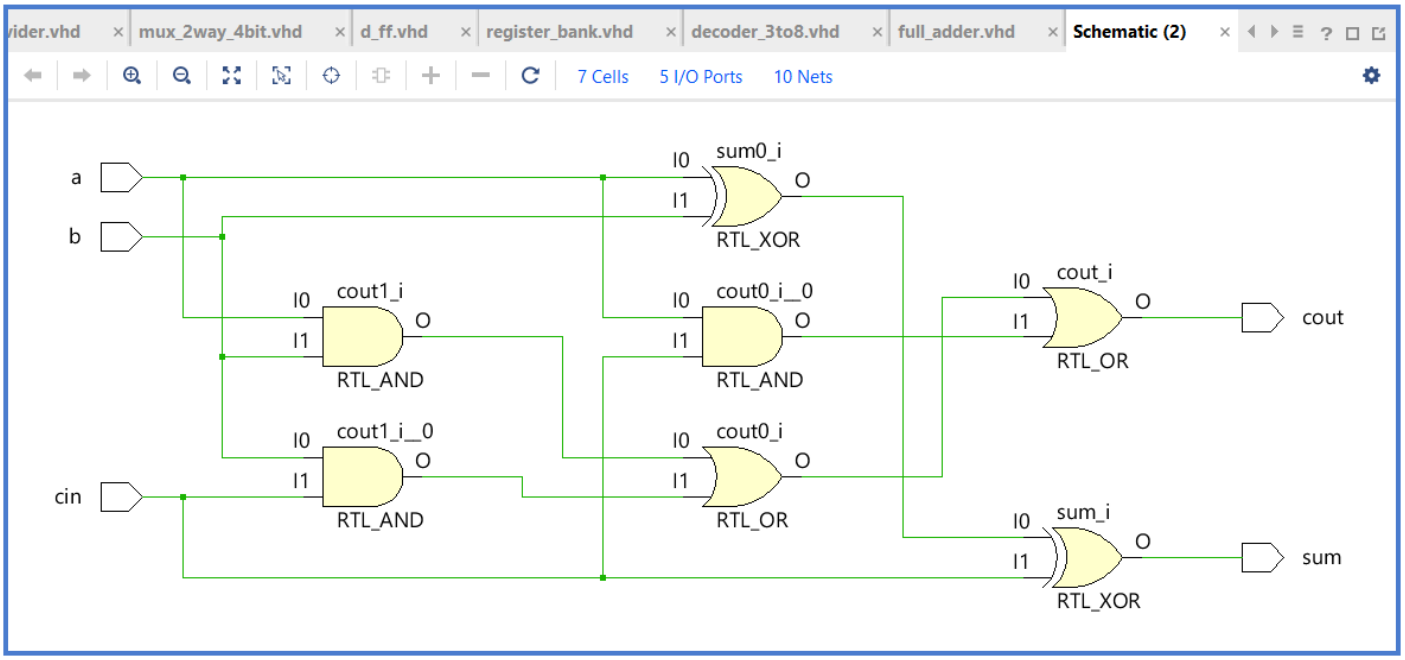

## d_ff.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;       -- async reset (active high)
        en    : in  std_logic;       -- sync enable
        d     : in  std_logic;
        q     : out std_logic
    );
end d_ff;

architecture behavioral of d_ff is
begin
    process(clk, reset)
    begin
        if reset = '1' then
            q <= '0';
        elsif rising_edge(clk) then
            if en = '1' then
                q <= d;
            end if;
        end if;
    end process;
end behavioral;
```



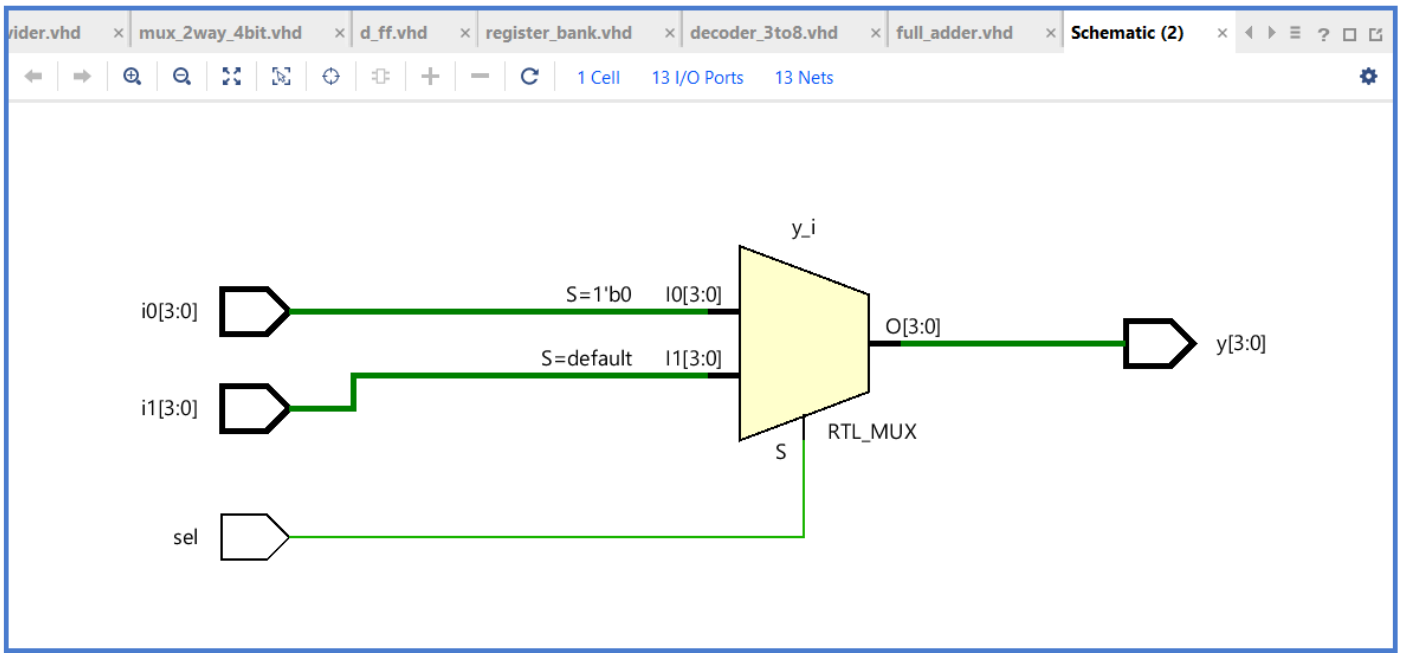

## full_adder.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
    port (
        a    : in  std_logic;
        b    : in  std_logic;
        cin  : in  std_logic;
        sum  : out std_logic;
        cout : out std_logic
    );
end full_adder;

architecture dataflow of full_adder is
begin
    sum  <= a xor b xor cin;
    cout <= (a and b) or (b and cin) or (a and cin);
end dataflow;
```



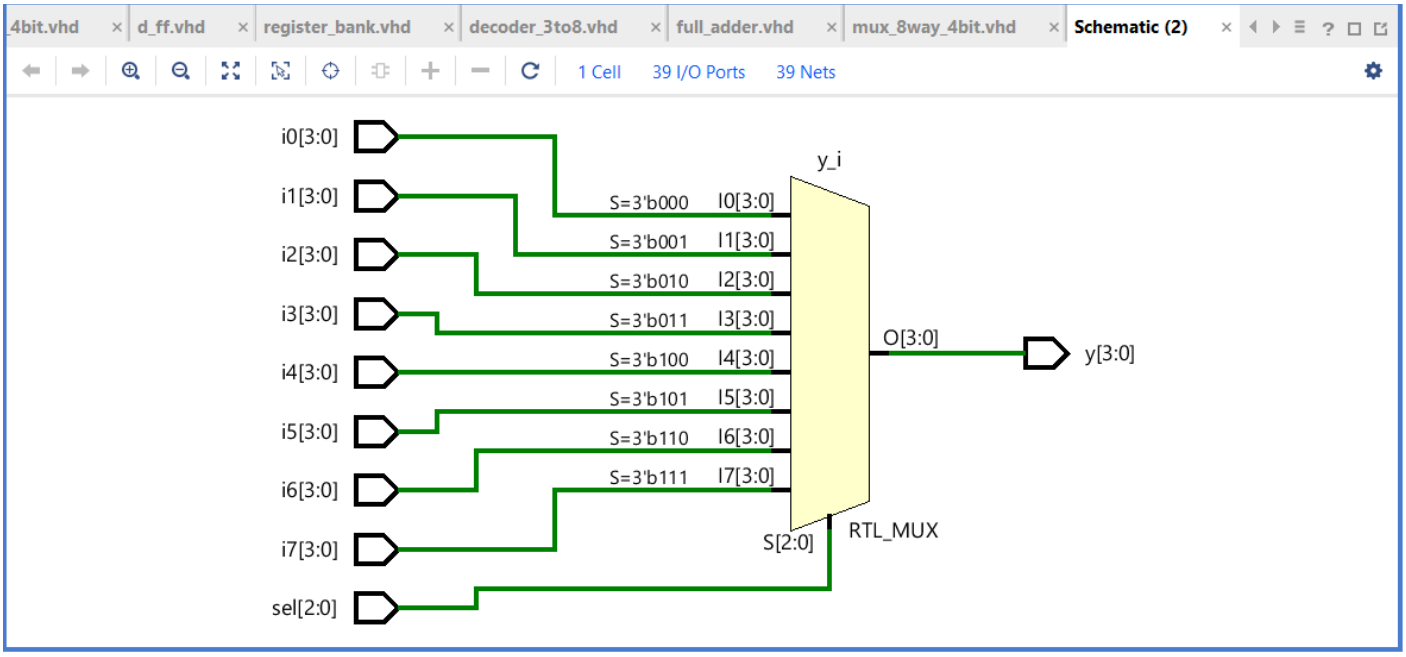

## mux_2way_4bit.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity mux_2way_4bit is
    port (
        sel : in  std_logic;
        i0  : in  std_logic_vector(3 downto 0);
        i1  : in  std_logic_vector(3 downto 0);
        y   : out std_logic_vector(3 downto 0)
    );
end mux_2way_4bit;

architecture dataflow of mux_2way_4bit is
begin
    y <= i0 when sel = '0' else i1;
end dataflow;
```




## mux_8way_4bit.vhd

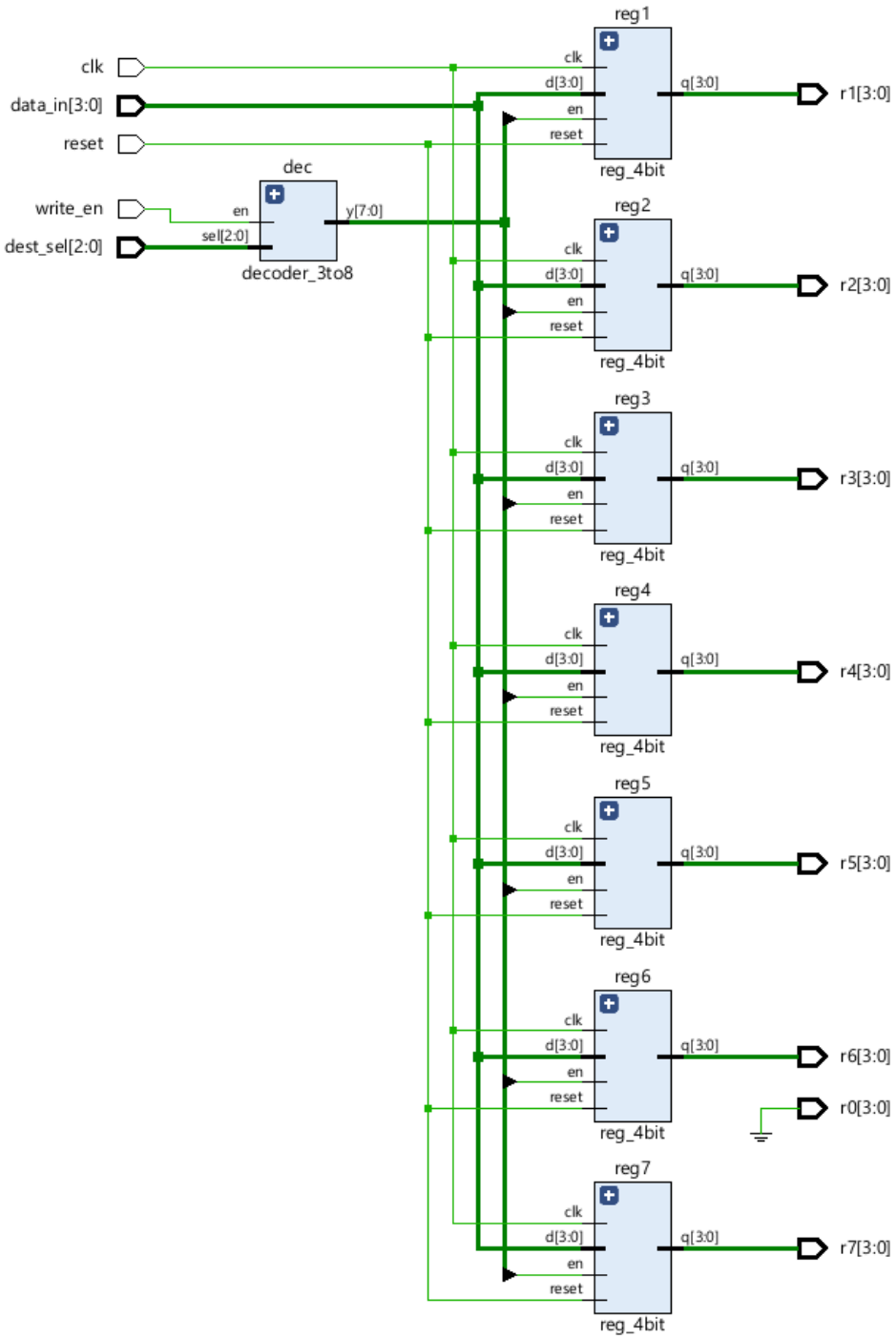
```
library ieee;
use ieee.std_logic_1164.all;

entity mux_8way_4bit is
    port (
        sel : in  std_logic_vector(2 downto 0);
        i0, i1, i2, i3 : in std_logic_vector(3 downto 0);
        i4, i5, i6, i7 : in std_logic_vector(3 downto 0);
        y   : out std_logic_vector(3 downto 0)
    );
end mux_8way_4bit;

architecture behavioral of mux_8way_4bit is
begin
    process(sel, i0, i1, i2, i3, i4, i5, i6, i7)
    begin
        case sel is
            when "000" => y <= i0;
            when "001" => y <= i1;
            when "010" => y <= i2;
            when "011" => y <= i3;
            when "100" => y <= i4;
            when "101" => y <= i5;
            when "110" => y <= i6;
            when "111" => y <= i7;
            when others => y <= (others => '0');
        end case;
    end process;
end behavioral;
```




## register_bank.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity register_bank is
    port (
        clk      : in  std_logic;
        reset    : in  std_logic;
        write_en : in  std_logic;                              -- master write-enable
        dest_sel : in  std_logic_vector(2 downto 0);     -- destination register
        data_in  : in  std_logic_vector(3 downto 0);     -- value to write
        r0, r1, r2, r3 : out std_logic_vector(3 downto 0);
        r4, r5, r6, r7 : out std_logic_vector(3 downto 0)
    );
end register_bank;

architecture structural of register_bank is
    component reg_4bit
        port (clk, reset, en : in std_logic;
              d : in std_logic_vector(3 downto 0);
              q : out std_logic_vector(3 downto 0));
    end component;
    component decoder_3to8
        port (sel : in std_logic_vector(2 downto 0);
              en  : in std_logic;
              y   : out std_logic_vector(7 downto 0));
    end component;

    signal en_lines : std_logic_vector(7 downto 0);
begin
    -- R0 is read-only zero (no flip-flops needed)
    r0 <= "0000";

    -- decoder gates the per-register enables with the master write_en
    dec : decoder_3to8 port map (sel => dest_sel, en => write_en, y =>
en_lines);

    -- en_lines(0) is intentionally unused (R0 is hardwired)
    reg1 : reg_4bit port map (clk, reset, en_lines(1), data_in, r1);
```

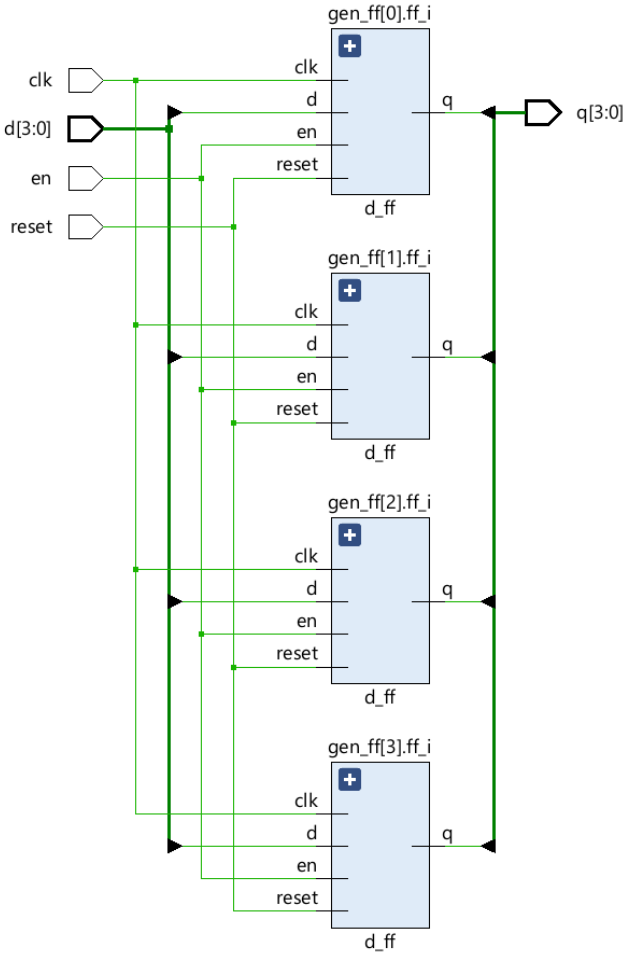
```
    reg2 : reg_4bit port map (clk, reset, en_lines(2), data_in, r2);
    reg3 : reg_4bit port map (clk, reset, en_lines(3), data_in, r3);
    reg4 : reg_4bit port map (clk, reset, en_lines(4), data_in, r4);
    reg5 : reg_4bit port map (clk, reset, en_lines(5), data_in, r5);
    reg6 : reg_4bit port map (clk, reset, en_lines(6), data_in, r6);
    reg7 : reg_4bit port map (clk, reset, en_lines(7), data_in, r7);
end structural;
```




## reg_4bit.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity reg_4bit is
```

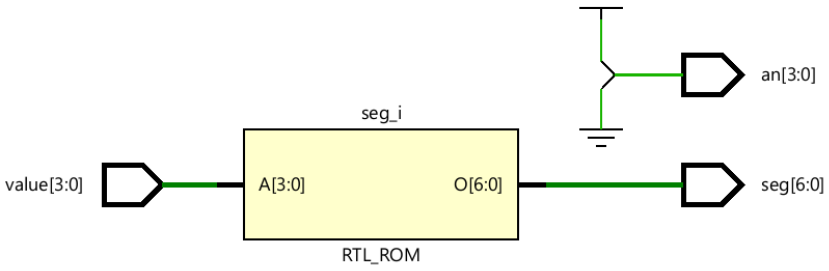
```
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        en    : in  std_logic;
        d     : in  std_logic_vector(3 downto 0);
        q     : out std_logic_vector(3 downto 0)
    );
end reg_4bit;

architecture structural of reg_4bit is
    component d_ff
        port (clk, reset, en : in std_logic; d : in std_logic; q : out
std_logic);
    end component;
begin
    gen_ff : for i in 0 to 3 generate
        ff_i : d_ff port map (clk => clk, reset => reset, en => en,
                              d => d(i), q => q(i));
    end generate;
end structural;
```




## seven_seg_display.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seven_seg_display is
    port (
        value : in  std_logic_vector(3 downto 0);
```

```
        seg   : out std_logic_vector(6 downto 0); -- a (MSB) .. g (LSB)
        an    : out std_logic_vector(3 downto 0)  -- digit anodes (active low)
    );
end seven_seg_display;

architecture behavioral of seven_seg_display is
begin
    -- Only enable rightmost digit (AN0). Others off.
    an <= "1110";

    process(value)
    begin
        case value is
            when "0000" => seg <= "1000000";  -- 0
            when "0001" => seg <= "1111001";  -- 1
            when "0010" => seg <= "0100100";  -- 2
            when "0011" => seg <= "0110000";  -- 3
            when "0100" => seg <= "0011001";  -- 4
            when "0101" => seg <= "0010010";  -- 5
            when "0110" => seg <= "0000010";  -- 6
            when "0111" => seg <= "1111000";  -- 7
            when "1000" => seg <= "0000000";  -- 8
            when "1001" => seg <= "0010000";  -- 9
            when "1010" => seg <= "0001000";  -- a
            when "1011" => seg <= "0000011";  -- b
            when "1100" => seg <= "1000110";  -- c
            when "1101" => seg <= "0100001";  -- d
            when "1110" => seg <= "0000110";  -- e
            when "1111" => seg <= "0001110";  -- f
            when others => seg <= "1111111";  -- blank
        end case;
    end process;
end behavioral;
```



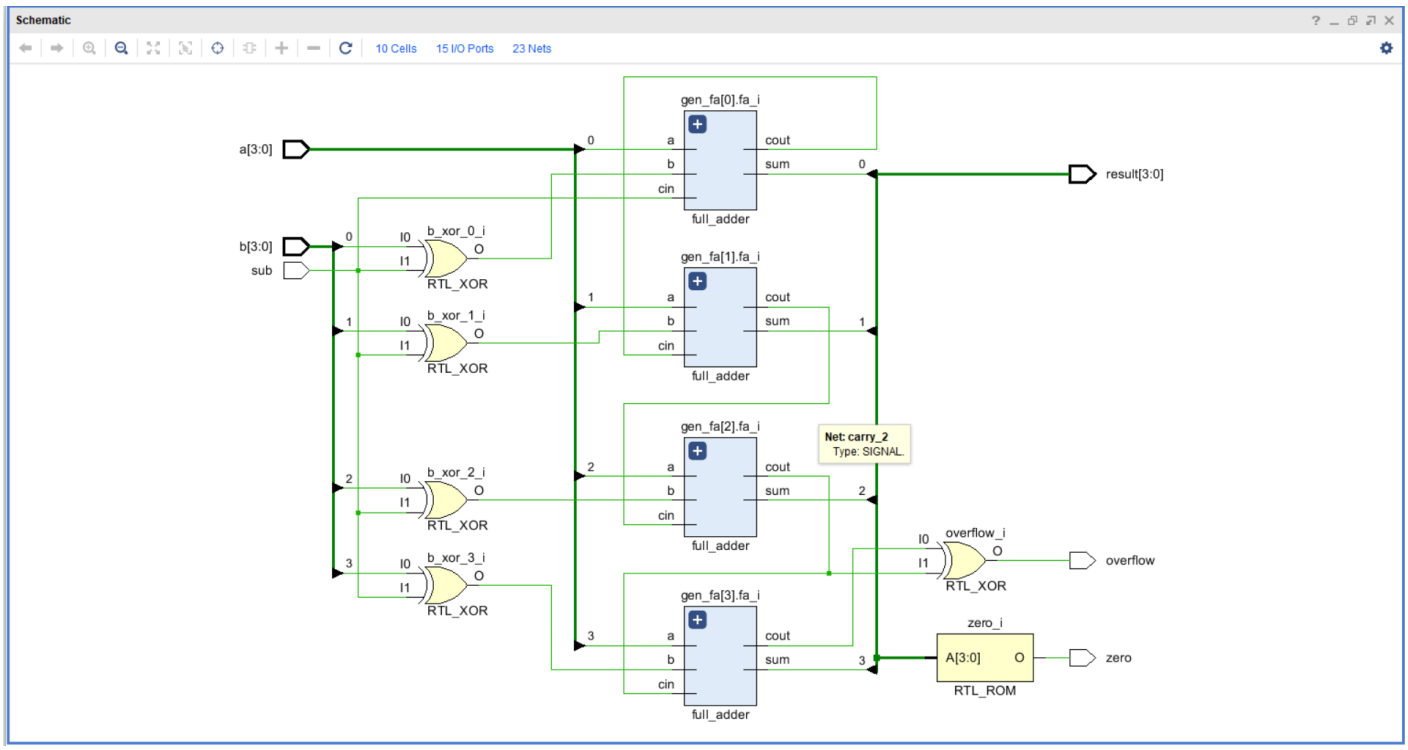
# Appendix B - Basic Nano Processor Unique VHDL Codes

## nanoprocessor_top.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity nanoprocessor_top is
    generic (
        SIM_MODE : boolean := false;
        DIVISOR  : integer := 100_000_000
    );
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        led   : out std_logic_vector(15 downto 0);
        seg   : out std_logic_vector(6 downto 0);
        an    : out std_logic_vector(3 downto 0)
    );
end nanoprocessor_top;

architecture structural of nanoprocessor_top is
    -- Component declarations
    component clock_divider
        generic (DIVISOR : integer);
        port (clk_in, reset : in std_logic; clk_out : out std_logic);
    end component;
    component program_counter
        port (clk, reset : in std_logic;
              d : in std_logic_vector(2 downto 0);
              q : out std_logic_vector(2 downto 0));
    end component;
    component adder_3bit
        port (a, b : in std_logic_vector(2 downto 0); cin : in std_logic;
              sum : out std_logic_vector(2 downto 0); cout : out std_logic);
    end component;
    component mux_2way_3bit
        port (sel : in std_logic;
              i0, i1 : in std_logic_vector(2 downto 0);
              y : out std_logic_vector(2 downto 0));
    end component;
    component program_rom
        port (addr : in std_logic_vector(2 downto 0);
              data : out std_logic_vector(11 downto 0));
    end component;
    component instruction_decoder
        port (instruction  : in std_logic_vector(11 downto 0);
              mux_a_sel    : out std_logic_vector(2 downto 0);
              mux_b_sel    : out std_logic_vector(2 downto 0);
              addsub_sel   : out std_logic;
              data_src_sel : out std_logic;
              imm_value    : out std_logic_vector(3 downto 0);
              dest_reg     : out std_logic_vector(2 downto 0);
              reg_write_en : out std_logic;
              jump_addr    : out std_logic_vector(2 downto 0);
              is_jzr       : out std_logic);
    end component;
    component register_bank
        port (clk, reset, write_en : in std_logic;
              dest_sel : in std_logic_vector(2 downto 0);
              data_in  : in std_logic_vector(3 downto 0);
```

```
            r0, r1, r2, r3 : out std_logic_vector(3 downto 0);
            r4, r5, r6, r7 : out std_logic_vector(3 downto 0));
    end component;
    component mux_8way_4bit
        port (sel : in std_logic_vector(2 downto 0);
              i0, i1, i2, i3 : in std_logic_vector(3 downto 0);
              i4, i5, i6, i7 : in std_logic_vector(3 downto 0);
              y : out std_logic_vector(3 downto 0));
    end component;
    component add_sub_4bit
        port (a, b : in std_logic_vector(3 downto 0);
              sub  : in std_logic;
              result   : out std_logic_vector(3 downto 0);
              overflow, zero : out std_logic);
    end component;
    component mux_2way_4bit
        port (sel : in std_logic;
              i0, i1 : in std_logic_vector(3 downto 0);
              y : out std_logic_vector(3 downto 0));
    end component;
    component seven_seg_display
        port (value : in std_logic_vector(3 downto 0);
              seg : out std_logic_vector(6 downto 0);
              an  : out std_logic_vector(3 downto 0));
    end component;

    -- Internal signals
    signal slow_clk        : std_logic;
    signal pc, pc_plus_1   : std_logic_vector(2 downto 0);
    signal pc_next, jump_addr_sig : std_logic_vector(2 downto 0);
    signal pc_carry        : std_logic;
    signal instruction     : std_logic_vector(11 downto 0);

    signal mux_a_sel_sig, mux_b_sel_sig, dest_reg_sig : std_logic_vector(2
downto 0);
    signal addsub_sel_sig, data_src_sel_sig          : std_logic;
    signal reg_write_en_sig, is_jzr_sig, jump_flag   : std_logic;
    signal imm_value_sig                              : std_logic_vector(3
downto 0);

    signal r0, r1, r2, r3, r4, r5, r6, r7            : std_logic_vector(3
downto 0);
    signal mux_a_out, mux_b_out                       : std_logic_vector(3
downto 0);
    signal alu_out, data_bus                          : std_logic_vector(3
downto 0);
    signal alu_overflow, alu_zero                     : std_logic;
begin
    gen_div : if not SIM_MODE generate
        cdiv : clock_divider
            generic map (DIVISOR => DIVISOR)
            port map (clk_in => clk, reset => reset, clk_out => slow_clk);
    end generate;
    gen_sim : if SIM_MODE generate
        slow_clk <= clk;     -- bypass divider in simulation
    end generate;

    pc_inst : program_counter
        port map (clk => slow_clk, reset => reset, d => pc_next, q => pc);

    pc_adder : adder_3bit
        port map (a => pc, b => "000", cin => '1',
                  sum => pc_plus_1, cout => pc_carry);
```

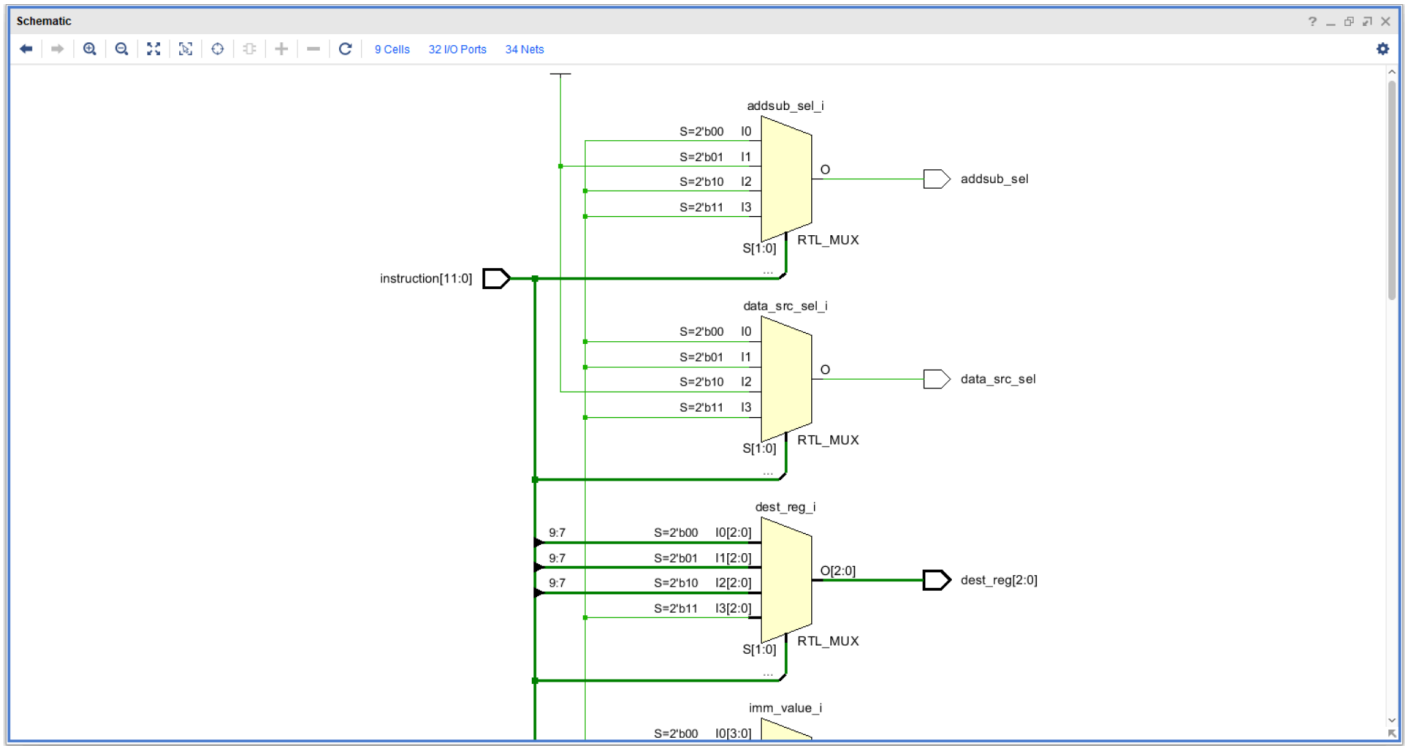
```
    pc_mux : mux_2way_3bit
        port map (sel => jump_flag, i0 => pc_plus_1, i1 => jump_addr_sig,
                  y => pc_next);

    rom : program_rom
        port map (addr => pc, data => instruction);

    dec : instruction_decoder
        port map (instruction  => instruction,
                  mux_a_sel    => mux_a_sel_sig,
                  mux_b_sel    => mux_b_sel_sig,
                  addsub_sel   => addsub_sel_sig,
                  data_src_sel => data_src_sel_sig,
                  imm_value    => imm_value_sig,
                  dest_reg     => dest_reg_sig,
                  reg_write_en => reg_write_en_sig,
                  jump_addr    => jump_addr_sig,
                  is_jzr       => is_jzr_sig);

    -- Final jump signal: only jump on JZR when ALU zero flag is set
    jump_flag <= is_jzr_sig and alu_zero;

    rb : register_bank
        port map (clk => slow_clk, reset => reset,
                  write_en => reg_write_en_sig,
                  dest_sel => dest_reg_sig,
                  data_in  => data_bus,
                  r0 => r0, r1 => r1, r2 => r2, r3 => r3,
                  r4 => r4, r5 => r5, r6 => r6, r7 => r7);

    mux_a : mux_8way_4bit
        port map (sel => mux_a_sel_sig,
                  i0 => r0, i1 => r1, i2 => r2, i3 => r3,
                  i4 => r4, i5 => r5, i6 => r6, i7 => r7,
                  y => mux_a_out);

    mux_b : mux_8way_4bit
        port map (sel => mux_b_sel_sig,
                  i0 => r0, i1 => r1, i2 => r2, i3 => r3,
                  i4 => r4, i5 => r5, i6 => r6, i7 => r7,
                  y => mux_b_out);

    alu : add_sub_4bit
        port map (a => mux_a_out, b => mux_b_out, sub => addsub_sel_sig,
                  result => alu_out,
                  overflow => alu_overflow, zero => alu_zero);

    data_mux : mux_2way_4bit
        port map (sel => data_src_sel_sig, i0 => alu_out, i1 => imm_value_sig,
                  y => data_bus);

    seg_disp : seven_seg_display
        port map (value => r7, seg => seg, an => an);

    led(3 downto 0)   <= r7;             -- LD0..LD3 show R7
    led(13 downto 4)  <= (others => '0');
    led(14)           <= alu_zero;       -- LD14 = zero flag
    led(15)           <= alu_overflow;   -- LD15 = overflow / carry flag
end structural;
```

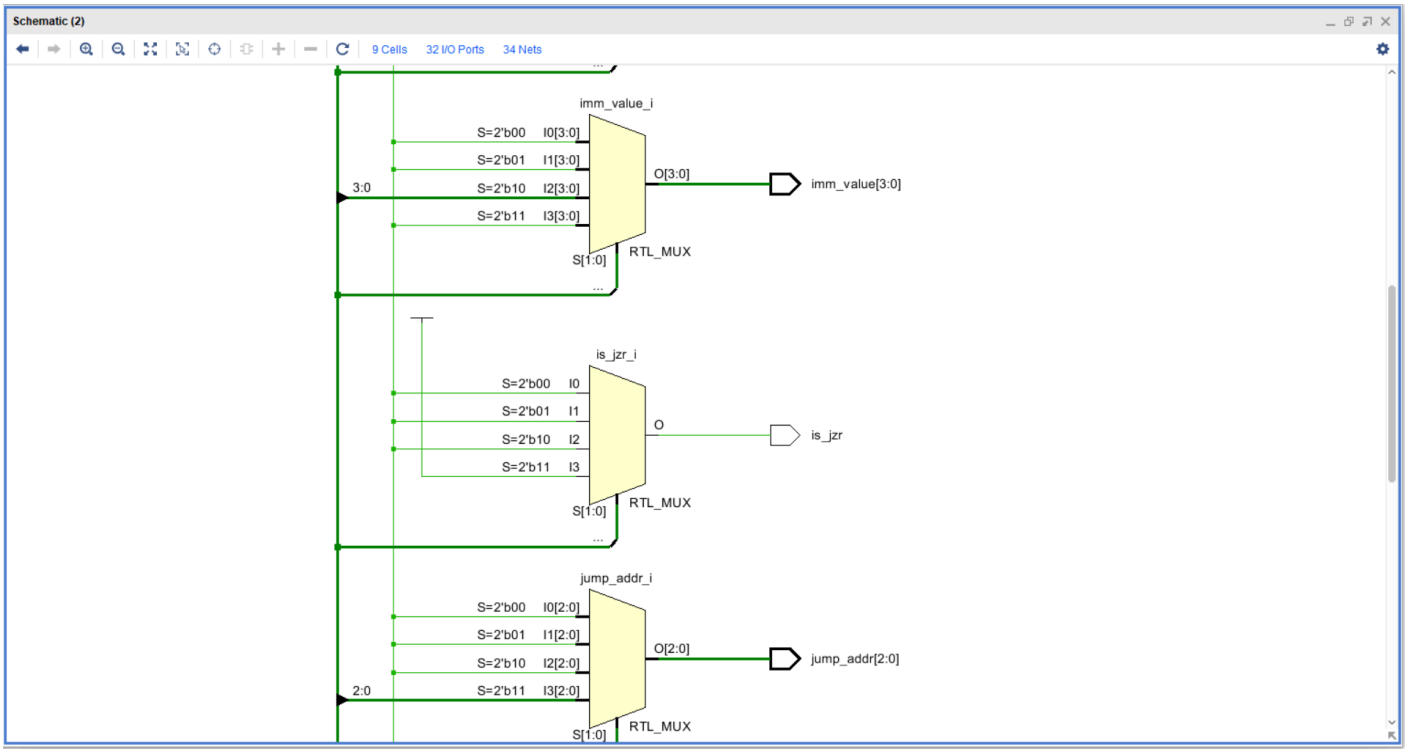


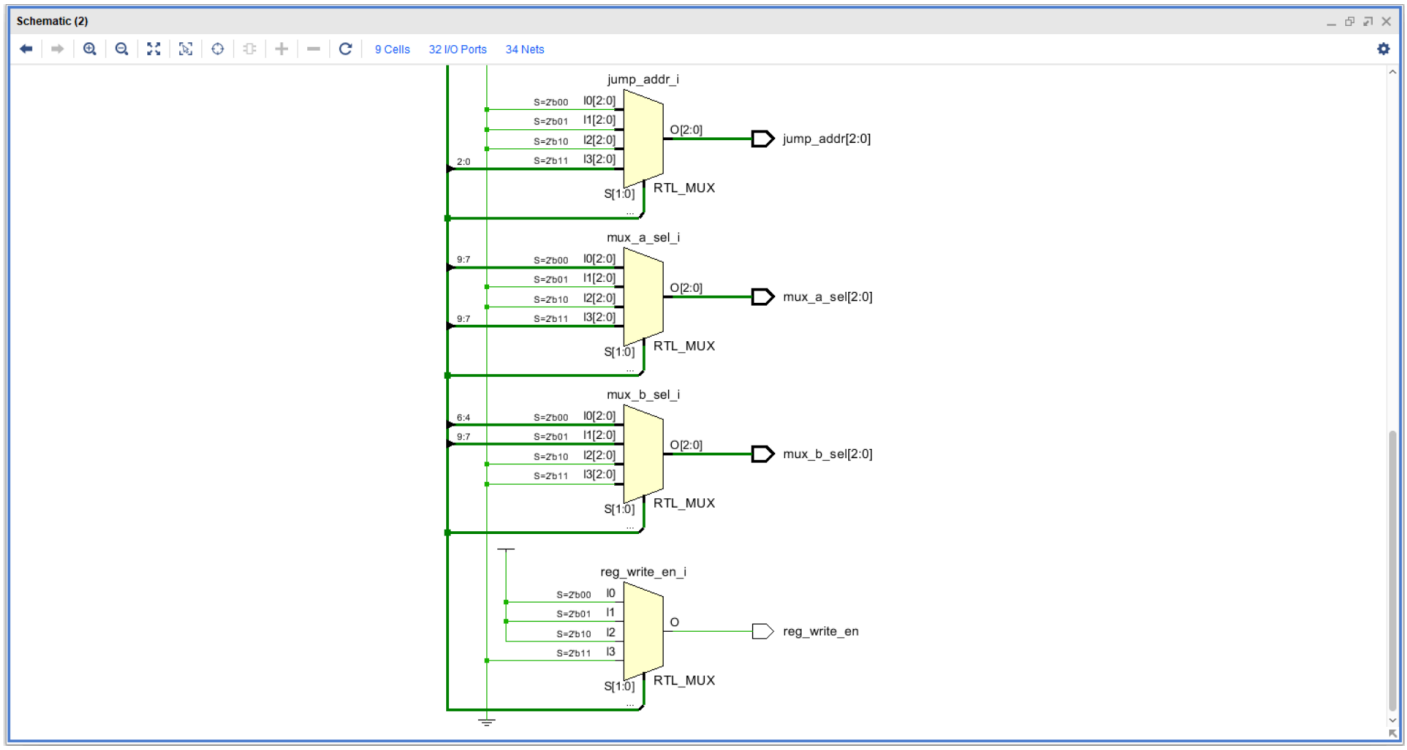

## instruction_decoder.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity instruction_decoder is
    port (
        instruction  : in  std_logic_vector(11 downto 0);
        mux_a_sel    : out std_logic_vector(2 downto 0);
        mux_b_sel    : out std_logic_vector(2 downto 0);
        addsub_sel   : out std_logic;
        data_src_sel : out std_logic;
        imm_value    : out std_logic_vector(3 downto 0);
        dest_reg     : out std_logic_vector(2 downto 0);
        reg_write_en : out std_logic;
        jump_addr    : out std_logic_vector(2 downto 0);
        is_jzr       : out std_logic
    );
end instruction_decoder;

architecture behavioral of instruction_decoder is
    signal opcode : std_logic_vector(1 downto 0);
begin
    opcode <= instruction(11 downto 10);

    process(instruction, opcode)
    begin
        -- safe defaults
        mux_a_sel    <= "000";
        mux_b_sel    <= "000";
        addsub_sel   <= '0';
        data_src_sel <= '0';
        imm_value    <= "0000";
        dest_reg     <= "000";
        reg_write_en <= '0';
        jump_addr    <= "000";
```

```
        is_jzr       <= '0';

        case opcode is
            -- ADD Ra, Rb : Ra <= Ra + Rb
            when "00" =>
                mux_a_sel    <= instruction(9 downto 7);  -- Ra
                mux_b_sel    <= instruction(6 downto 4);  -- Rb
                addsub_sel   <= '0';                      -- add
                data_src_sel <= '0';                      -- write ALU result
                dest_reg     <= instruction(9 downto 7);  -- target Ra
                reg_write_en <= '1';

            -- NEG R : R <= 0 - R   (R0 is hardwired zero)
            when "01" =>
                mux_a_sel    <= "000";                    -- R0 = 0
                mux_b_sel    <= instruction(9 downto 7);  -- R
                addsub_sel   <= '1';                      -- subtract
                data_src_sel <= '0';                      -- ALU result
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- MOVI R, d : R <= d
            when "10" =>
                data_src_sel <= '1';                      -- pass immediate
                imm_value    <= instruction(3 downto 0);
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- JZR R, d : if R = 0 then PC <= d else PC <= PC + 1
            when "11" =>
                mux_a_sel    <= instruction(9 downto 7);  -- R
                mux_b_sel    <= "000";                    -- + 0
                addsub_sel   <= '0';                      -- ALU sees R + 0 = R
                reg_write_en <= '0';                      -- no register write
                jump_addr    <= instruction(2 downto 0);
                is_jzr       <= '1';

            when others =>
                null;
        end case;
    end process;
end behavioral;
```


Schematic
9 Cells
32 I/O Ports
34 Nets
addsub_sel_i
S=2'b00 I0
S=2'b01 I1
S=2'b10 I2
S=2'b11 I3
O
addsub_sel
S[1:0]
RTL_MUX
instruction[11:0]
data_src_sel_i
S=2'b00 I0
S=2'b01 I1
S=2'b10 I2
S=2'b11 I3
O
data_src_sel
S[1:0]
RTL_MUX
dest_reg_i
9:7
S=2'b00 I0[2:0]
9:7
S=2'b01 I1[2:0]
9:7
S=2'b10 I2[2:0]
S=2'b11 I3[2:0]
O[2:0]
dest_reg[2:0]
S[1:0]
RTL_MUX
imm_value_i
S=2'b00 I0[3:0]

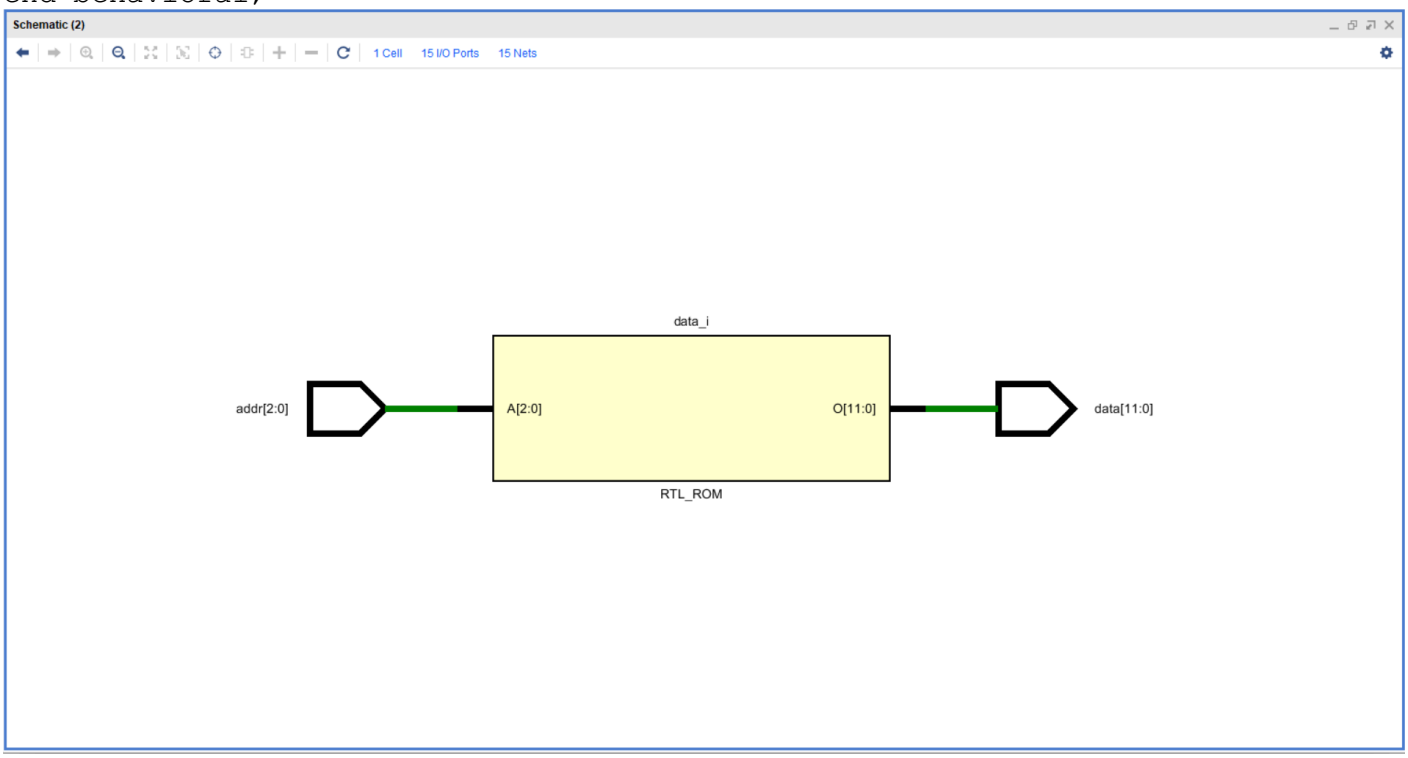



## program_rom.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity program_rom is
    port (
        addr : in  std_logic_vector(2 downto 0);
        data : out std_logic_vector(11 downto 0)
    );
end program_rom;
```

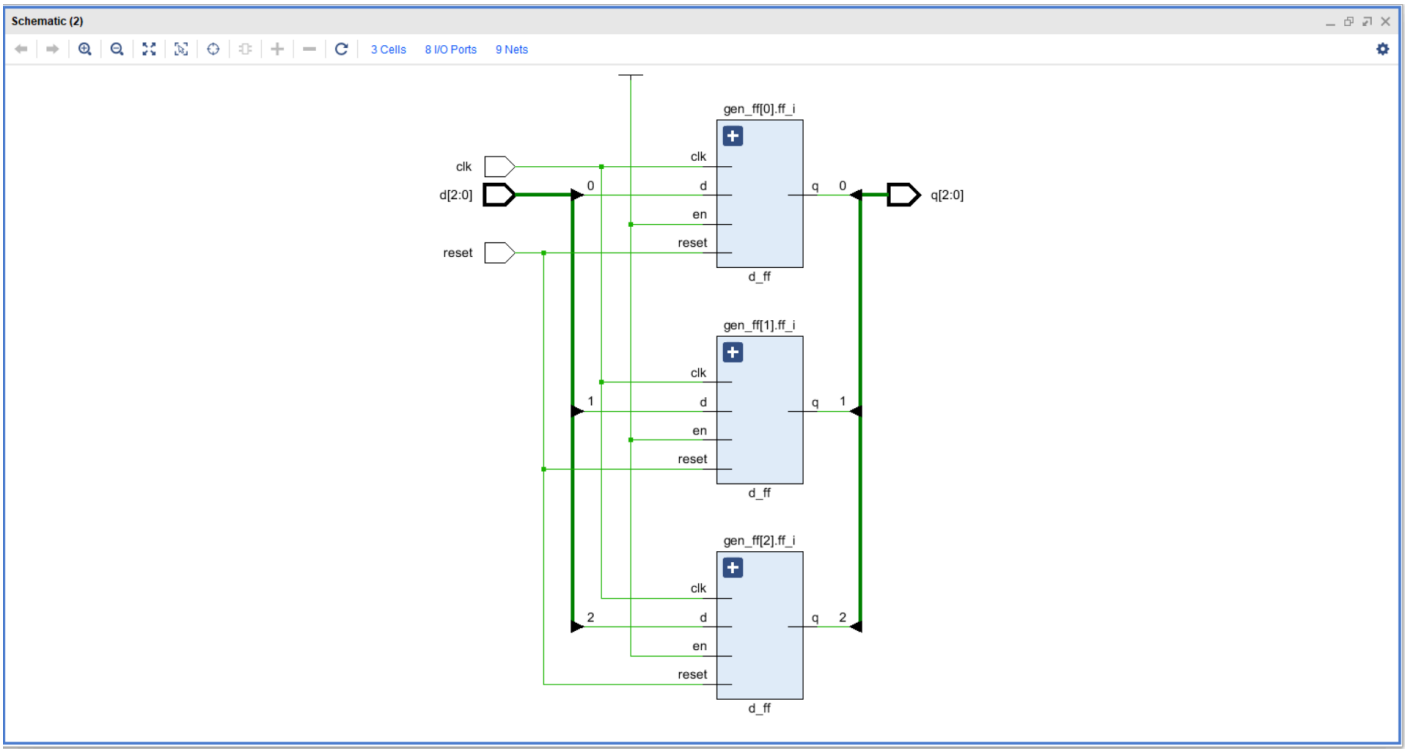
```
architecture behavioral of program_rom is
begin
    process(addr)
    begin
        case addr is
            when "000" => data <= x"881";  -- MOVI R1, 1       (load 1)
            when "001" => data <= x"390";  -- ADD  R7, R1      (R7 = 0+1 = 1)
            when "010" => data <= x"882";  -- MOVI R1, 2       (load 2)
            when "011" => data <= x"390";  -- ADD  R7, R1      (R7 = 1+2 = 3)
            when "100" => data <= x"883";  -- MOVI R1, 3       (load 3)
            when "101" => data <= x"390";  -- ADD  R7, R1      (R7 = 3+3 = 6)
            when "110" => data <= x"C06";  -- JZR  R0, 6       (halt loop)
            when "111" => data <= x"C07";  -- JZR  R0, 7       (safety halt))
            when others => data <= (others => '0');
        end case;
    end process;
end behavioral;
```




## program_counter.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity program_counter is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        d     : in  std_logic_vector(2 downto 0);
        q     : out std_logic_vector(2 downto 0)
    );
end program_counter;

architecture structural of program_counter is
    component d_ff
        port (clk, reset, en : in std_logic; d : in std_logic; q : out
std_logic);
    end component;
begin
```

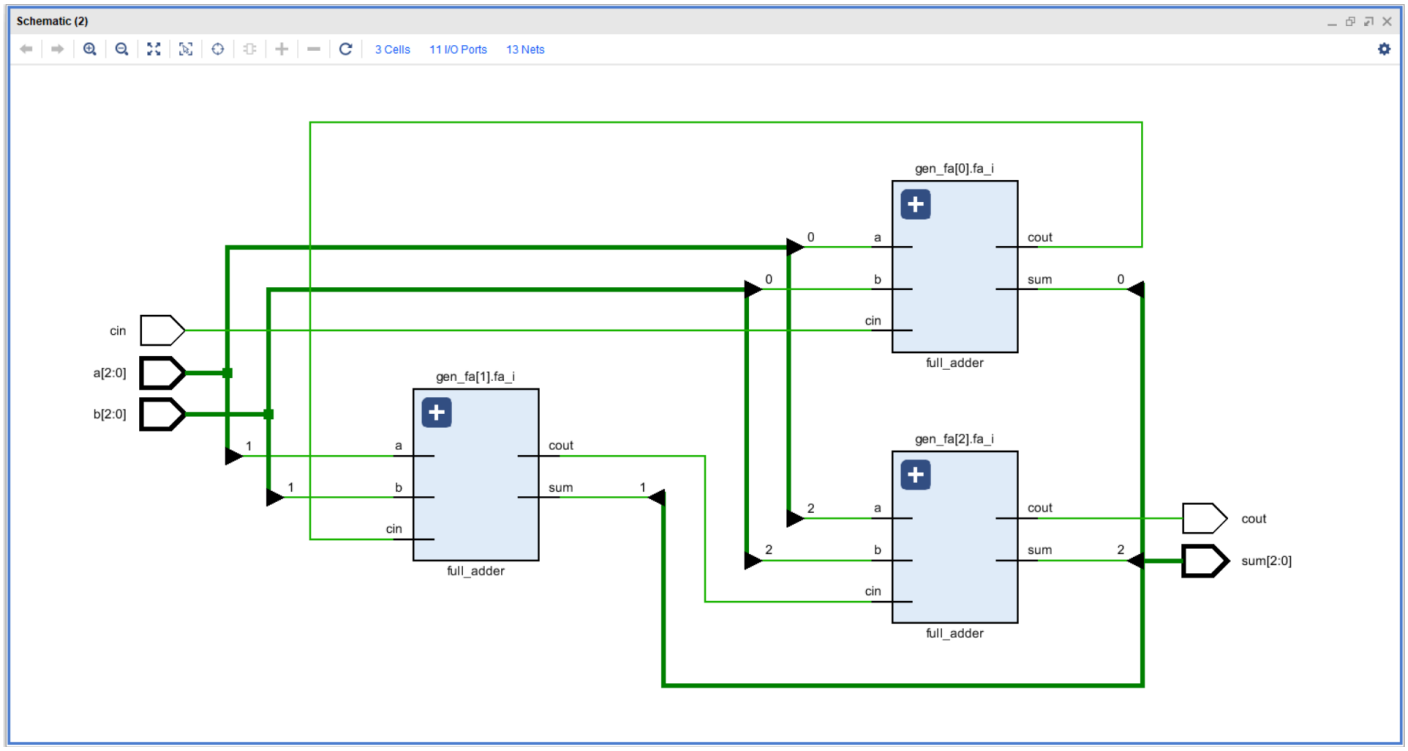
```
    gen_ff : for i in 0 to 2 generate
        ff_i : d_ff port map (clk => clk, reset => reset, en => '1',
                              d => d(i), q => q(i));
    end generate;
end structural;
```




## adder_3bit.vhd

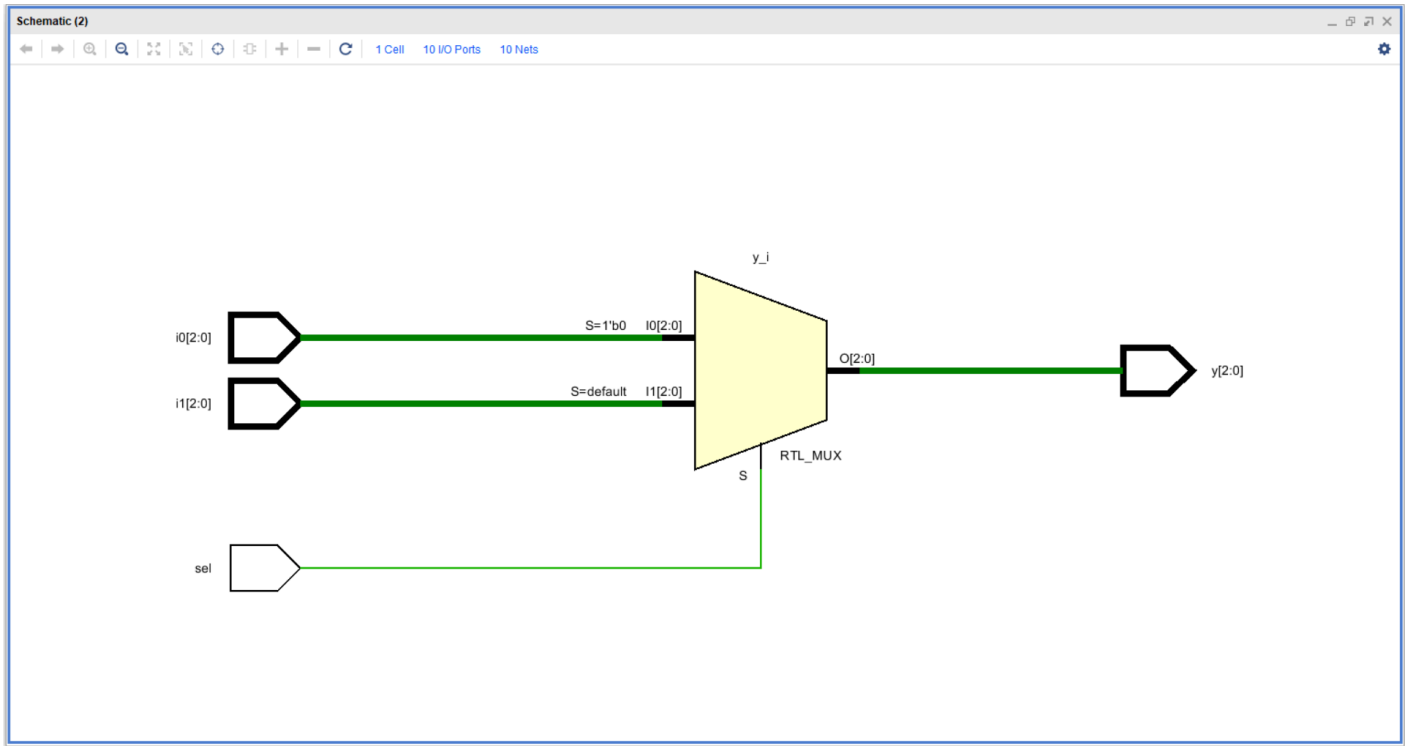
```
library ieee;
use ieee.std_logic_1164.all;

entity adder_3bit is
    port (
        a    : in  std_logic_vector(2 downto 0);
        b    : in  std_logic_vector(2 downto 0);
        cin  : in  std_logic;
        sum  : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end adder_3bit;

architecture structural of adder_3bit is
    component full_adder
        port (a, b, cin : in std_logic; sum, cout : out std_logic);
    end component;

    signal carry : std_logic_vector(3 downto 0);
begin
    carry(0) <= cin;

    gen_fa : for i in 0 to 2 generate
        fa_i : full_adder
            port map (a    => a(i),
                      b    => b(i),
                      cin  => carry(i),
                      sum  => sum(i),
                      cout => carry(i+1));
    end generate;
```

```
    cout <= carry(3);
end structural;
```




## mux_2way_3bit.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity mux_2way_3bit is
    port (
        sel : in  std_logic;
        i0  : in  std_logic_vector(2 downto 0);
        i1  : in  std_logic_vector(2 downto 0);
        y   : out std_logic_vector(2 downto 0)
    );
end mux_2way_3bit;

architecture dataflow of mux_2way_3bit is
begin
    y <= i0 when sel = '0' else i1;
end dataflow;
```


Schematic (2)
1 Cell
10 I/O Ports
10 Nets
y_i
i0[2:0]
i1[2:0]
S=1'b0
I0[2:0]
S=default
I1[2:0]
O[2:0]
y[2:0]
RTL_MUX
S
sel

# Appendix C - Extended Nano Processor Unique VHDL Codes

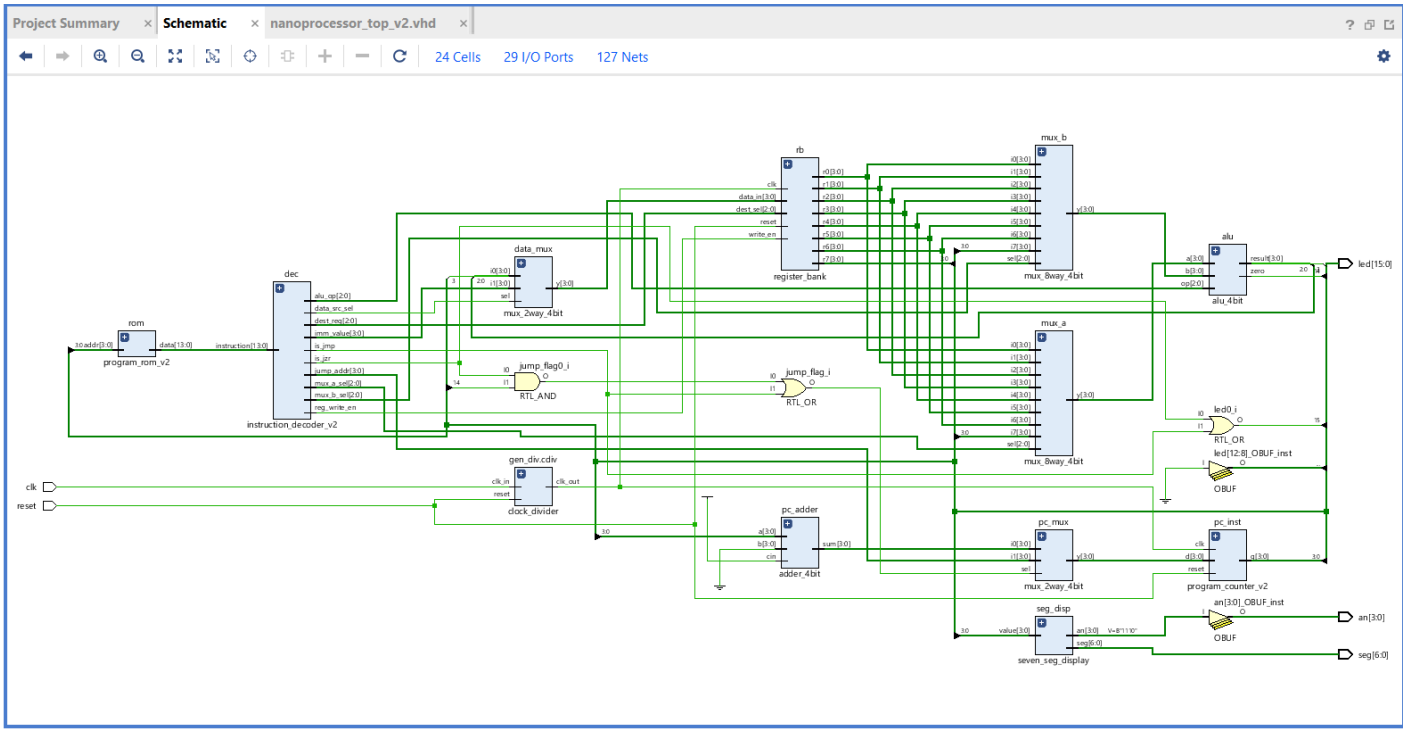
## nanoprocessor_top_v2.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity nanoprocessor_top_v2 is
    generic (
        SIM_MODE : boolean := false;
        DIVISOR  : integer := 100_000_000
    );
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        led   : out std_logic_vector(15 downto 0);
        seg   : out std_logic_vector(6 downto 0);
        an    : out std_logic_vector(3 downto 0)
    );
end nanoprocessor_top_v2;

architecture structural of nanoprocessor_top_v2 is

    component clock_divider
        generic (DIVISOR : integer);
        port (clk_in, reset : in  std_logic;
              clk_out       : out std_logic);
    end component;

    component program_counter_v2
        port (clk, reset : in  std_logic;
              d          : in  std_logic_vector(3 downto 0);
              q          : out std_logic_vector(3 downto 0));
    end component;

    component adder_4bit
        port (a, b : in  std_logic_vector(3 downto 0);
              cin  : in  std_logic;
              sum  : out std_logic_vector(3 downto 0);
              cout : out std_logic);
    end component;

    component mux_2way_4bit
        port (sel    : in  std_logic;
              i0, i1 : in  std_logic_vector(3 downto 0);
              y      : out std_logic_vector(3 downto 0));
    end component;

    component program_rom_v2
        port (addr : in  std_logic_vector(3 downto 0);
              data : out std_logic_vector(13 downto 0));
    end component;

    component instruction_decoder_v2
        port (instruction  : in  std_logic_vector(13 downto 0);
              mux_a_sel    : out std_logic_vector(2 downto 0);
              mux_b_sel    : out std_logic_vector(2 downto 0);
              alu_op       : out std_logic_vector(2 downto 0);
              data_src_sel : out std_logic;
              imm_value    : out std_logic_vector(3 downto 0);
              dest_reg     : out std_logic_vector(2 downto 0);
              reg_write_en : out std_logic;
              jump_addr    : out std_logic_vector(3 downto 0);
              is_jzr       : out std_logic;
```

```
            is_jmp       : out std_logic);
    end component;

    component register_bank
        port (clk, reset, write_en : in  std_logic;
              dest_sel             : in  std_logic_vector(2 downto 0);
              data_in              : in  std_logic_vector(3 downto 0);
              r0, r1, r2, r3, r4, r5, r6, r7 : out std_logic_vector(3 downto
0));
    end component;

    component mux_8way_4bit
        port (sel             : in  std_logic_vector(2 downto 0);
              i0, i1, i2, i3,
              i4, i5, i6, i7  : in  std_logic_vector(3 downto 0);
              y               : out std_logic_vector(3 downto 0));
    end component;

    component alu_4bit
        port (a, b     : in  std_logic_vector(3 downto 0);
              op       : in  std_logic_vector(2 downto 0);
              result   : out std_logic_vector(3 downto 0);
              zero     : out std_logic;
              overflow : out std_logic);
    end component;

    component seven_seg_display
        port (value : in  std_logic_vector(3 downto 0);
              seg   : out std_logic_vector(6 downto 0);
              an    : out std_logic_vector(3 downto 0));
    end component;

    signal slow_clk        : std_logic;

    signal pc, pc_plus_1   : std_logic_vector(3 downto 0);
    signal pc_next         : std_logic_vector(3 downto 0);
    signal jump_addr_sig   : std_logic_vector(3 downto 0);
    signal pc_carry        : std_logic;

    signal instruction     : std_logic_vector(13 downto 0);

    signal mux_a_sel_sig, mux_b_sel_sig, dest_reg_sig : std_logic_vector(2
downto 0);
    signal alu_op_sig                                 : std_logic_vector(2
downto 0);
    signal data_src_sel_sig                           : std_logic;
    signal reg_write_en_sig                           : std_logic;
    signal is_jzr_sig, is_jmp_sig                     : std_logic;
    signal jump_flag                                  : std_logic;
    signal imm_value_sig                              : std_logic_vector(3
downto 0);

    signal r0, r1, r2, r3, r4, r5, r6, r7 : std_logic_vector(3 downto 0);
    signal mux_a_out, mux_b_out           : std_logic_vector(3 downto 0);
    signal alu_out, data_bus              : std_logic_vector(3 downto 0);
    signal alu_zero, alu_overflow         : std_logic;
    signal neg_flag                       : std_logic;  -- MSB of ALU result
(2's-complement sign bit)

begin
    gen_div : if not SIM_MODE generate
        cdiv : clock_divider
            generic map (DIVISOR => DIVISOR)
            port map (clk_in => clk, reset => reset, clk_out => slow_clk);
    end generate;
```

```
gen_sim : if SIM_MODE generate
    slow_clk <= clk;
end generate;

pc_inst : program_counter_v2
    port map (clk => slow_clk, reset => reset, d => pc_next, q => pc);

pc_adder : adder_4bit
    port map (a    => pc,
              b    => "0000",
              cin  => '1',          -- PC + 1
              sum  => pc_plus_1,
              cout => pc_carry);

pc_mux : mux_2way_4bit
    port map (sel => jump_flag,
              i0  => pc_plus_1,
              i1  => jump_addr_sig,
              y   => pc_next);

rom : program_rom_v2
    port map (addr => pc, data => instruction);

dec : instruction_decoder_v2
    port map (instruction  => instruction,
              mux_a_sel    => mux_a_sel_sig,
              mux_b_sel    => mux_b_sel_sig,
              alu_op       => alu_op_sig,
              data_src_sel => data_src_sel_sig,
              imm_value    => imm_value_sig,
              dest_reg     => dest_reg_sig,
              reg_write_en => reg_write_en_sig,
              jump_addr    => jump_addr_sig,
              is_jzr       => is_jzr_sig,
              is_jmp       => is_jmp_sig);

-- Final jump signal:
--   JZR jumps if ALU zero flag is set,
--   JMP jumps unconditionally.
jump_flag <= (is_jzr_sig and alu_zero) or is_jmp_sig;

rb : register_bank
    port map (clk => slow_clk, reset => reset,
              write_en => reg_write_en_sig,
              dest_sel => dest_reg_sig,
              data_in  => data_bus,
              r0 => r0, r1 => r1, r2 => r2, r3 => r3,
              r4 => r4, r5 => r5, r6 => r6, r7 => r7);

mux_a : mux_8way_4bit
    port map (sel => mux_a_sel_sig,
              i0 => r0, i1 => r1, i2 => r2, i3 => r3,
              i4 => r4, i5 => r5, i6 => r6, i7 => r7,
              y  => mux_a_out);

mux_b : mux_8way_4bit
    port map (sel => mux_b_sel_sig,
              i0 => r0, i1 => r1, i2 => r2, i3 => r3,
              i4 => r4, i5 => r5, i6 => r6, i7 => r7,
              y  => mux_b_out);

alu : alu_4bit
    port map (a        => mux_a_out,
              b        => mux_b_out,
              op       => alu_op_sig,
```

```
                result   => alu_out,
                zero     => alu_zero,
                overflow => alu_overflow);

    data_mux : mux_2way_4bit
        port map (sel => data_src_sel_sig,
                  i0  => alu_out,
                  i1  => imm_value_sig,
                  y   => data_bus);

    seg_disp : seven_seg_display
        port map (value => r7, seg => seg, an => an);

    led(3  downto 0)  <= r7;                          -- LD0..3   = R7
    led(7  downto 4)  <= pc;                          -- LD4..7   = PC
    neg_flag          <= alu_out(3);
    led(12 downto 8)  <= (others => '0');
    led(13)           <= neg_flag;                    -- LD13     = neg flag
(result MSB)
    led(14)           <= alu_zero;                    -- LD14     = zero
    led(15)           <= is_jzr_sig or is_jmp_sig;    -- LD15     = jump-active

end structural;
```



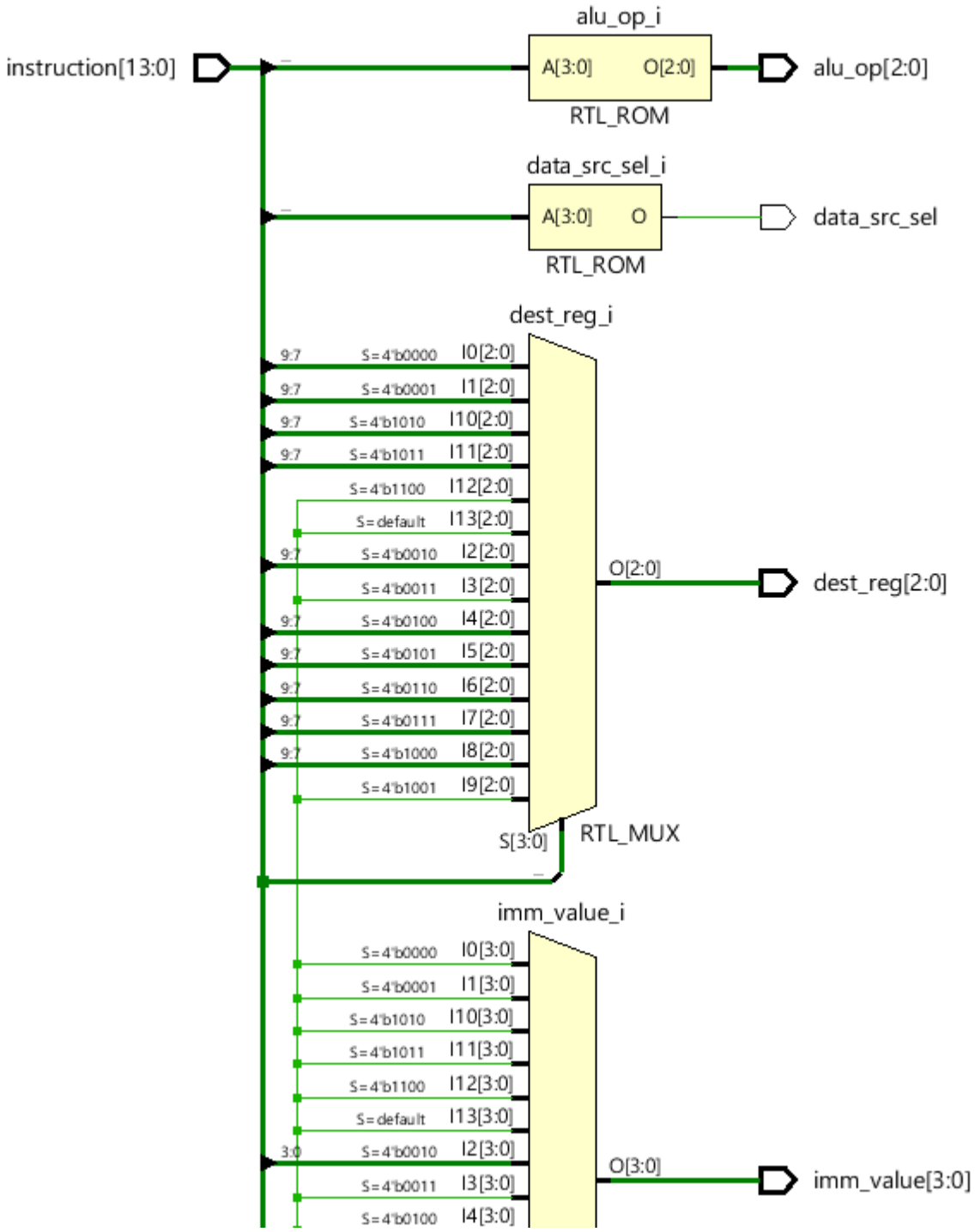

## instruction_decoder_v2.vhd

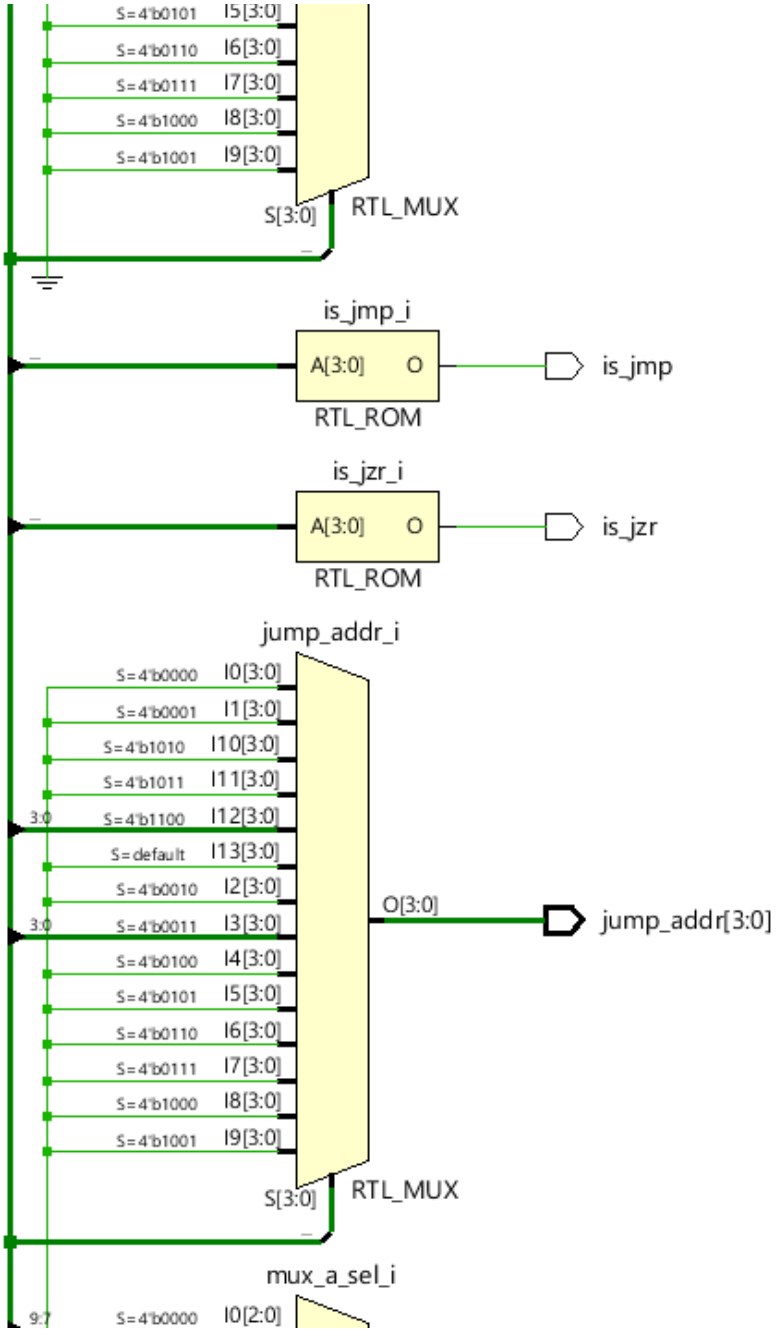
```
library ieee;
use ieee.std_logic_1164.all;

entity instruction_decoder_v2 is
    port (
        instruction  : in  std_logic_vector(13 downto 0);
        mux_a_sel    : out std_logic_vector(2 downto 0);
        mux_b_sel    : out std_logic_vector(2 downto 0);
        alu_op       : out std_logic_vector(2 downto 0);
        data_src_sel : out std_logic;
        imm_value    : out std_logic_vector(3 downto 0);
```

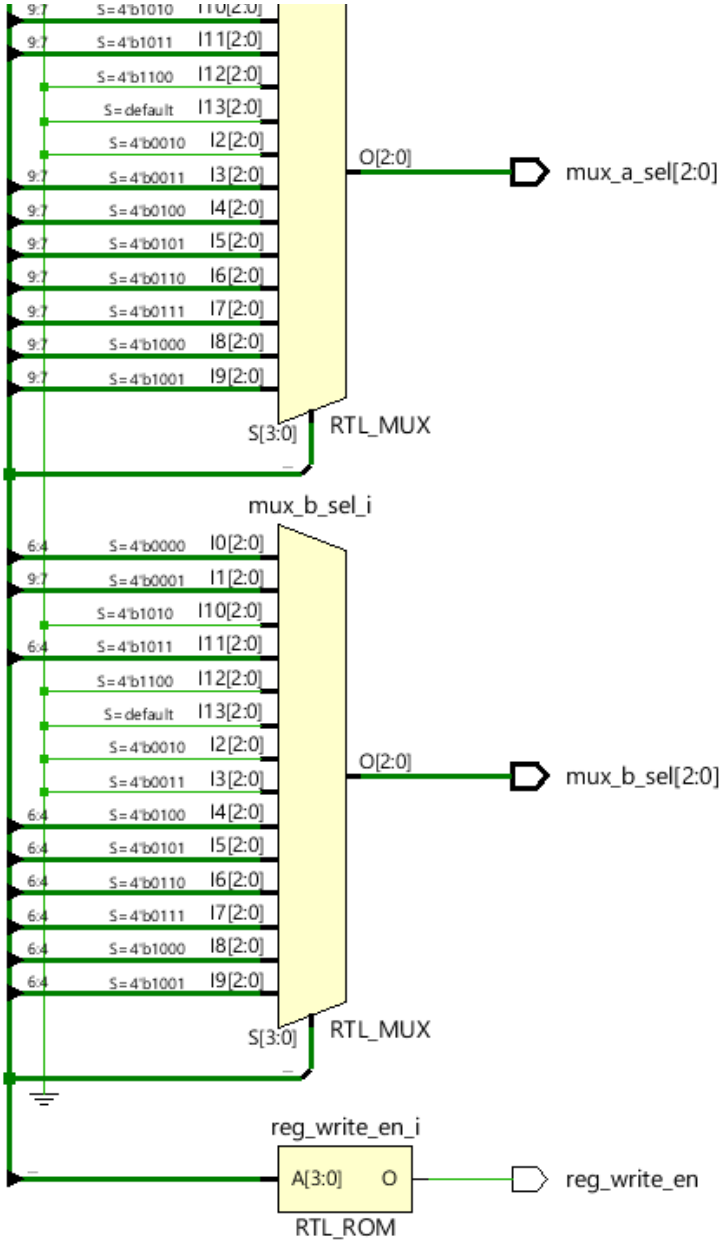
```
        dest_reg     : out std_logic_vector(2 downto 0);
        reg_write_en : out std_logic;
        jump_addr    : out std_logic_vector(3 downto 0);
        is_jzr       : out std_logic;
        is_jmp       : out std_logic
    );
end instruction_decoder_v2;

architecture behavioral of instruction_decoder_v2 is
    signal opcode : std_logic_vector(3 downto 0);

    -- ALU op constants (must match alu_4bit)
    constant ALU_ADD : std_logic_vector(2 downto 0) := "000";
    constant ALU_SUB : std_logic_vector(2 downto 0) := "001";
    constant ALU_AND : std_logic_vector(2 downto 0) := "010";
    constant ALU_OR  : std_logic_vector(2 downto 0) := "011";
    constant ALU_XOR : std_logic_vector(2 downto 0) := "100";
    constant ALU_NOT : std_logic_vector(2 downto 0) := "101";
    constant ALU_MUL : std_logic_vector(2 downto 0) := "110";
    constant ALU_DIV : std_logic_vector(2 downto 0) := "111";
begin
    opcode <= instruction(13 downto 10);

    process (instruction, opcode)
    begin
        -- safe defaults
        mux_a_sel    <= "000";
        mux_b_sel    <= "000";
        alu_op       <= ALU_ADD;
        data_src_sel <= '0';
        imm_value    <= "0000";
        dest_reg     <= "000";
        reg_write_en <= '0';
        jump_addr    <= "0000";
        is_jzr       <= '0';
        is_jmp       <= '0';

        case opcode is
            -- ADD Ra, Rb : Ra <= Ra + Rb
            when "0000" =>
                mux_a_sel    <= instruction(9 downto 7);
                mux_b_sel    <= instruction(6 downto 4);
                alu_op       <= ALU_ADD;
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- NEG R : R <= 0 - R   (R0 - R)
            when "0001" =>
                mux_a_sel    <= "000";                    -- R0 = 0
                mux_b_sel    <= instruction(9 downto 7);
                alu_op       <= ALU_SUB;
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- MOVI R, d : R <= d
            when "0010" =>
                data_src_sel <= '1';
                imm_value    <= instruction(3 downto 0);
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- JZR R, d : if R = 0 then PC <= d
            -- (ALU computes R + 0 = R so its zero flag is the jump condition)
            when "0011" =>
```

```
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= "000";
        alu_op       <= ALU_ADD;
        jump_addr    <= instruction(3 downto 0);
        is_jzr       <= '1';

    -- SUB Ra, Rb : Ra <= Ra - Rb
    when "0100" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_SUB;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- AND Ra, Rb
    when "0101" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_AND;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- OR  Ra, Rb
    when "0110" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_OR;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- XOR Ra, Rb
    when "0111" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_XOR;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- MUL Ra, Rb : Ra <= (Ra*Rb)(3..0)
    when "1000" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_MUL;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- CMP Ra, Rb : Z flag <= (Ra - Rb = 0); no register write
    when "1001" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= instruction(6 downto 4);
        alu_op       <= ALU_SUB;
        reg_write_en <= '0';

    -- NOT R : R <= NOT R
    when "1010" =>
        mux_a_sel    <= instruction(9 downto 7);
        mux_b_sel    <= "000";
        alu_op       <= ALU_NOT;
        dest_reg     <= instruction(9 downto 7);
        reg_write_en <= '1';

    -- DIV Ra, Rb : Ra <= Ra / Rb (b=0 -> 1111, overflow flag)
    when "1011" =>
        mux_a_sel    <= instruction(9 downto 7);
```

```
                mux_b_sel    <= instruction(6 downto 4);
                alu_op       <= ALU_DIV;
                dest_reg     <= instruction(9 downto 7);
                reg_write_en <= '1';

            -- JMP d : unconditional jump
            when "1100" =>
                jump_addr    <= instruction(3 downto 0);
                is_jmp       <= '1';

            when others =>
                null;
        end case;
    end process;
end behavioral;
```


alu_op_i
instruction[13:0]
A[3:0]
O[2:0]
alu_op[2:0]
RTL_ROM
data_src_sel_i
A[3:0]
O
data_src_sel
RTL_ROM
dest_reg_i
9:7
S=4'b0000
I0[2:0]
S=4'b0001
I1[2:0]
S=4'b1010
I10[2:0]
S=4'b1011
I11[2:0]
S=4'b1100
I12[2:0]
S=default
I13[2:0]
S=4'b0010
I2[2:0]
S=4'b0011
I3[2:0]
S=4'b0100
I4[2:0]
S=4'b0101
I5[2:0]
S=4'b0110
I6[2:0]
S=4'b0111
I7[2:0]
S=4'b1000
I8[2:0]
S=4'b1001
I9[2:0]
O[2:0]
dest_reg[2:0]
S[3:0]
RTL_MUX
imm_value_i
S=4'b0000
I0[3:0]
S=4'b0001
I1[3:0]
S=4'b1010
I10[3:0]
S=4'b1011
I11[3:0]
S=4'b1100
I12[3:0]
S=default
I13[3:0]
3:0
S=4'b0010
I2[3:0]
S=4'b0011
I3[3:0]
S=4'b0100
I4[3:0]
O[3:0]
imm_value[3:0]

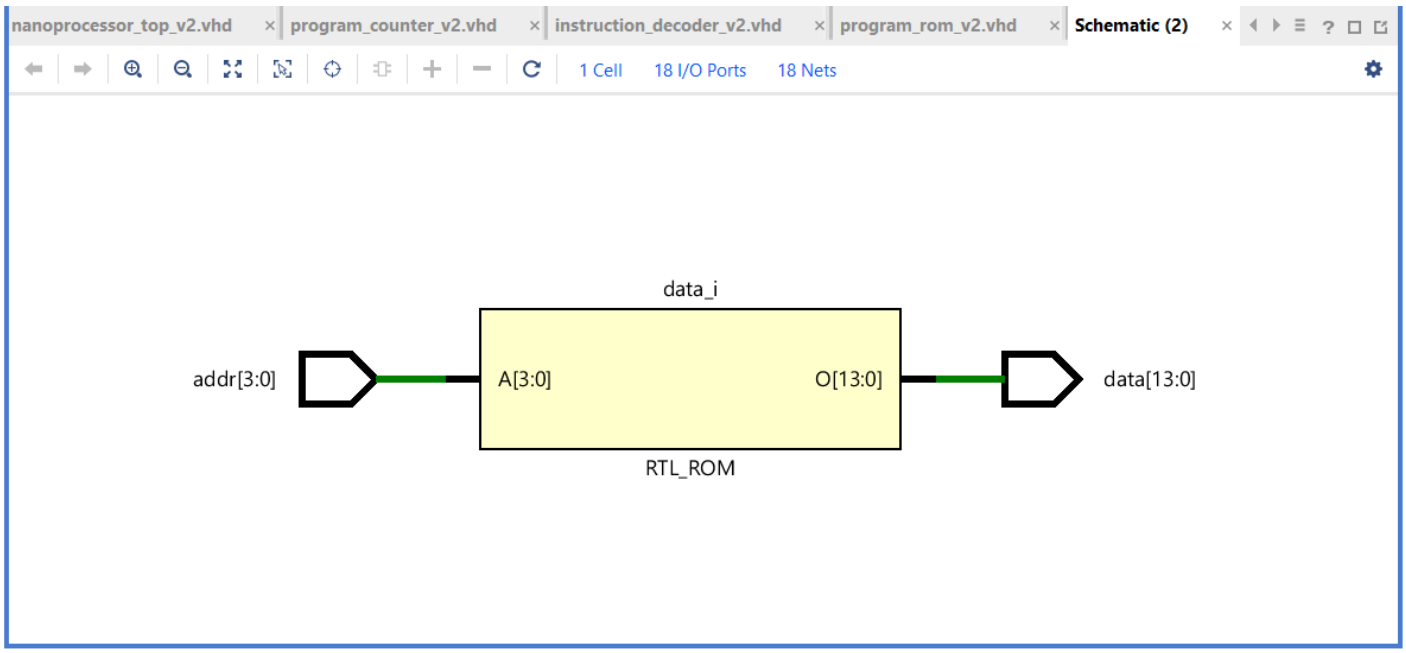

S=4'b0101 I5[3:0]
S=4'b0110 I6[3:0]
S=4'b0111 I7[3:0]
S=4'b1000 I8[3:0]
S=4'b1001 I9[3:0]
S[3:0]
RTL_MUX
is_jmp_i
A[3:0]
O
RTL_ROM
is_jmp
is_jzr_i
A[3:0]
O
RTL_ROM
is_jzr
jump_addr_i
S=4'b0000 I0[3:0]
S=4'b0001 I1[3:0]
S=4'b1010 I10[3:0]
S=4'b1011 I11[3:0]
3:0
S=4'b1100 I12[3:0]
S=default I13[3:0]
S=4'b0010 I2[3:0]
3:0
S=4'b0011 I3[3:0]
S=4'b0100 I4[3:0]
S=4'b0101 I5[3:0]
S=4'b0110 I6[3:0]
S=4'b0111 I7[3:0]
S=4'b1000 I8[3:0]
S=4'b1001 I9[3:0]
S[3:0]
RTL_MUX
O[3:0]
jump_addr[3:0]
mux_a_sel_i
9:7
S=4'b0000 I0[2:0]




## program_rom_v2.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity program_rom_v2 is
    port (
        addr : in  std_logic_vector(3 downto 0);
        data : out std_logic_vector(13 downto 0)
    );
end program_rom_v2;

architecture rtl of program_rom_v2 is
begin
    process (addr)
    begin
        case addr is
            when "0000" => data <= "00100010000011"; -- 0  MOVI R1, 3   ->
R1=3
```

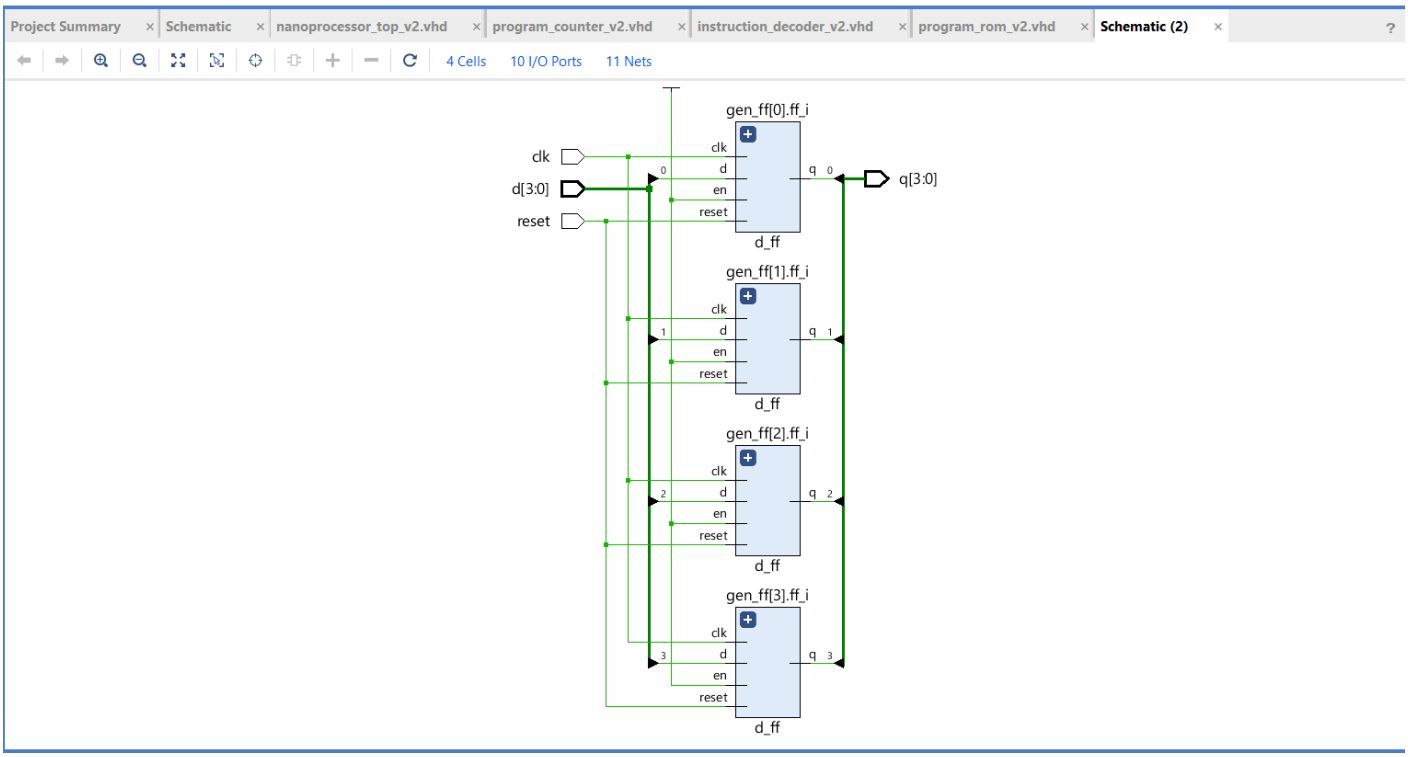
```
            when "0001" => data <= "001001000000010"; -- 1  MOVI R2, 2   -> R2=2
            when "0010" => data <= "100000101000000"; -- 2  MUL  R1, R2  -> R1=6
            when "0011" => data <= "011111111110000"; -- 3  XOR  R7, R7  -> R7=0
(clear)
            when "0100" => data <= "000011100010000"; -- 4  ADD  R7, R1  -> R7=6
(intermediate 1)
            when "0101" => data <= "001001100010000"; -- 5  MOVI R3, 8   -> R3=8
            when "0110" => data <= "001010000000010"; -- 6  MOVI R4, 2   -> R4=2
            when "0111" => data <= "101101110000000"; -- 7  DIV  R3, R4  -> R3=4
            when "1000" => data <= "000011101100000"; -- 8  ADD  R7, R3  ->
R7=10 (intermediate 2)
            when "1001" => data <= "001010100000101"; -- 9  MOVI R5, 5   -> R5=5
            when "1010" => data <= "001011000000011"; -- 10 MOVI R6, 3   -> R6=3
            when "1011" => data <= "010110111000000"; -- 11 AND  R5, R6  -> R5=1
            when "1100" => data <= "010011110100000"; -- 12 SUB  R7, R5  -> R7=9
(final)
            when "1101" => data <= "110000000011101"; -- 13 JMP  13      -> halt
loop
            when "1110" => data <= "110000000011111"; -- 14 JMP  15      ->
safety halt
            when "1111" => data <= "110000000011111"; -- 15 JMP  15      ->
safety halt
            when others => data <= (others => '0');

        end case;
    end process;
end rtl;
```




## program_counter_v2.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity program_counter_v2 is
    port (
        clk, reset : in  std_logic;
        d          : in  std_logic_vector(3 downto 0);
        q          : out std_logic_vector(3 downto 0)
```

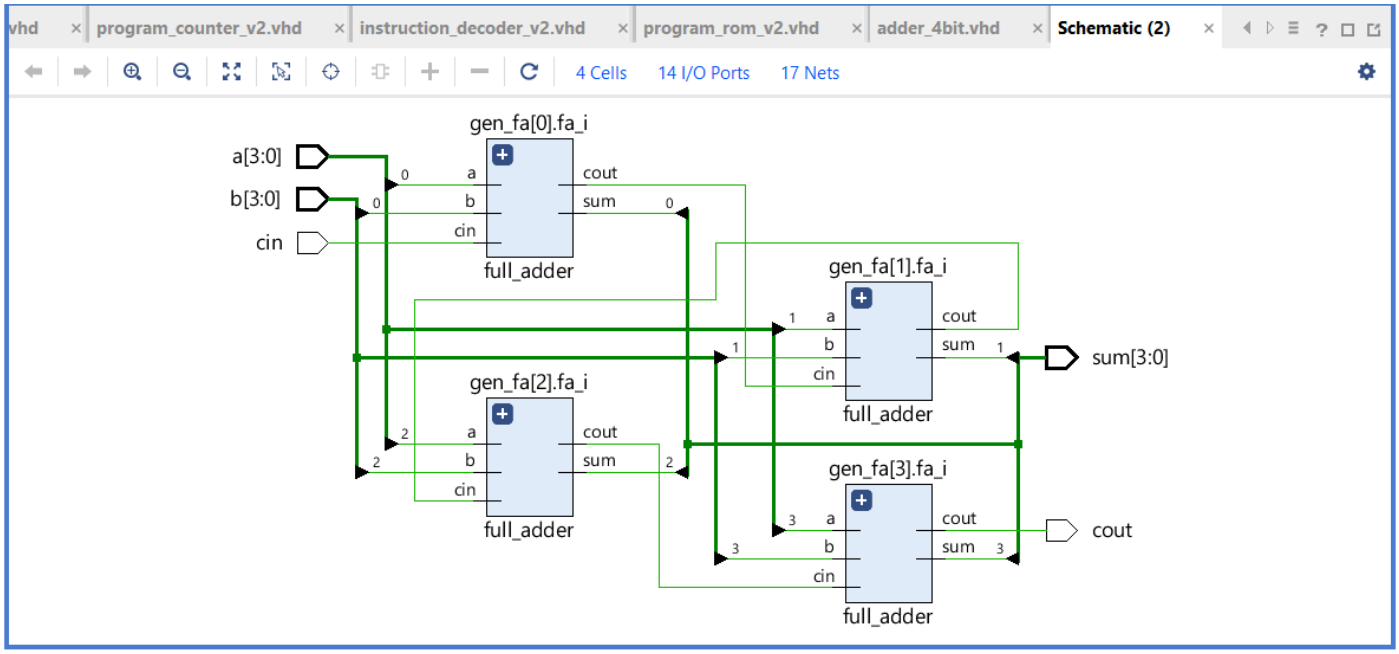
```
    );
end program_counter_v2;

architecture structural of program_counter_v2 is

    component d_ff
        port (clk, reset, en, d : in  std_logic;
              q                 : out std_logic);
    end component;

begin
    gen_ff : for i in 0 to 3 generate
        ff_i : d_ff
            port map (clk   => clk,
                      reset => reset,
                      en    => '1',
                      d     => d(i),
                      q     => q(i));
    end generate;
end structural;
```




## adder_4bit.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity adder_4bit is
    port (
        a, b : in  std_logic_vector(3 downto 0);
        cin  : in  std_logic;
        sum  : out std_logic_vector(3 downto 0);
        cout : out std_logic
    );
end adder_4bit;

architecture structural of adder_4bit is

    component full_adder
```

```
        port (a, b, cin : in  std_logic;
              sum, cout : out std_logic);
    end component;

    signal carry : std_logic_vector(4 downto 0);

begin
    carry(0) <= cin;

    gen_fa : for i in 0 to 3 generate
        fa_i : full_adder
            port map (a    => a(i),
                      b    => b(i),
                      cin  => carry(i),
                      sum  => sum(i),
                      cout => carry(i+1));
    end generate;

    cout <= carry(4);
end structural;
```

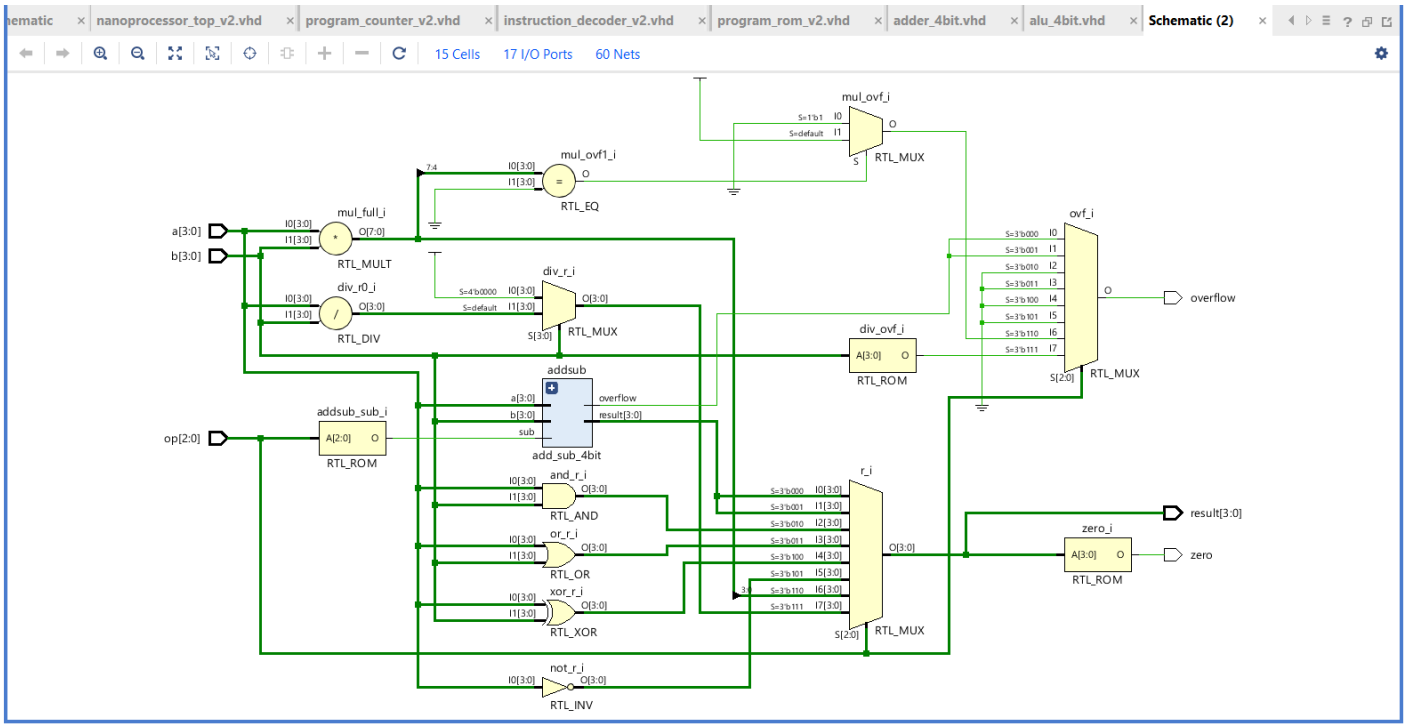



## alu_4bit.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu_4bit is
    port (
        a, b     : in  std_logic_vector(3 downto 0);
        op       : in  std_logic_vector(2 downto 0);
        result   : out std_logic_vector(3 downto 0);
        zero     : out std_logic;
        overflow : out std_logic
    );
end alu_4bit;

architecture mixed of alu_4bit is

    component add_sub_4bit
        port (a, b          : in  std_logic_vector(3 downto 0);
```

```
            sub            : in  std_logic;
            result         : out std_logic_vector(3 downto 0);
            overflow, zero : out std_logic);
    end component;

    -- ADD/SUB sub-result
    signal addsub_sub    : std_logic;
    signal addsub_result : std_logic_vector(3 downto 0);
    signal addsub_ovf    : std_logic;
    signal addsub_zero   : std_logic;

    -- Logical sub-results
    signal and_r, or_r, xor_r, not_r : std_logic_vector(3 downto 0);

    -- MUL sub-result  (full 8-bit product, low 4 bits feed result mux)
    signal mul_full : unsigned(7 downto 0);
    signal mul_r    : std_logic_vector(3 downto 0);
    signal mul_ovf  : std_logic;

    -- DIV sub-result
    signal div_r    : std_logic_vector(3 downto 0);
    signal div_ovf  : std_logic;

    -- Final mux
    signal r        : std_logic_vector(3 downto 0);
    signal ovf      : std_logic;

begin
    addsub_sub <= '1' when op = "001" else '0';

    addsub : add_sub_4bit
        port map (a        => a,
                  b        => b,
                  sub      => addsub_sub,
                  result   => addsub_result,
                  overflow => addsub_ovf,
                  zero     => addsub_zero);

    and_r <= a and b;
    or_r  <= a or  b;
    xor_r <= a xor b;
    not_r <= not a;

    mul_full <= unsigned(a) * unsigned(b);
    mul_r    <= std_logic_vector(mul_full(3 downto 0));
    mul_ovf  <= '0' when mul_full(7 downto 4) = "0000" else '1';

    div_proc : process (a, b)
    begin
        if b = "0000" then
            div_r   <= "1111";
            div_ovf <= '1';
        else
            div_r   <= std_logic_vector(unsigned(a) / unsigned(b));
            div_ovf <= '0';
        end if;
    end process;

    with op select
        r <= addsub_result when "000",
             addsub_result when "001",
             and_r          when "010",
             or_r           when "011",
             xor_r          when "100",
             not_r          when "101",
             mul_r          when "110",
```

```
            div_r          when "111",
            "0000"         when others;

    with op select
        ovf <= addsub_ovf when "000",
               addsub_ovf when "001",
               '0'        when "010",
               '0'        when "011",
               '0'        when "100",
               '0'        when "101",
               mul_ovf    when "110",
               div_ovf    when "111",
               '0'        when others;

    result   <= r;
    overflow <= ovf;
    zero     <= '1' when r = "0000" else '0';
end mixed;
```



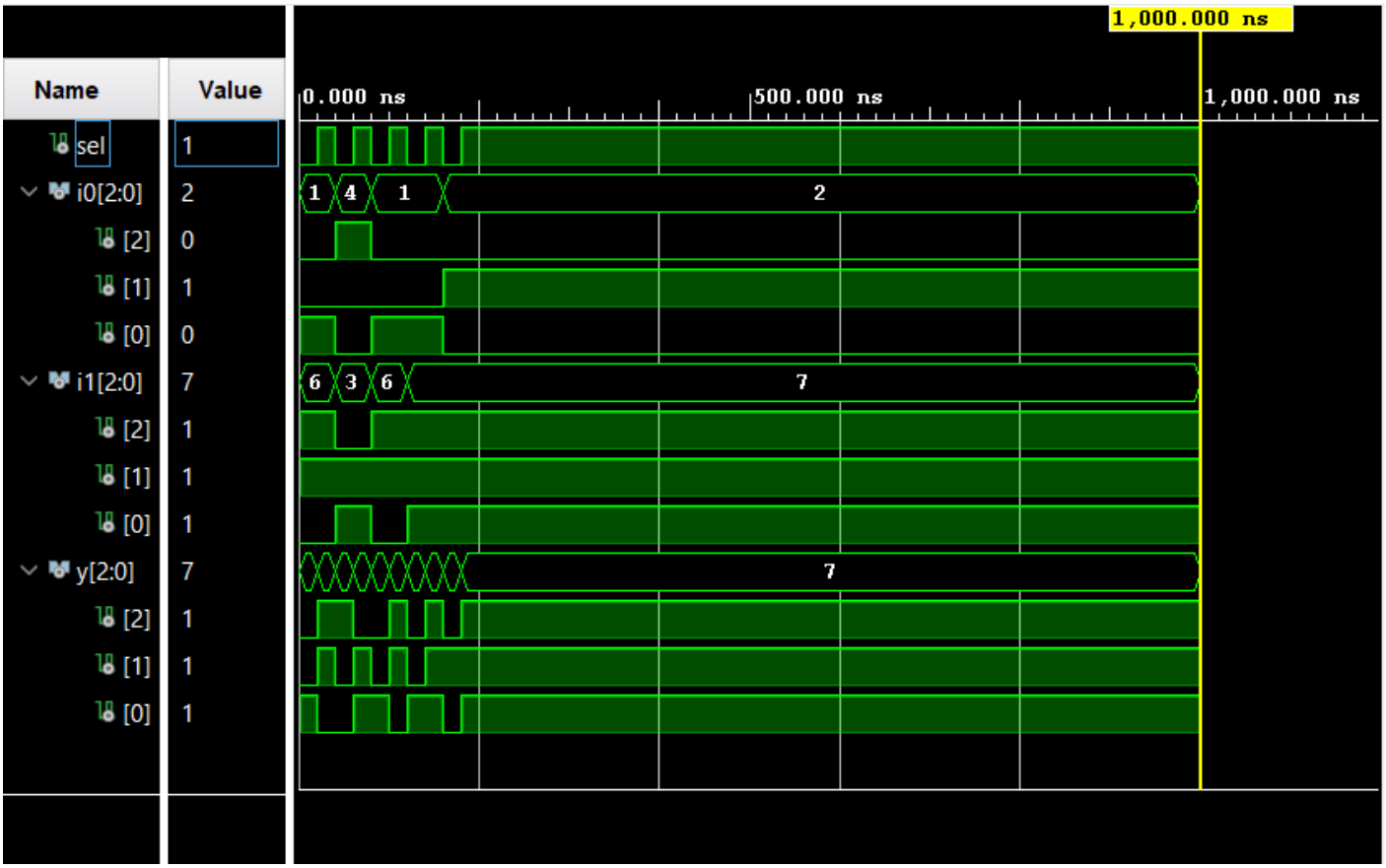
# Appendix D - Testbench Codes

## D.1 Common Component Testbenches

These testbenches verify the reusable component-level blocks. The files below are taken from the basic project simulation folder.

### Address_Selector_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Address_Selector_TB is
end Address_Selector_TB;

architecture sim of Address_Selector_TB is
    component mux_2way_3bit
        port (sel : in  std_logic;
              i0  : in  std_logic_vector(2 downto 0);
              i1  : in  std_logic_vector(2 downto 0);
              y   : out std_logic_vector(2 downto 0));
    end component;

    signal sel     : std_logic := '0';
    signal i0, i1, y : std_logic_vector(2 downto 0) := (others => '0');
begin
    uut : mux_2way_3bit port map (sel, i0, i1, y);

    stim : process
    begin
        -- sel=0 -> y=i0 (PC+1 path)
        i0 <= "001"; i1 <= "110"; sel <= '0'; wait for 20 ns;
        -- sel=1 -> y=i1 (jump path)
        sel <= '1'; wait for 20 ns;

        -- 240180: i0=bits[2:0]="100", i1="011" (bits[6:4])
        i0 <= "100"; i1 <= "011"; sel <= '0'; wait for 20 ns; -- y="100"
        sel <= '1'; wait for 20 ns;                            -- y="011"

        -- 240225: i0=bits[2:0]="001", i1="110" (bits[6:4])
        i0 <= "001"; i1 <= "110"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        -- 240497: i0=bits[2:0]="001", i1="111" (bits[6:4])
        i0 <= "001"; i1 <= "111"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        -- 240498: i0=bits[2:0]="010", i1="111" (bits[6:4])
        i0 <= "010"; i1 <= "111"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        wait;
    end process;
end sim;
```




## Add_Sub_4_TB.vhd

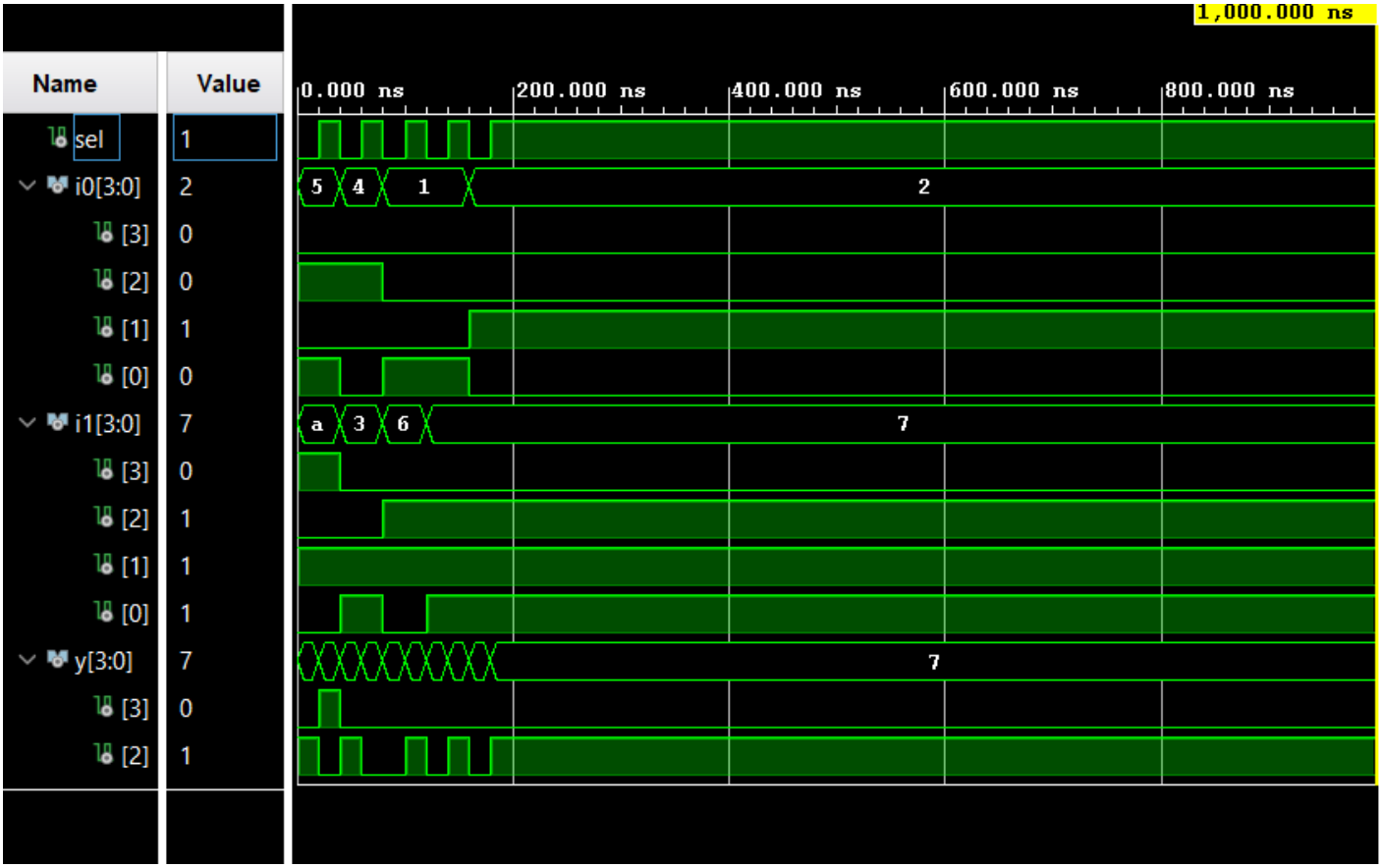
```
library ieee;
use ieee.std_logic_1164.all;

entity Add_Sub_4_TB is
end Add_Sub_4_TB;

architecture sim of Add_Sub_4_TB is
    component add_sub_4bit
        port (a, b   : in  std_logic_vector(3 downto 0);
              sub    : in  std_logic;
              result : out std_logic_vector(3 downto 0);
              overflow, zero : out std_logic);
    end component;

    signal a, b, result : std_logic_vector(3 downto 0) := (others => '0');
    signal sub          : std_logic := '0';
    signal overflow, zero : std_logic;
begin
    uut : add_sub_4bit port map (a, b, sub, result, overflow, zero);

    stim : process
    begin
        -- 3 + 5 = 8 (overflow)
        a <= "0011"; b <= "0101"; sub <= '0'; wait for 20 ns;
        -- 5 - 3 = 2
        a <= "0101"; b <= "0011"; sub <= '1'; wait for 20 ns;
        -- 3 - 3 = 0 (zero=1)
        a <= "0011"; b <= "0011"; sub <= '1'; wait for 20 ns;
        -- 0 - 1 = -1
        a <= "0000"; b <= "0001"; sub <= '1'; wait for 20 ns;

        -- 240180: a=bits[3:0]="0100", b=bits[7:4]="0011"
```

```
        a <= "0100"; b <= "0011"; sub <= '0'; wait for 20 ns; -- 4+3=7
        a <= "0100"; b <= "0011"; sub <= '1'; wait for 20 ns; -- 4-3=1

        -- 240225: a=bits[3:0]="0001", b=bits[7:4]="0110"
        a <= "0001"; b <= "0110"; sub <= '0'; wait for 20 ns; -- 1+6=7
        a <= "0001"; b <= "0110"; sub <= '1'; wait for 20 ns; -- 1-6=-5

        -- 240497: a=bits[3:0]="0001", b=bits[7:4]="0111"
        a <= "0001"; b <= "0111"; sub <= '0'; wait for 20 ns; -- 1+7=8 overflow
        a <= "0001"; b <= "0111"; sub <= '1'; wait for 20 ns; -- 1-7=-6

        -- 240498: a=bits[3:0]="0010", b=bits[7:4]="0111"
        a <= "0010"; b <= "0111"; sub <= '0'; wait for 20 ns; -- 2+7=9 overflow
        a <= "0010"; b <= "0111"; sub <= '1'; wait for 20 ns; -- 2-7=-5

        wait;
    end process;
end sim;
```


## Load_Selector_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Load_Selector_TB is
end Load_Selector_TB;
```

```
architecture sim of Load_Selector_TB is
    component mux_2way_4bit
        port (sel : in  std_logic;
              i0  : in  std_logic_vector(3 downto 0);
              i1  : in  std_logic_vector(3 downto 0);
              y   : out std_logic_vector(3 downto 0));
    end component;

    signal sel     : std_logic := '0';
    signal i0, i1, y : std_logic_vector(3 downto 0) := (others => '0');
begin
    uut : mux_2way_4bit port map (sel, i0, i1, y);

    stim : process
    begin
        -- sel=0 -> y=i0 (ALU result)
        i0 <= "0101"; i1 <= "1010"; sel <= '0'; wait for 20 ns;
        -- sel=1 -> y=i1 (immediate)
        sel <= '1'; wait for 20 ns;

        -- 240180: i0=bits[3:0]="0100", i1=bits[7:4]="0011"
        i0 <= "0100"; i1 <= "0011"; sel <= '0'; wait for 20 ns; -- y="0100"
        sel <= '1'; wait for 20 ns;                              -- y="0011"

        -- 240225: i0=bits[3:0]="0001", i1=bits[7:4]="0110"
        i0 <= "0001"; i1 <= "0110"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        -- 240497: i0=bits[3:0]="0001", i1=bits[7:4]="0111"
        i0 <= "0001"; i1 <= "0111"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        -- 240498: i0=bits[3:0]="0010", i1=bits[7:4]="0111"
        i0 <= "0010"; i1 <= "0111"; sel <= '0'; wait for 20 ns;
        sel <= '1'; wait for 20 ns;

        wait;
    end process;
end sim;
```




## LUT_16_7_TB.vhd

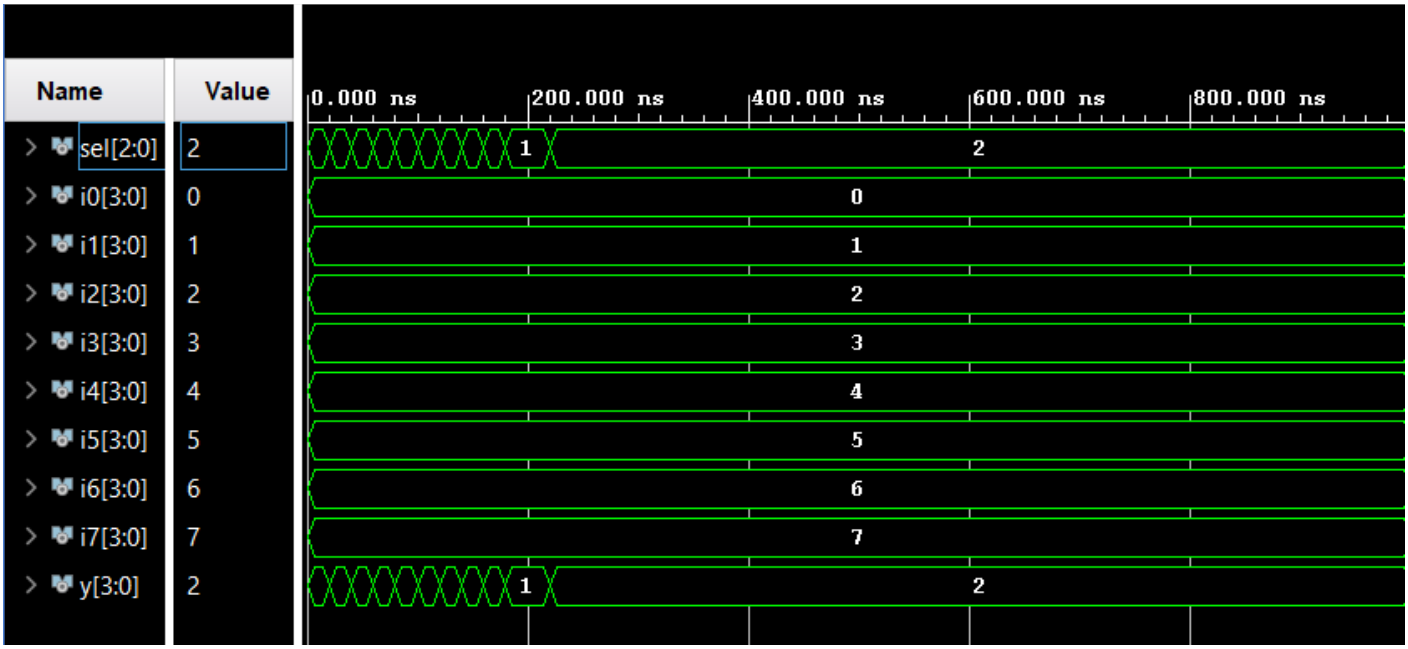
```
library ieee;
use ieee.std_logic_1164.all;

entity LUT_16_7_TB is
end LUT_16_7_TB;

architecture sim of LUT_16_7_TB is
    component seven_seg_display
        port (value : in  std_logic_vector(3 downto 0);
              seg   : out std_logic_vector(6 downto 0);
              an    : out std_logic_vector(3 downto 0));
    end component;

    signal value : std_logic_vector(3 downto 0) := (others => '0');
    signal seg   : std_logic_vector(6 downto 0);
    signal an    : std_logic_vector(3 downto 0);
begin
    uut : seven_seg_display port map (value, seg, an);

    stim : process
    begin
        -- sweep all 16 inputs
        value <= "0000"; wait for 20 ns; -- 0
        value <= "0001"; wait for 20 ns; -- 1
        value <= "0010"; wait for 20 ns; -- 2
        value <= "0011"; wait for 20 ns; -- 3
        value <= "0100"; wait for 20 ns; -- 4
        value <= "0101"; wait for 20 ns; -- 5
        value <= "0110"; wait for 20 ns; -- 6
        value <= "0111"; wait for 20 ns; -- 7
        value <= "1000"; wait for 20 ns; -- 8
        value <= "1001"; wait for 20 ns; -- 9
```

```
        value <= "1010"; wait for 20 ns; -- A
        value <= "1011"; wait for 20 ns; -- B
        value <= "1100"; wait for 20 ns; -- C
        value <= "1101"; wait for 20 ns; -- D
        value <= "1110"; wait for 20 ns; -- E
        value <= "1111"; wait for 20 ns; -- F

        -- 240180: bits[3:0]="0100"
        value <= "0100"; wait for 20 ns;
        -- 240225: bits[3:0]="0001"
        value <= "0001"; wait for 20 ns;
        -- 240497: bits[3:0]="0001"
        value <= "0001"; wait for 20 ns;
        -- 240498: bits[3:0]="0010"
        value <= "0010"; wait for 20 ns;

        wait;
    end process;
end sim;
```


## RegisterData_Multiplexer_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity RegisterData_Multiplexer_TB is
```

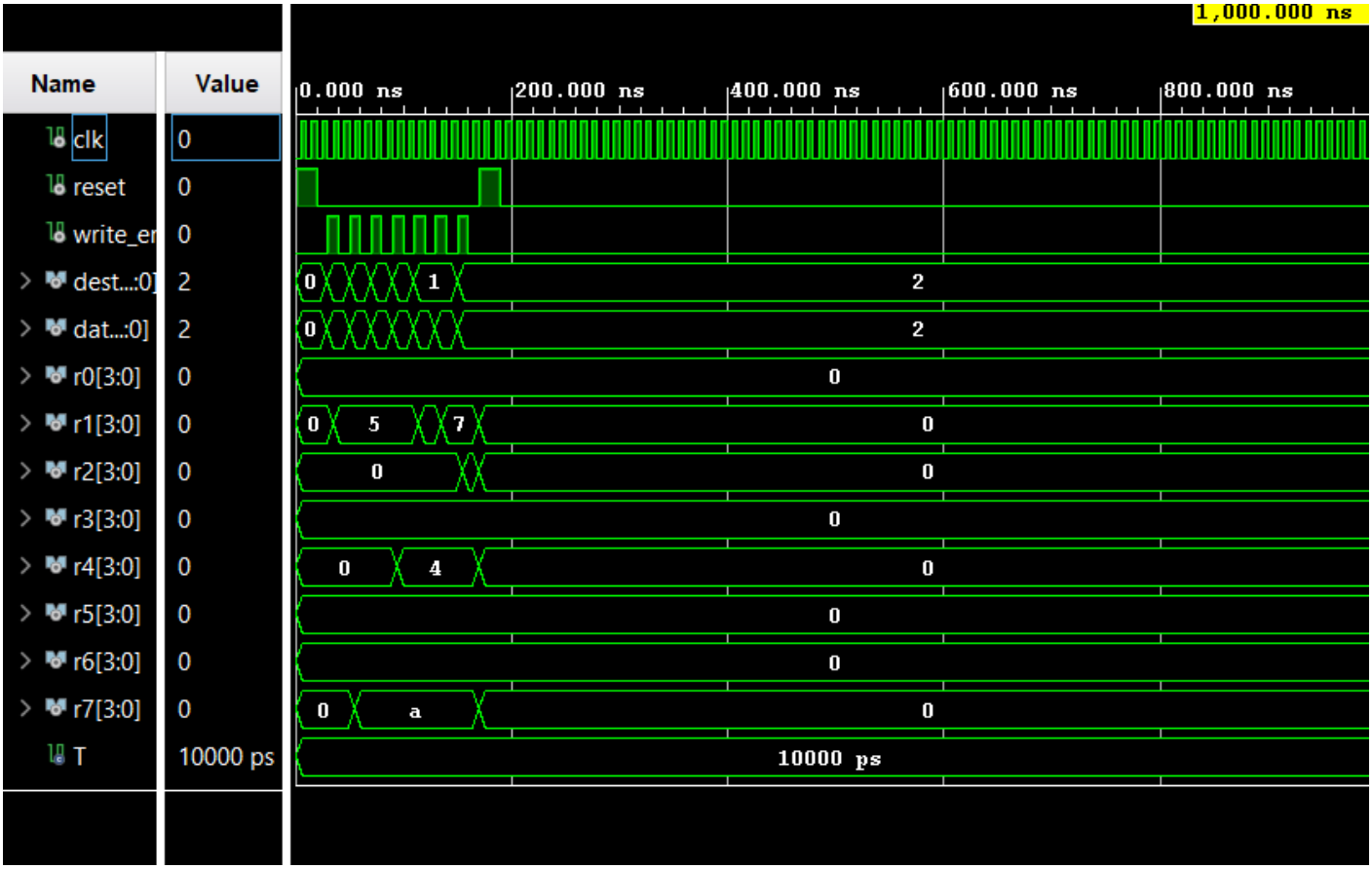
```
end RegisterData_Multiplexer_TB;

architecture sim of RegisterData_Multiplexer_TB is
    component mux_8way_4bit
        port (sel            : in  std_logic_vector(2 downto 0);
              i0, i1, i2, i3 : in  std_logic_vector(3 downto 0);
              i4, i5, i6, i7 : in  std_logic_vector(3 downto 0);
              y              : out std_logic_vector(3 downto 0));
    end component;

    signal sel            : std_logic_vector(2 downto 0) := "000";
    signal i0, i1, i2, i3 : std_logic_vector(3 downto 0) := (others => '0');
    signal i4, i5, i6, i7 : std_logic_vector(3 downto 0) := (others => '0');
    signal y              : std_logic_vector(3 downto 0);
begin
    -- Load register values derived from index numbers
    -- 240180: "0100", 240225: "0001", 240497: "0001", 240498: "0010"
    i0 <= "0000"; -- R0 hardwired 0
    i1 <= "0001"; -- 240225 bits[3:0]
    i2 <= "0010"; -- 240498 bits[3:0]
    i3 <= "0011"; -- general test
    i4 <= "0100"; -- 240180 bits[3:0]
    i5 <= "0101"; -- general test
    i6 <= "0110"; -- 240225 bits[7:4]
    i7 <= "0111"; -- 240497 bits[7:4]

    uut : mux_8way_4bit
        port map (sel, i0, i1, i2, i3, i4, i5, i6, i7, y);

    stim : process
    begin
        -- sweep all 8 selects
        sel <= "000"; wait for 20 ns; -- y=i0="0000"
        sel <= "001"; wait for 20 ns; -- y=i1="0001"
        sel <= "010"; wait for 20 ns; -- y=i2="0010"
        sel <= "011"; wait for 20 ns; -- y=i3="0011"
        sel <= "100"; wait for 20 ns; -- y=i4="0100"
        sel <= "101"; wait for 20 ns; -- y=i5="0101"
        sel <= "110"; wait for 20 ns; -- y=i6="0110"
        sel <= "111"; wait for 20 ns; -- y=i7="0111"

        -- 240180: sel=bits[2:0]="100" -> y=i4="0100"
        sel <= "100"; wait for 20 ns; -- 240180

        -- 240225: sel=bits[2:0]="001" -> y=i1="0001"
        sel <= "001"; wait for 20 ns; -- 240225

        -- 240497: sel=bits[2:0]="001" -> y=i1="0001"
        sel <= "001"; wait for 20 ns; -- 240497

        -- 240498: sel=bits[2:0]="010" -> y=i2="0010"
        sel <= "010"; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```




## Register_Bank_TB.vhd

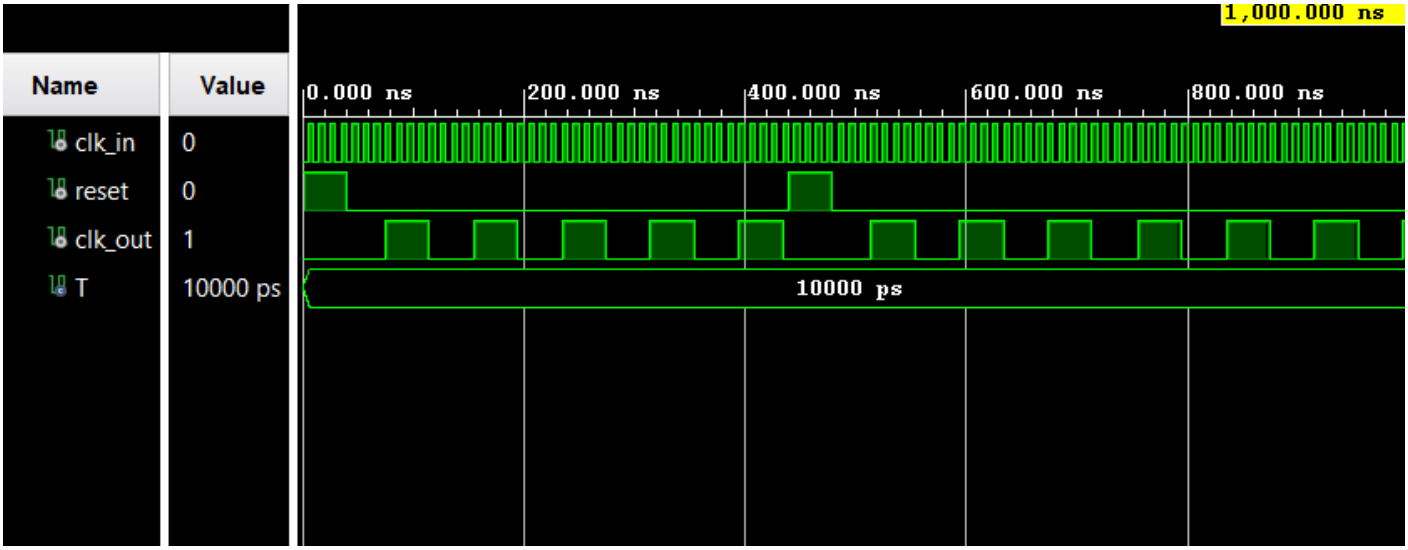
```
library ieee;
use ieee.std_logic_1164.all;

entity Register_Bank_TB is
end Register_Bank_TB;

architecture sim of Register_Bank_TB is
    component register_bank
        port (clk, reset, write_en : in std_logic;
              dest_sel : in  std_logic_vector(2 downto 0);
              data_in  : in  std_logic_vector(3 downto 0);
              r0, r1, r2, r3 : out std_logic_vector(3 downto 0);
              r4, r5, r6, r7 : out std_logic_vector(3 downto 0));
    end component;

    signal clk, reset, write_en : std_logic := '0';
    signal dest_sel : std_logic_vector(2 downto 0) := "000";
    signal data_in  : std_logic_vector(3 downto 0) := (others => '0');
    signal r0, r1, r2, r3, r4, r5, r6, r7 : std_logic_vector(3 downto 0);
    constant T : time := 10 ns;
begin
    uut : register_bank
        port map (clk, reset, write_en, dest_sel, data_in,
                  r0, r1, r2, r3, r4, r5, r6, r7);

    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 2*T;
        reset <= '0'; wait for T;

        -- write 5 to R1
        dest_sel <= "001"; data_in <= "0101"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;
```

```
        -- write A to R7
        dest_sel <= "111"; data_in <= "1010"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;

        -- attempt write to R0 (hardwired, should be ignored)
        dest_sel <= "000"; data_in <= "1111"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;

        -- 240180: data_in=bits[3:0]="0100", dest=bits[2:0]="100" (R4)
        dest_sel <= "100"; data_in <= "0100"; write_en <= '1'; wait for T; --
240180
        write_en <= '0'; wait for T;

        -- 240225: data_in=bits[3:0]="0001", dest=bits[2:0]="001" (R1)
        dest_sel <= "001"; data_in <= "0001"; write_en <= '1'; wait for T; --
240225
        write_en <= '0'; wait for T;

        -- 240497: data_in=bits[7:4]="0111", dest=bits[2:0]="001" (R1)
        dest_sel <= "001"; data_in <= "0111"; write_en <= '1'; wait for T; --
240497
        write_en <= '0'; wait for T;

        -- 240498: data_in=bits[3:0]="0010", dest=bits[2:0]="010" (R2)
        dest_sel <= "010"; data_in <= "0010"; write_en <= '1'; wait for T; --
240498
        write_en <= '0'; wait for T;

        -- reset clears all registers
        reset <= '1'; wait for 2*T;
        reset <= '0'; wait for T;

        wait;
    end process;
end sim;
```



## Slow_Clk_TB.vhd

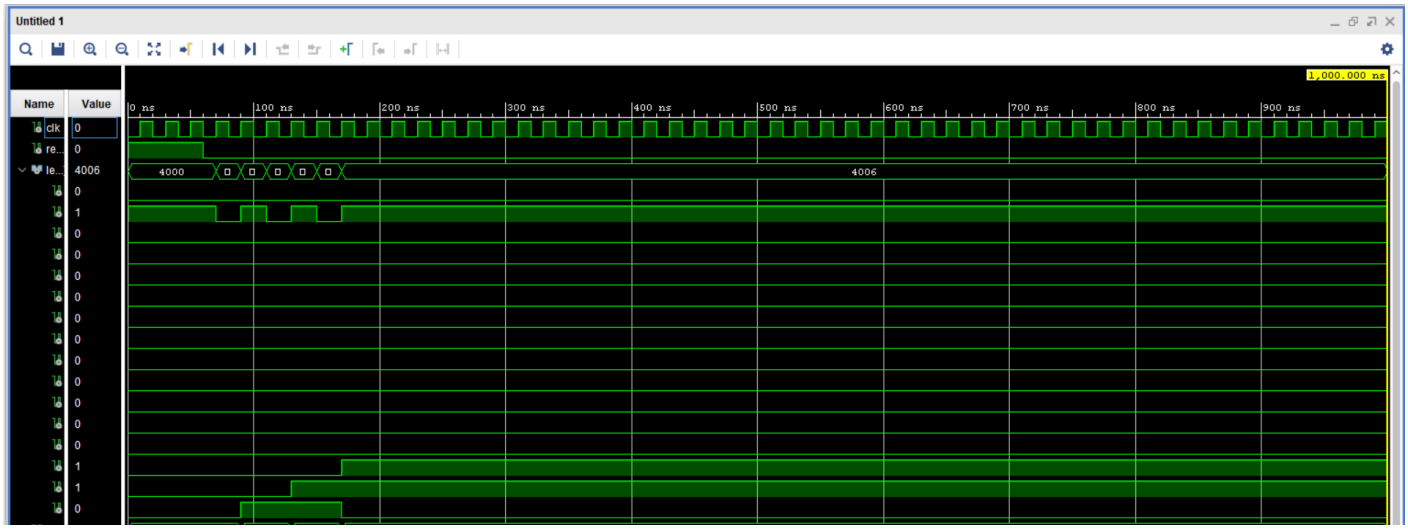
```
library ieee;
use ieee.std_logic_1164.all;

entity Slow_Clk_TB is
end Slow_Clk_TB;

architecture sim of Slow_Clk_TB is
    component clock_divider
        generic (DIVISOR : integer := 100_000_000);
        port (clk_in  : in  std_logic;
              reset   : in  std_logic;
              clk_out : out std_logic);
    end component;

    signal clk_in, reset, clk_out : std_logic := '0';
    constant T : time := 10 ns;
begin
    uut : clock_divider
        generic map (DIVISOR => 4)
        port map (clk_in, reset, clk_out);

    clk_gen : process
    begin
        clk_in <= '0'; wait for T/2;
        clk_in <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        -- apply reset
        reset <= '1'; wait for 4*T;
        reset <= '0';

        -- observe several slow clock cycles
        wait for 40*T;

        -- mid-run reset to verify counter clears
        reset <= '1'; wait for 4*T;
        reset <= '0';

        -- observe again after reset
        wait for 40*T;

        wait;
    end process;
end sim;
```

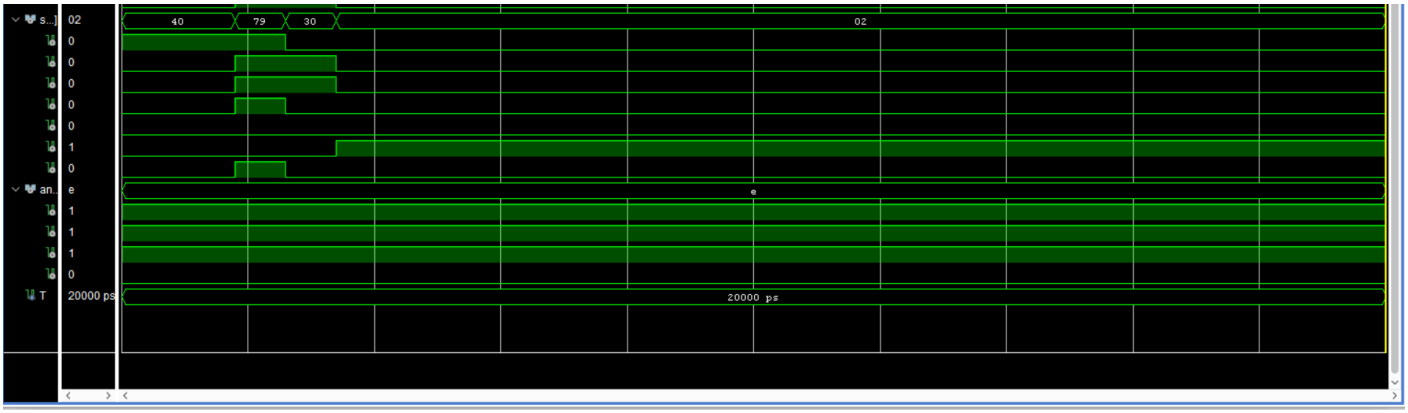

Name
Value
clk_in
0
reset
0
clk_out
1
T
10000 ps
1,000.000 ns
0.000 ns
200.000 ns
400.000 ns
600.000 ns
800.000 ns
10000 ps

## D.2 Basic Nano Processor Testbenches

### NanoProcessor_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity NanoProcessor_TB is
end NanoProcessor_TB;

architecture sim of NanoProcessor_TB is
    component nanoprocessor_top
        generic (SIM_MODE : boolean; DIVISOR : integer);
        port (clk, reset : in  std_logic;
              led        : out std_logic_vector(15 downto 0);
              seg        : out std_logic_vector(6 downto 0);
              an         : out std_logic_vector(3 downto 0));
    end component;

    signal clk, reset : std_logic := '0';
    signal led        : std_logic_vector(15 downto 0);
    signal seg        : std_logic_vector(6 downto 0);
    signal an         : std_logic_vector(3 downto 0);
    constant T : time := 20 ns;
begin
    uut : nanoprocessor_top
        generic map (SIM_MODE => true, DIVISOR => 4)
        port map (clk, reset, led, seg, an);

    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 3*T;
        reset <= '0';

        wait;
    end process;
end sim;
```

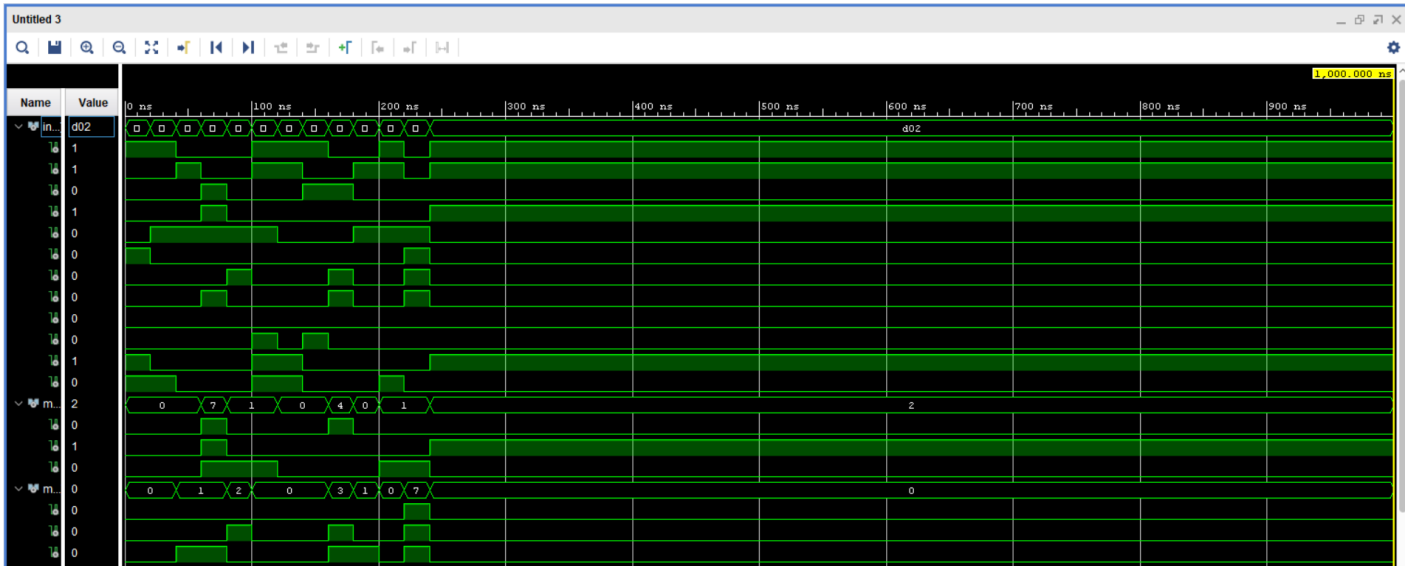


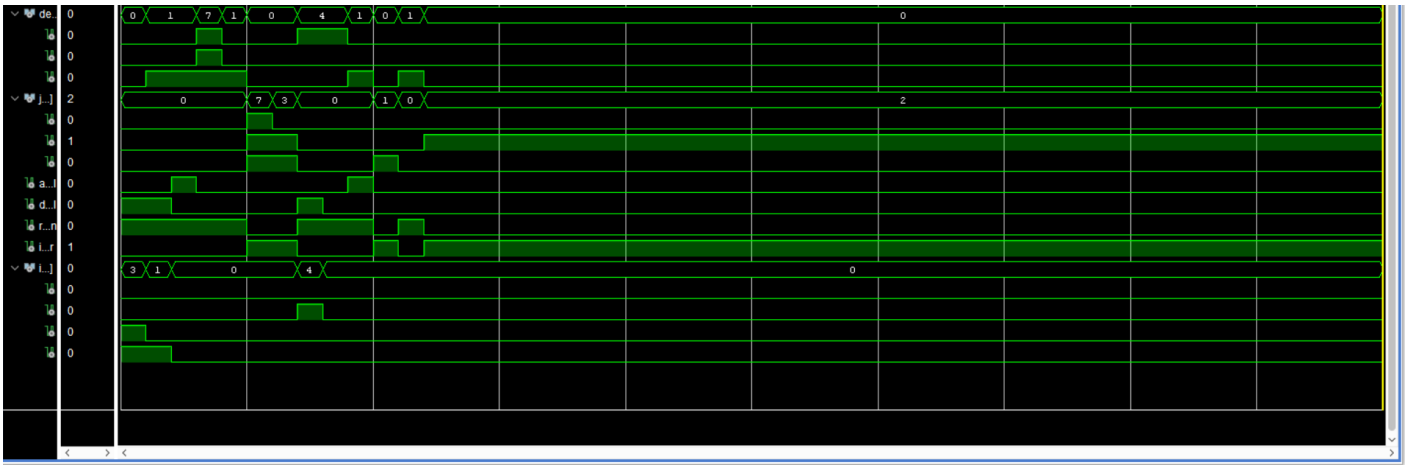



## Instruction_Decoder_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Instruction_Decoder_TB is
end Instruction_Decoder_TB;

architecture sim of Instruction_Decoder_TB is
    component instruction_decoder
        port (instruction  : in  std_logic_vector(11 downto 0);
              mux_a_sel    : out std_logic_vector(2 downto 0);
              mux_b_sel    : out std_logic_vector(2 downto 0);
              addsub_sel   : out std_logic;
              data_src_sel : out std_logic;
              imm_value    : out std_logic_vector(3 downto 0);
              dest_reg     : out std_logic_vector(2 downto 0);
              reg_write_en : out std_logic;
              jump_addr    : out std_logic_vector(2 downto 0);
              is_jzr       : out std_logic);
    end component;

    signal instruction  : std_logic_vector(11 downto 0) := (others => '0');
    signal mux_a_sel, mux_b_sel, dest_reg, jump_addr : std_logic_vector(2
downto 0);
    signal addsub_sel, data_src_sel, reg_write_en, is_jzr : std_logic;
    signal imm_value : std_logic_vector(3 downto 0);
begin
    uut : instruction_decoder
        port map (instruction, mux_a_sel, mux_b_sel, addsub_sel,
                  data_src_sel, imm_value, dest_reg, reg_write_en,
                  jump_addr, is_jzr);

    stim : process
    begin
        -- MOVI R1, 3 -> 1 0 001 000 0011
        instruction <= "100001000011"; wait for 20 ns;
        -- MOVI R2, 1 -> 1 0 010 000 0001
        instruction <= "100010000001"; wait for 20 ns;
        -- NEG R2     -> 0 1 010 000 0000
        instruction <= "010010000000"; wait for 20 ns;
        -- ADD R7, R1 -> 0 0 111 001 0000
        instruction <= "001110010000"; wait for 20 ns;
        -- ADD R1, R2 -> 0 0 001 010 0000
        instruction <= "000010100000"; wait for 20 ns;
        -- JZR R1, 7  -> 1 1 001 000 0111
        instruction <= "110010000111"; wait for 20 ns;
        -- JZR R0, 3  -> 1 1 000 000 0011
        instruction <= "110000000011"; wait for 20 ns;
```

```
        -- 240180: MOVI R4,4 -> 1 0 100 000 0100
        instruction <= "101000000100"; wait for 20 ns;
        -- 240180: ADD R4,R3  -> 0 0 100 011 0000
        instruction <= "001000110000"; wait for 20 ns;

        -- 240225: NEG R1     -> 0 1 001 000 0000
        instruction <= "010010000000"; wait for 20 ns;
        -- 240225: JZR R1,1   -> 1 1 001 000 0001
        instruction <= "110010000001"; wait for 20 ns;

        -- 240497: ADD R1,R7  -> 0 0 001 111 0000
        instruction <= "000011110000"; wait for 20 ns;

        -- 240498: JZR R2,2   -> 1 1 010 000 0010
        instruction <= "110100000010"; wait for 20 ns;

        wait;
    end process;
end sim;
```

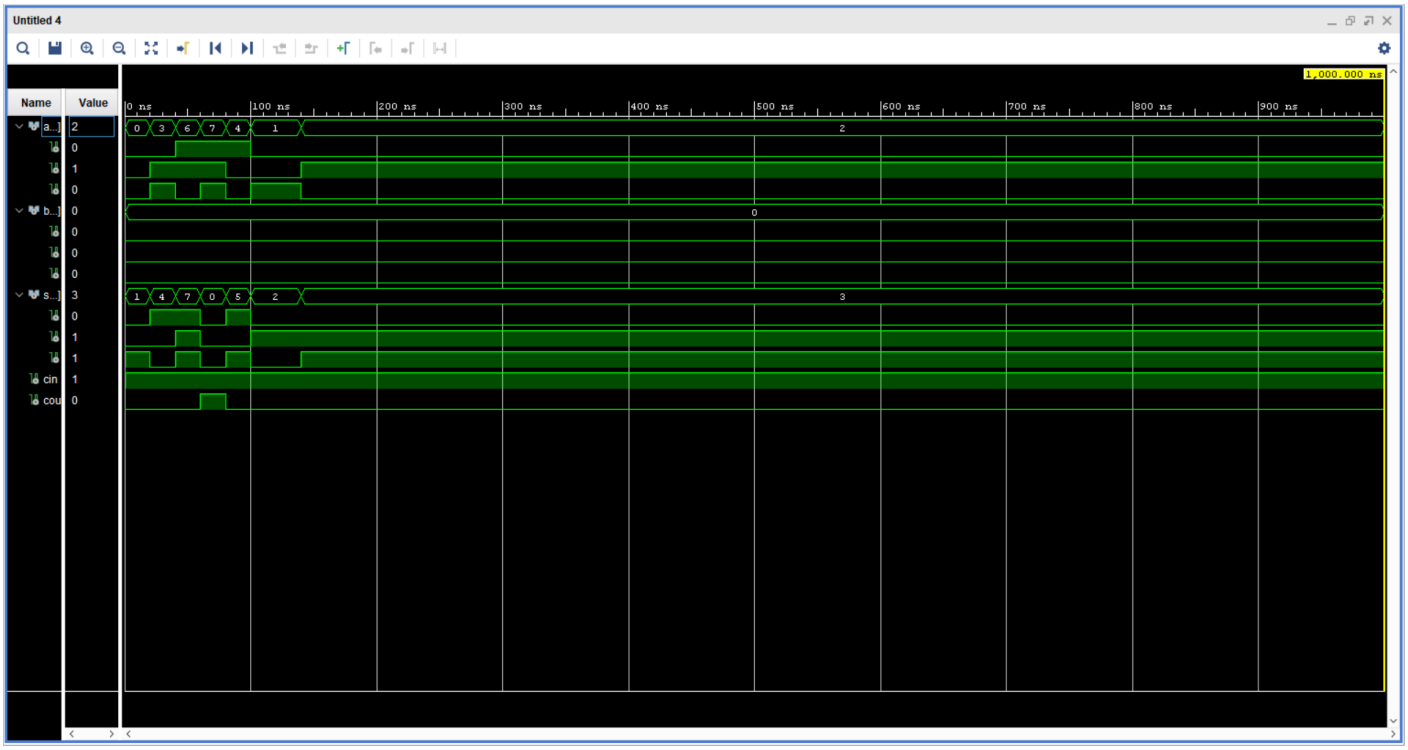


## PC_Adder_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity PC_Adder_TB is
end PC_Adder_TB;

architecture sim of PC_Adder_TB is
```

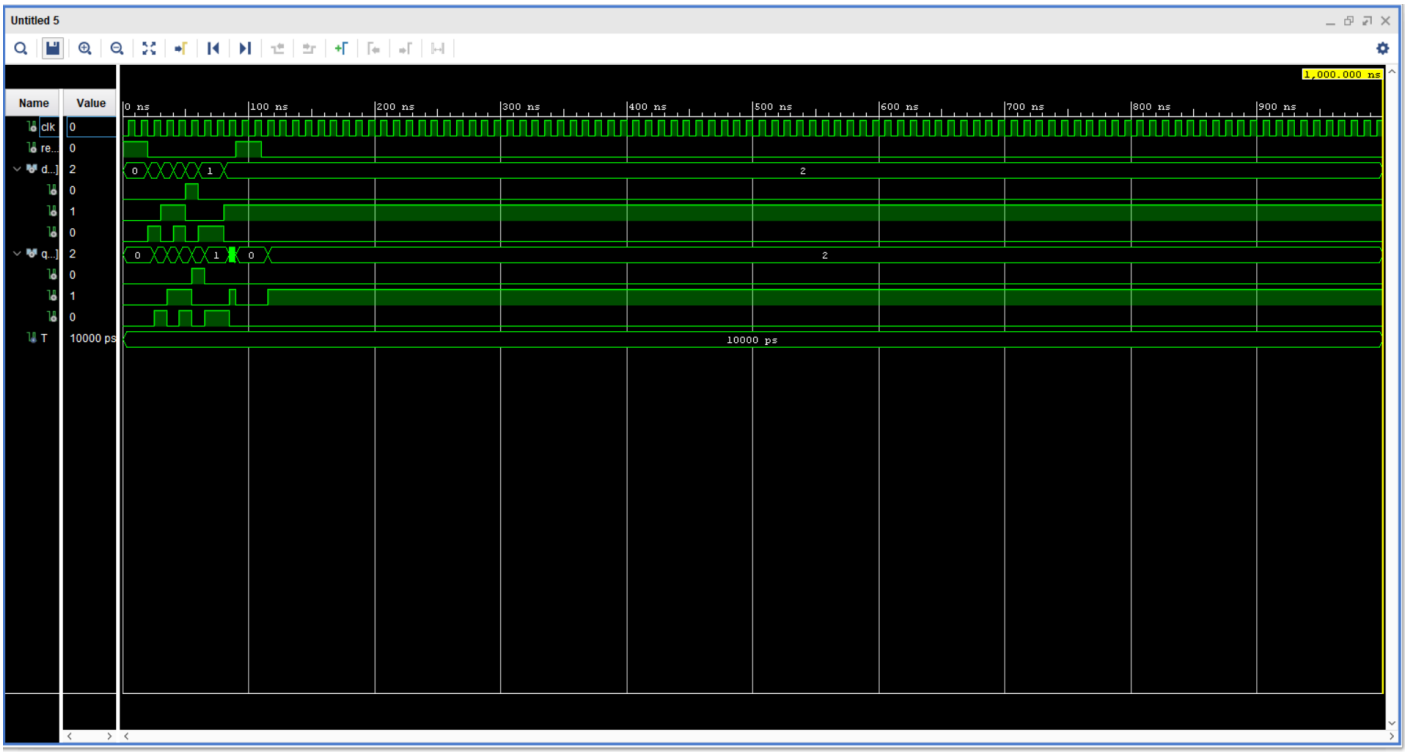
```
    component adder_3bit
        port (a    : in  std_logic_vector(2 downto 0);
              b    : in  std_logic_vector(2 downto 0);
              cin  : in  std_logic;
              sum  : out std_logic_vector(2 downto 0);
              cout : out std_logic);
    end component;

    signal a, b, sum : std_logic_vector(2 downto 0) := (others => '0');
    signal cin, cout : std_logic := '0';
begin
    uut : adder_3bit port map (a, b, cin, sum, cout);

    stim : process
    begin
        -- PC+1 behaviour (b="000", cin='1')
        a <= "000"; b <= "000"; cin <= '1'; wait for 20 ns; -- 0+1=1
        a <= "011"; b <= "000"; cin <= '1'; wait for 20 ns; -- 3+1=4
        a <= "110"; b <= "000"; cin <= '1'; wait for 20 ns; -- 6+1=7
        a <= "111"; b <= "000"; cin <= '1'; wait for 20 ns; -- 7+1=0 (wrap)

        -- 240180: a=bits[2:0]="100", b="000", cin='1'  -> sum="101"
        a <= "100"; b <= "000"; cin <= '1'; wait for 20 ns; -- 240180
        -- 240225: a=bits[2:0]="001", cin='1'            -> sum="010"
        a <= "001"; b <= "000"; cin <= '1'; wait for 20 ns; -- 240225
        -- 240497: a=bits[2:0]="001", cin='1'            -> sum="010"
        a <= "001"; b <= "000"; cin <= '1'; wait for 20 ns; -- 240497
        -- 240498: a=bits[2:0]="010", cin='1'            -> sum="011"
        a <= "010"; b <= "000"; cin <= '1'; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```




## Program_Counter_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;
```

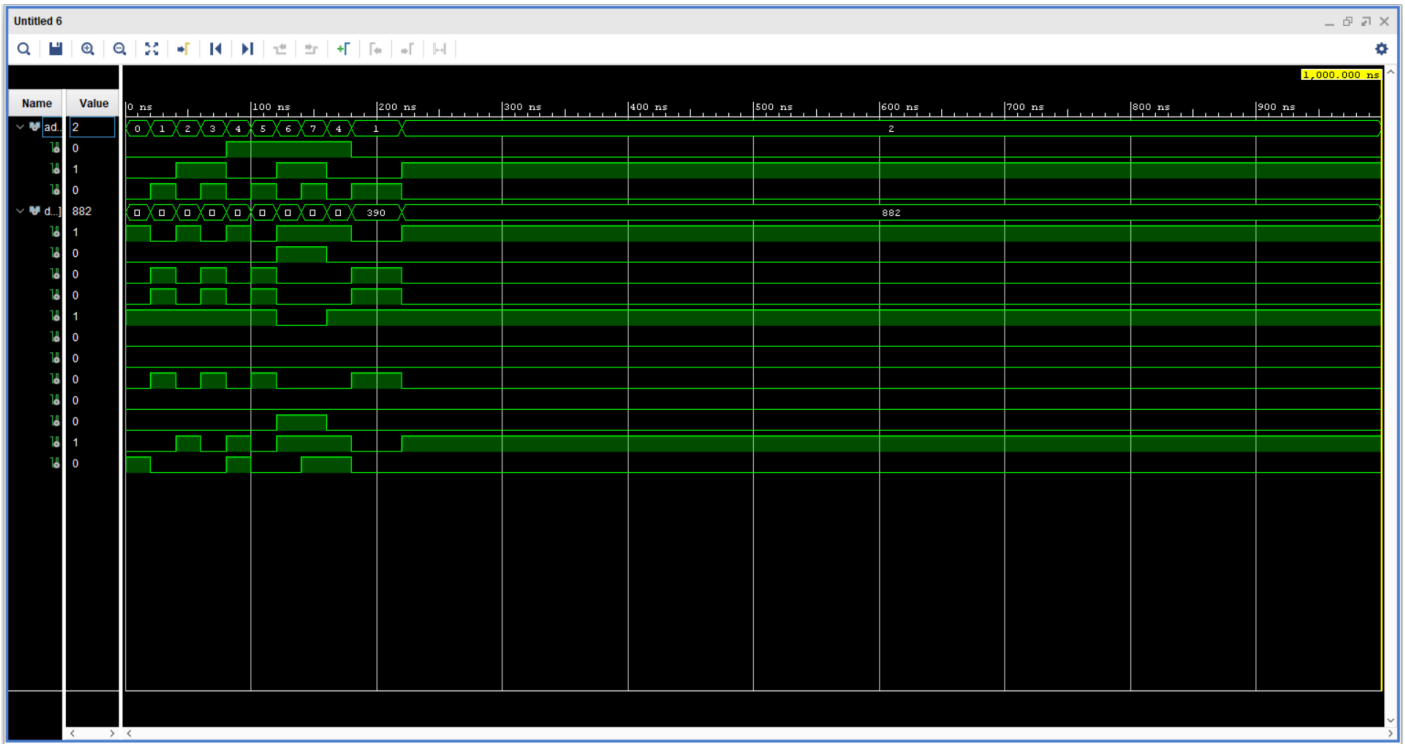
```
entity Program_Counter_TB is
end Program_Counter_TB;

architecture sim of Program_Counter_TB is
    component program_counter
        port (clk   : in  std_logic;
              reset : in  std_logic;
              d     : in  std_logic_vector(2 downto 0);
              q     : out std_logic_vector(2 downto 0));
    end component;

    signal clk, reset : std_logic := '0';
    signal d, q       : std_logic_vector(2 downto 0) := (others => '0');
    constant T : time := 10 ns;
begin
    uut : program_counter port map (clk, reset, d, q);

    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 2*T;
        reset <= '0';

        -- load sequential addresses
        d <= "001"; wait for T; -- q="001"
        d <= "010"; wait for T; -- q="010"
        d <= "011"; wait for T; -- q="011"

        -- 240180: d=bits[2:0]="100"
        d <= "100"; wait for T; -- 240180, q="100"
        -- 240225: d=bits[2:0]="001"
        d <= "001"; wait for T; -- 240225, q="001"
        -- 240497: d=bits[2:0]="001"
        d <= "001"; wait for T; -- 240497, q="001"
        -- 240498: d=bits[2:0]="010"
        d <= "010"; wait for T; -- 240498, q="010"

        -- verify reset clears to 0
        reset <= '1'; wait for 2*T;
        reset <= '0';

        wait;
    end process;
end sim;
```




## Program_ROM_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Program_ROM_TB is
end Program_ROM_TB;

architecture sim of Program_ROM_TB is
    component program_rom
        port (addr : in  std_logic_vector(2 downto 0);
              data : out std_logic_vector(11 downto 0));
    end component;

    signal addr : std_logic_vector(2 downto 0) := "000";
    signal data : std_logic_vector(11 downto 0);
begin
    uut : program_rom port map (addr, data);

    stim : process
    begin
        -- sweep all 8 addresses
        addr <= "000"; wait for 20 ns; -- MOVI R1,3  -> "100001000011"
        addr <= "001"; wait for 20 ns; -- MOVI R2,1  -> "100010000001"
        addr <= "010"; wait for 20 ns; -- NEG  R2    -> "010010000000"
        addr <= "011"; wait for 20 ns; -- ADD  R7,R1 -> "001110010000"
        addr <= "100"; wait for 20 ns; -- ADD  R1,R2 -> "000010100000"
        addr <= "101"; wait for 20 ns; -- JZR  R1,7  -> "110010000111"
        addr <= "110"; wait for 20 ns; -- JZR  R0,3  -> "110000000011"
        addr <= "111"; wait for 20 ns; -- JZR  R0,7  -> "110000000111"

        -- 240180: addr=bits[2:0]="100" -> "000010100000" (ADD R1,R2)
        addr <= "100"; wait for 20 ns; -- 240180

        -- 240225: addr=bits[2:0]="001" -> "100010000001" (MOVI R2,1)
        addr <= "001"; wait for 20 ns; -- 240225

        -- 240497: addr=bits[2:0]="001" -> "100010000001" (MOVI R2,1)
        addr <= "001"; wait for 20 ns; -- 240497
```

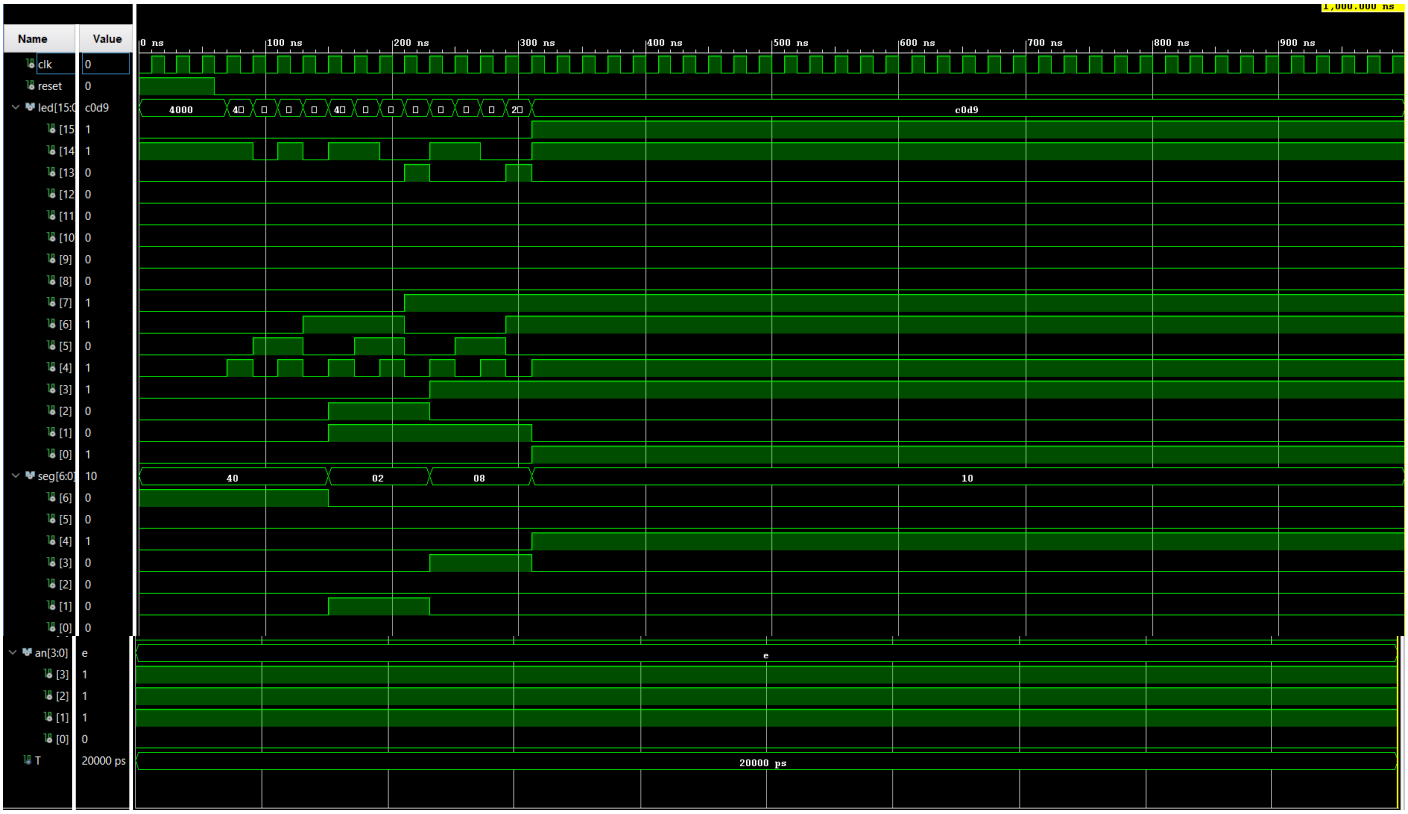
```
        -- 240498: addr=bits[2:0]="010" -> "010010000000" (NEG R2)
        addr <= "010"; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```



## D.3 Extended Nano Processor Testbenches

### NanoProcessor_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity NanoProcessor_TB is
end NanoProcessor_TB;

-- Expected final state: R7="1001" (9), LD14=zero flag, LD15=jump active
-- Program halts at PC=15 (JMP 15 loop)

architecture sim of NanoProcessor_TB is
    component nanoprocessor_top_v2
        generic (SIM_MODE : boolean; DIVISOR : integer);
        port (clk, reset : in  std_logic;
              led        : out std_logic_vector(15 downto 0);
              seg        : out std_logic_vector(6 downto 0);
              an         : out std_logic_vector(3 downto 0));
    end component;

    signal clk, reset : std_logic := '0';
    signal led        : std_logic_vector(15 downto 0);
    signal seg        : std_logic_vector(6 downto 0);
    signal an         : std_logic_vector(3 downto 0);
    constant T : time := 20 ns;
begin
    uut : nanoprocessor_top_v2
        generic map (SIM_MODE => true, DIVISOR => 4)
        port map (clk, reset, led, seg, an);

    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 3*T;
        reset <= '0';

        wait;
    end process;
end sim;
```

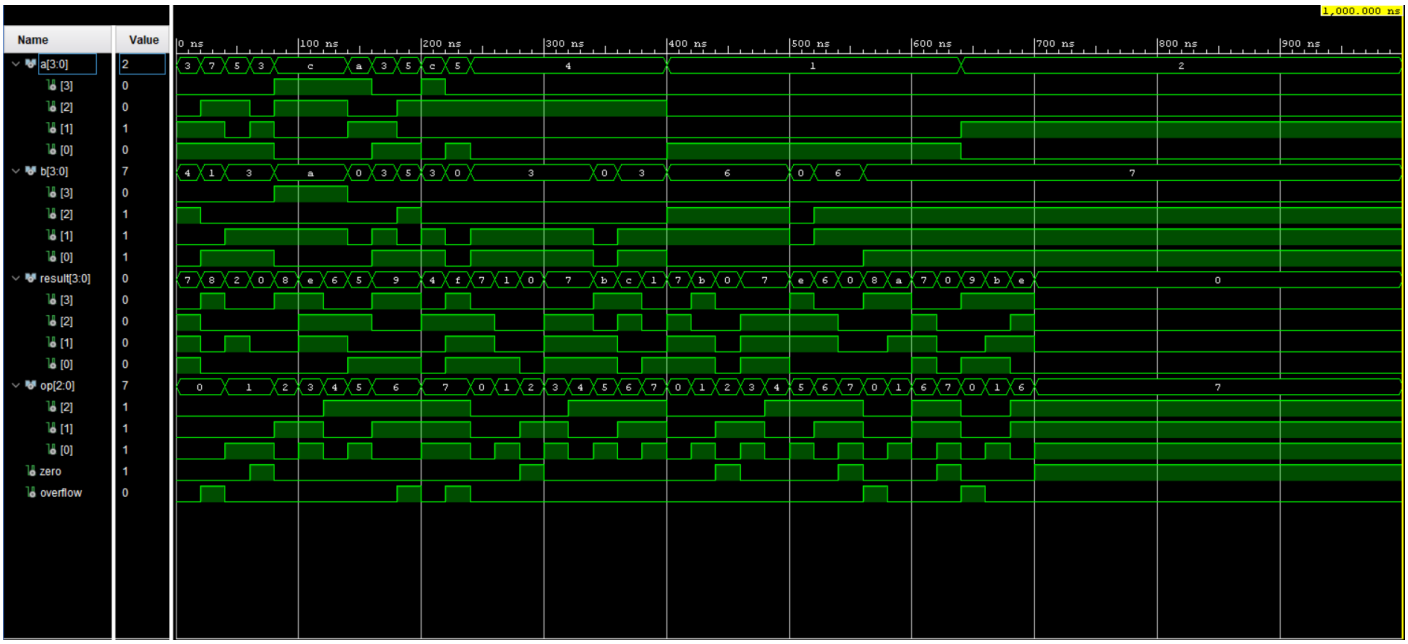



## ALU_4bit_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity ALU_4bit_TB is
end ALU_4bit_TB;

architecture sim of ALU_4bit_TB is
    component alu_4bit
        port (a, b   : in  std_logic_vector(3 downto 0);
              op     : in  std_logic_vector(2 downto 0);
              result : out std_logic_vector(3 downto 0);
              zero, overflow : out std_logic);
    end component;

    signal a, b, result : std_logic_vector(3 downto 0) := (others => '0');
    signal op           : std_logic_vector(2 downto 0) := (others => '0');
    signal zero, overflow : std_logic;
begin
    uut : alu_4bit port map (a, b, op, result, zero, overflow);

    stim : process
    begin
        -- ADD (op="000")
        a <= "0011"; b <= "0100"; op <= "000"; wait for 20 ns; -- 3+4=7
        a <= "0111"; b <= "0001"; op <= "000"; wait for 20 ns; -- 7+1=8
overflow

        -- SUB (op="001")
        a <= "0101"; b <= "0011"; op <= "001"; wait for 20 ns; -- 5-3=2
        a <= "0011"; b <= "0011"; op <= "001"; wait for 20 ns; -- 3-3=0 zero=1

        -- AND (op="010")
```

```
        a <= "1100"; b <= "1010"; op <= "010"; wait for 20 ns; -- 1100 AND 1010
= 1000

        -- OR  (op="011")
        a <= "1100"; b <= "1010"; op <= "011"; wait for 20 ns; -- 1100 OR 1010
= 1110

        -- XOR (op="100")
        a <= "1100"; b <= "1010"; op <= "100"; wait for 20 ns; -- 1100 XOR 1010
= 0110

        -- NOT (op="101")
        a <= "1010"; b <= "0000"; op <= "101"; wait for 20 ns; -- NOT 1010 =
0101

        -- MUL (op="110")
        a <= "0011"; b <= "0011"; op <= "110"; wait for 20 ns; -- 3*3=9
        a <= "0101"; b <= "0101"; op <= "110"; wait for 20 ns; -- 5*5=25
overflow

        -- DIV (op="111")
        a <= "1100"; b <= "0011"; op <= "111"; wait for 20 ns; -- 12/3=4
        a <= "0101"; b <= "0000"; op <= "111"; wait for 20 ns; -- div by zero
ovf=1

        -- 240180: a=bits[3:0]="0100", b=bits[7:4]="0011"
        a <= "0100"; b <= "0011"; op <= "000"; wait for 20 ns; -- ADD 4+3=7
        a <= "0100"; b <= "0011"; op <= "001"; wait for 20 ns; -- SUB 4-3=1
        a <= "0100"; b <= "0011"; op <= "010"; wait for 20 ns; -- AND
0100&0011=0000 zero=1
        a <= "0100"; b <= "0011"; op <= "011"; wait for 20 ns; -- OR
0100|0011=0111
        a <= "0100"; b <= "0011"; op <= "100"; wait for 20 ns; -- XOR
0100^0011=0111
        a <= "0100"; b <= "0000"; op <= "101"; wait for 20 ns; -- NOT 0100=1011
        a <= "0100"; b <= "0011"; op <= "110"; wait for 20 ns; -- MUL 4*3=12
        a <= "0100"; b <= "0011"; op <= "111"; wait for 20 ns; -- DIV 4/3=1

        -- 240225: a=bits[3:0]="0001", b=bits[7:4]="0110"
        a <= "0001"; b <= "0110"; op <= "000"; wait for 20 ns; -- ADD 1+6=7
        a <= "0001"; b <= "0110"; op <= "001"; wait for 20 ns; -- SUB 1-6=-5
        a <= "0001"; b <= "0110"; op <= "010"; wait for 20 ns; -- AND
0001&0110=0000 zero=1
        a <= "0001"; b <= "0110"; op <= "011"; wait for 20 ns; -- OR
0001|0110=0111
        a <= "0001"; b <= "0110"; op <= "100"; wait for 20 ns; -- XOR
0001^0110=0111
        a <= "0001"; b <= "0000"; op <= "101"; wait for 20 ns; -- NOT 0001=1110
        a <= "0001"; b <= "0110"; op <= "110"; wait for 20 ns; -- MUL 1*6=6
        a <= "0001"; b <= "0110"; op <= "111"; wait for 20 ns; -- DIV 1/6=0
zero=1

        -- 240497: a=bits[3:0]="0001", b=bits[7:4]="0111"
        a <= "0001"; b <= "0111"; op <= "000"; wait for 20 ns; -- ADD 1+7=8
overflow
        a <= "0001"; b <= "0111"; op <= "001"; wait for 20 ns; -- SUB 1-7=-6
        a <= "0001"; b <= "0111"; op <= "110"; wait for 20 ns; -- MUL 1*7=7
        a <= "0001"; b <= "0111"; op <= "111"; wait for 20 ns; -- DIV 1/7=0
zero=1

        -- 240498: a=bits[3:0]="0010", b=bits[7:4]="0111"
        a <= "0010"; b <= "0111"; op <= "000"; wait for 20 ns; -- ADD 2+7=9
overflow
        a <= "0010"; b <= "0111"; op <= "001"; wait for 20 ns; -- SUB 2-7=-5
        a <= "0010"; b <= "0111"; op <= "110"; wait for 20 ns; -- MUL 2*7=14
```

```
        a <= "0010"; b <= "0111"; op <= "111"; wait for 20 ns; -- DIV 2/7=0
zero=1

        wait;
    end process;
end sim;
```

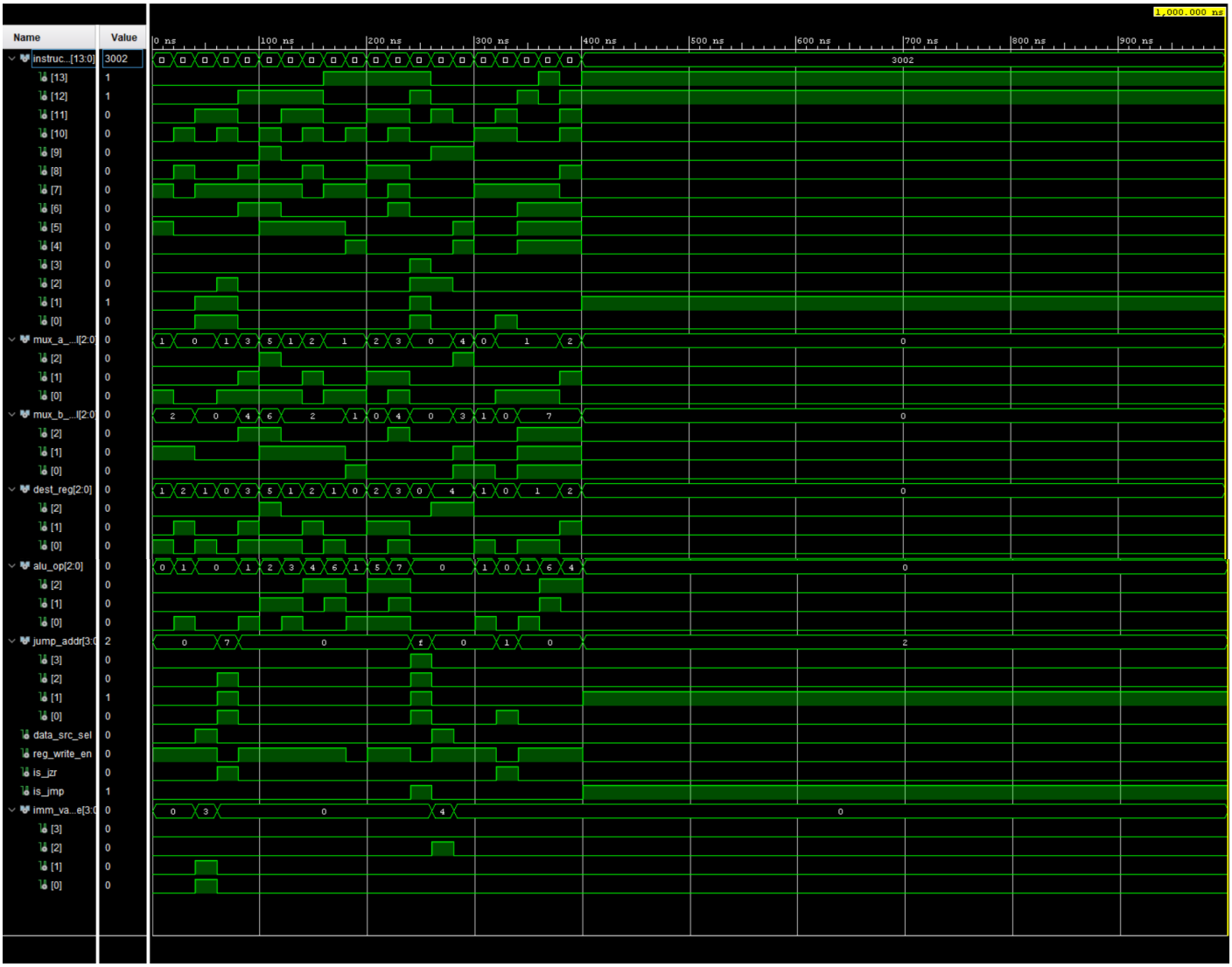



## Instruction_Decoder_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Instruction_Decoder_TB is
end Instruction_Decoder_TB;

-- Instruction format (14 bits): [13:10]=opcode [9:7]=Ra [6:4]=Rb
[3:0]=imm/jump
-- Opcodes: 0000=ADD 0001=NEG 0010=MOVI 0011=JZR 0100=SUB 0101=AND
--          0110=OR  0111=XOR 1000=MUL  1001=CMP  1010=NOT 1011=DIV 1100=JMP

architecture sim of Instruction_Decoder_TB is
    component instruction_decoder_v2
        port (instruction  : in  std_logic_vector(13 downto 0);
              mux_a_sel    : out std_logic_vector(2 downto 0);
              mux_b_sel    : out std_logic_vector(2 downto 0);
              alu_op       : out std_logic_vector(2 downto 0);
              data_src_sel : out std_logic;
              imm_value    : out std_logic_vector(3 downto 0);
              dest_reg     : out std_logic_vector(2 downto 0);
              reg_write_en : out std_logic;
              jump_addr    : out std_logic_vector(3 downto 0);
              is_jzr       : out std_logic;
              is_jmp       : out std_logic);
    end component;

    signal instruction  : std_logic_vector(13 downto 0) := (others => '0');
    signal mux_a_sel, mux_b_sel, dest_reg : std_logic_vector(2 downto 0);
    signal alu_op       : std_logic_vector(2 downto 0);
    signal jump_addr    : std_logic_vector(3 downto 0);
    signal data_src_sel, reg_write_en, is_jzr, is_jmp : std_logic;
    signal imm_value    : std_logic_vector(3 downto 0);
```

```
begin
    uut : instruction_decoder_v2
        port map (instruction, mux_a_sel, mux_b_sel, alu_op, data_src_sel,
                  imm_value, dest_reg, reg_write_en, jump_addr, is_jzr,
is_jmp);

    stim : process
    begin
        -- ADD  R1, R2 -> 0000 001 010 0000
        instruction <= "000000010100000"; wait for 20 ns;
        -- NEG  R2     -> 0001 010 000 0000
        instruction <= "000101000000000"; wait for 20 ns;
        -- MOVI R1, 3  -> 0010 001 000 0011
        instruction <= "001000010000011"; wait for 20 ns;
        -- JZR  R1, 7  -> 0011 001 000 0111
        instruction <= "001100010000111"; wait for 20 ns;
        -- SUB  R3, R4 -> 0100 011 100 0000
        instruction <= "010001110000000"; wait for 20 ns;
        -- AND  R5, R6 -> 0101 101 110 0000
        instruction <= "010110111000000"; wait for 20 ns;
        -- OR   R1, R2 -> 0110 001 010 0000
        instruction <= "011000010100000"; wait for 20 ns;
        -- XOR  R2, R2 -> 0111 010 010 0000
        instruction <= "011101001000000"; wait for 20 ns;
        -- MUL  R1, R2 -> 1000 001 010 0000
        instruction <= "100000101000000"; wait for 20 ns;
        -- CMP  R1, R1 -> 1001 001 001 0000
        instruction <= "100100100100000"; wait for 20 ns;
        -- NOT  R2     -> 1010 010 000 0000
        instruction <= "101001000000000"; wait for 20 ns;
        -- DIV  R3, R4 -> 1011 011 100 0000
        instruction <= "101101110000000"; wait for 20 ns;
        -- JMP  15     -> 1100 000 000 1111
        instruction <= "110000000001111"; wait for 20 ns;

        -- 240180: R=bits[2:0]="100"(R4), Rb=bits[6:4]="011"(R3),
d=bits[3:0]="0100"(4)
        -- MOVI R4, 4  -> 0010 100 000 0100
        instruction <= "001010000000100"; wait for 20 ns; -- 240180
        -- ADD  R4, R3 -> 0000 100 011 0000
        instruction <= "000010001100000"; wait for 20 ns; -- 240180

        -- 240225: R=bits[2:0]="001"(R1), Rb=bits[6:4]="110"(R6),
d=bits[3:0]="0001"(1)
        -- NEG  R1     -> 0001 001 000 0000
        instruction <= "000100100000000"; wait for 20 ns; -- 240225
        -- JZR  R1, 1  -> 0011 001 000 0001
        instruction <= "001100100000001"; wait for 20 ns; -- 240225

        -- 240497: R=bits[2:0]="001"(R1), Rb=bits[6:4]="111"(R7)
        -- SUB  R1, R7 -> 0100 001 111 0000
        instruction <= "010000111100000"; wait for 20 ns; -- 240497
        -- MUL  R1, R7 -> 1000 001 111 0000
        instruction <= "100000111100000"; wait for 20 ns; -- 240497

        -- 240498: R=bits[2:0]="010"(R2), Rb=bits[6:4]="111"(R7),
d=bits[3:0]="0010"(2)
        -- XOR  R2, R7 -> 0111 010 111 0000
        instruction <= "011101011100000"; wait for 20 ns; -- 240498
        -- JMP  2      -> 1100 000 000 0010
        instruction <= "110000000000010"; wait for 20 ns; -- 240498

        wait;
    end process;
```

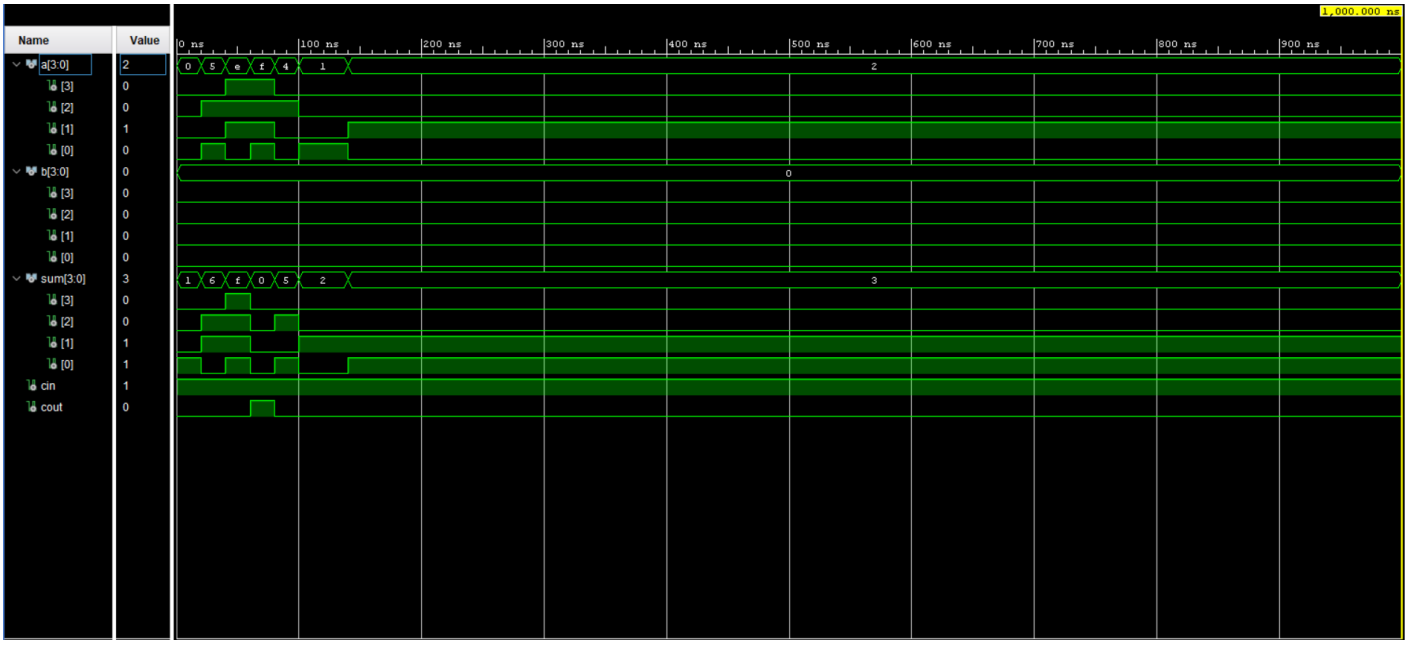
```
end sim;
```




## PC_Adder_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity PC_Adder_TB is
end PC_Adder_TB;

architecture sim of PC_Adder_TB is
    component adder_4bit
        port (a, b : in  std_logic_vector(3 downto 0);
              cin  : in  std_logic;
              sum  : out std_logic_vector(3 downto 0);
              cout : out std_logic);
    end component;

    signal a, b, sum : std_logic_vector(3 downto 0) := (others => '0');
    signal cin, cout : std_logic := '0';
begin
    uut : adder_4bit port map (a, b, cin, sum, cout);

    stim : process
    begin
```

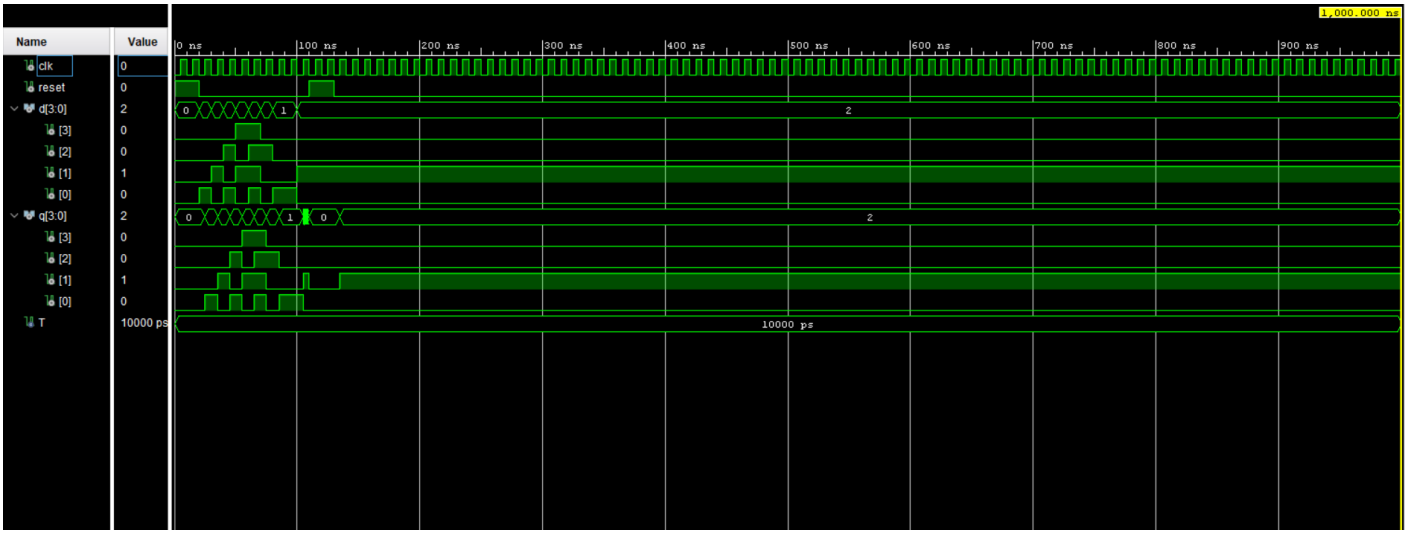
```
        -- PC+1 behaviour (b="0000", cin='1')
        a <= "0000"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 0+1=1
        a <= "0101"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 5+1=6
        a <= "1110"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 14+1=15
        a <= "1111"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 15+1=0 wrap
cout=1

        -- 240180: a=bits[3:0]="0100", b="0000", cin='1' -> sum="0101"
        a <= "0100"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 240180
        -- 240225: a=bits[3:0]="0001", cin='1'           -> sum="0010"
        a <= "0001"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 240225
        -- 240497: a=bits[3:0]="0001", cin='1'           -> sum="0010"
        a <= "0001"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 240497
        -- 240498: a=bits[3:0]="0010", cin='1'           -> sum="0011"
        a <= "0010"; b <= "0000"; cin <= '1'; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```




## Program_Counter_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Program_Counter_TB is
end Program_Counter_TB;

architecture sim of Program_Counter_TB is
    component program_counter_v2
        port (clk, reset : in  std_logic;
              d          : in  std_logic_vector(3 downto 0);
              q          : out std_logic_vector(3 downto 0));
    end component;

    signal clk, reset : std_logic := '0';
    signal d, q       : std_logic_vector(3 downto 0) := (others => '0');
    constant T : time := 10 ns;
begin
    uut : program_counter_v2 port map (clk, reset, d, q);
```

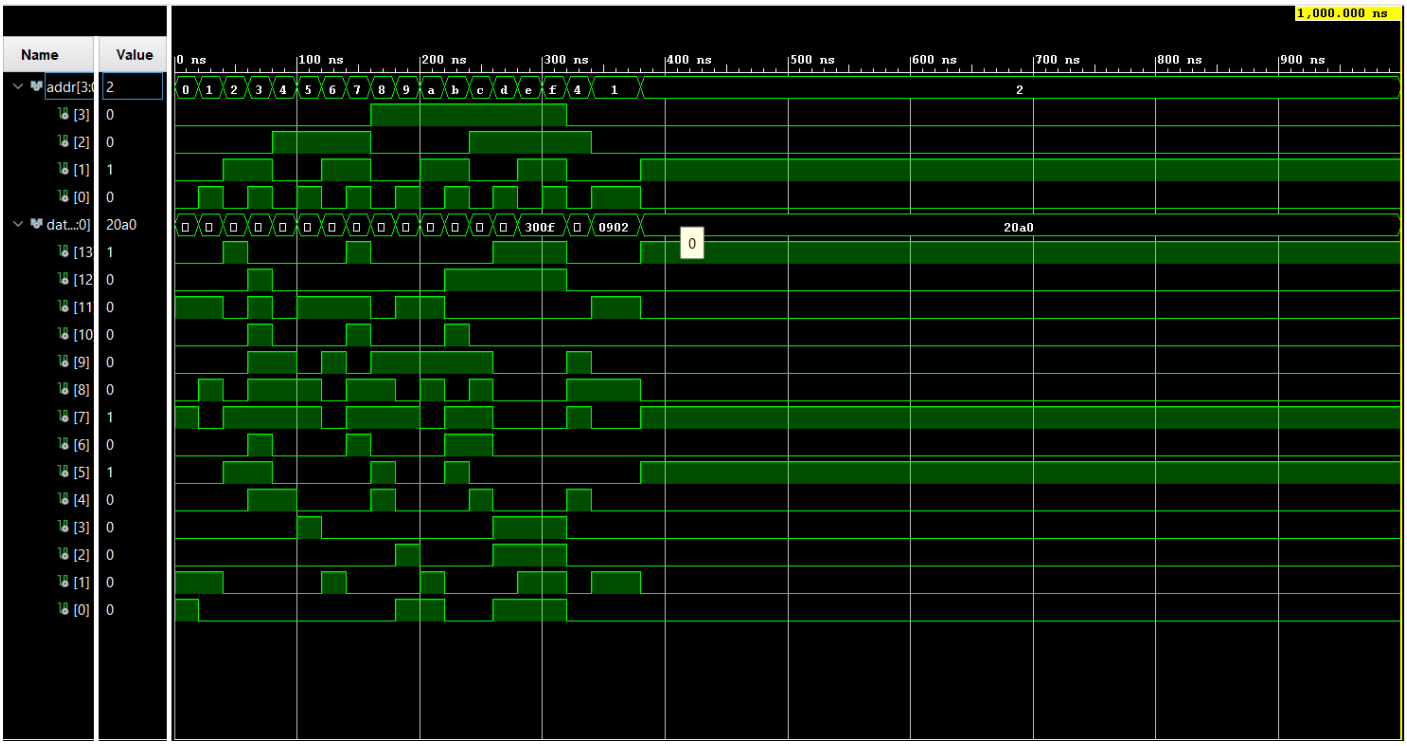
```
    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 2*T;
        reset <= '0';

        d <= "0001"; wait for T; -- q="0001"
        d <= "0010"; wait for T; -- q="0010"
        d <= "0101"; wait for T; -- q="0101"
        d <= "1010"; wait for T; -- q="1010"
        d <= "1111"; wait for T; -- q="1111"

        -- 240180: d=bits[3:0]="0100"
        d <= "0100"; wait for T; -- 240180 q="0100"
        -- 240225: d=bits[3:0]="0001"
        d <= "0001"; wait for T; -- 240225 q="0001"
        -- 240497: d=bits[3:0]="0001"
        d <= "0001"; wait for T; -- 240497 q="0001"
        -- 240498: d=bits[3:0]="0010"
        d <= "0010"; wait for T; -- 240498 q="0010"

        -- reset clears to 0000
        reset <= '1'; wait for 2*T;
        reset <= '0';

        wait;
    end process;
end sim;
```




## Program_ROM_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Program_ROM_TB is
end Program_ROM_TB;


architecture sim of Program_ROM_TB is
    component program_rom_v2
```

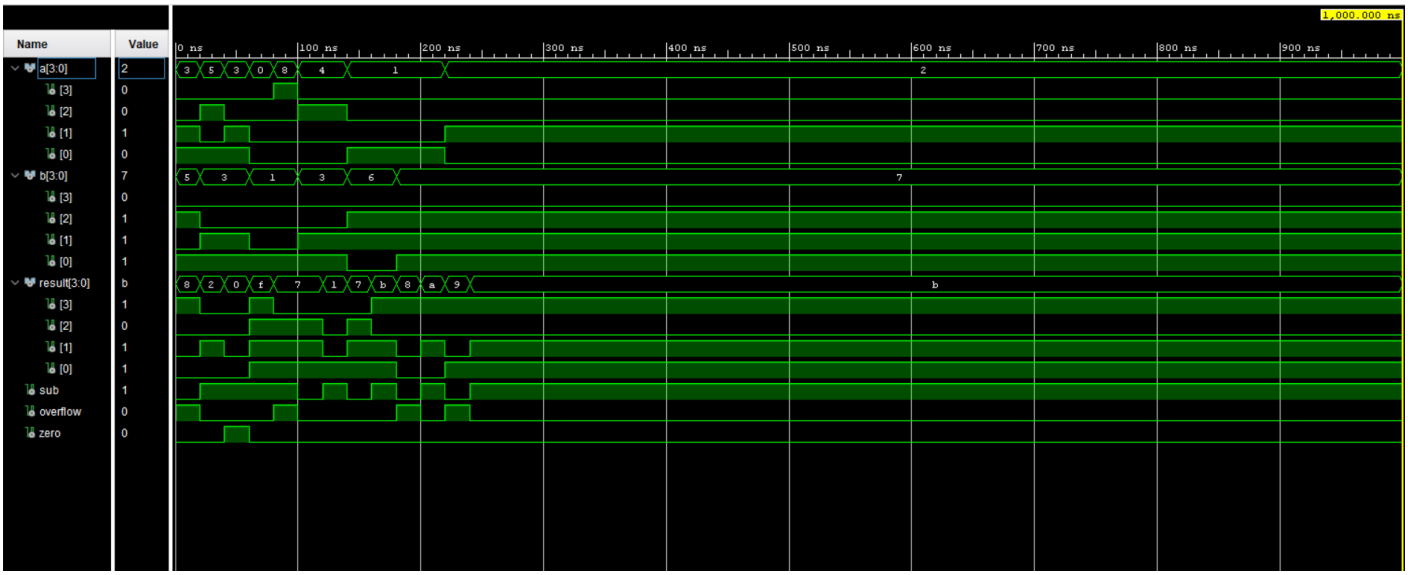
```
        port (addr : in  std_logic_vector(3 downto 0);
              data : out std_logic_vector(13 downto 0));
    end component;

    signal addr : std_logic_vector(3 downto 0) := "0000";
    signal data : std_logic_vector(13 downto 0);
begin
    uut : program_rom_v2 port map (addr, data);

    stim : process
    begin
        -- sweep all 16 addresses
        addr <= "0000"; wait for 20 ns;
        addr <= "0001"; wait for 20 ns;
        addr <= "0010"; wait for 20 ns;
        addr <= "0011"; wait for 20 ns;
        addr <= "0100"; wait for 20 ns;
        addr <= "0101"; wait for 20 ns;
        addr <= "0110"; wait for 20 ns;
        addr <= "0111"; wait for 20 ns;
        addr <= "1000"; wait for 20 ns;
        addr <= "1001"; wait for 20 ns;
        addr <= "1010"; wait for 20 ns;
        addr <= "1011"; wait for 20 ns;
        addr <= "1100"; wait for 20 ns;
        addr <= "1101"; wait for 20 ns;
        addr <= "1110"; wait for 20 ns;
        addr <= "1111"; wait for 20 ns;

        -- 240180: addr=bits[3:0]="0100" -> MOVI R4,2
        addr <= "0100"; wait for 20 ns; -- 240180
        -- 240225: addr=bits[3:0]="0001" -> MOVI R2,2
        addr <= "0001"; wait for 20 ns; -- 240225
        -- 240497: addr=bits[3:0]="0001" -> MOVI R2,2
        addr <= "0001"; wait for 20 ns; -- 240497
        -- 240498: addr=bits[3:0]="0010" -> MUL R1,R2
        addr <= "0010"; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```




## Add_Sub_4_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Add_Sub_4_TB is
end Add_Sub_4_TB;

architecture sim of Add_Sub_4_TB is
    component add_sub_4bit
        port (a, b   : in  std_logic_vector(3 downto 0);
              sub    : in  std_logic;
              result : out std_logic_vector(3 downto 0);
              overflow, zero : out std_logic);
    end component;

    signal a, b, result : std_logic_vector(3 downto 0) := (others => '0');
    signal sub          : std_logic := '0';
    signal overflow, zero : std_logic;
begin
    uut : add_sub_4bit port map (a, b, sub, result, overflow, zero);

    stim : process
    begin
        a <= "0011"; b <= "0101"; sub <= '0'; wait for 20 ns; -- 3+5=8 overflow
        a <= "0101"; b <= "0011"; sub <= '1'; wait for 20 ns; -- 5-3=2
        a <= "0011"; b <= "0011"; sub <= '1'; wait for 20 ns; -- 3-3=0 zero=1
        a <= "0000"; b <= "0001"; sub <= '1'; wait for 20 ns; -- 0-1=-1
        a <= "1000"; b <= "0001"; sub <= '1'; wait for 20 ns; -- -8-1 overflow

        -- 240180: a=bits[3:0]="0100", b=bits[7:4]="0011"
        a <= "0100"; b <= "0011"; sub <= '0'; wait for 20 ns; -- 4+3=7
        a <= "0100"; b <= "0011"; sub <= '1'; wait for 20 ns; -- 4-3=1

        -- 240225: a=bits[3:0]="0001", b=bits[7:4]="0110"
        a <= "0001"; b <= "0110"; sub <= '0'; wait for 20 ns; -- 1+6=7
        a <= "0001"; b <= "0110"; sub <= '1'; wait for 20 ns; -- 1-6=-5
```

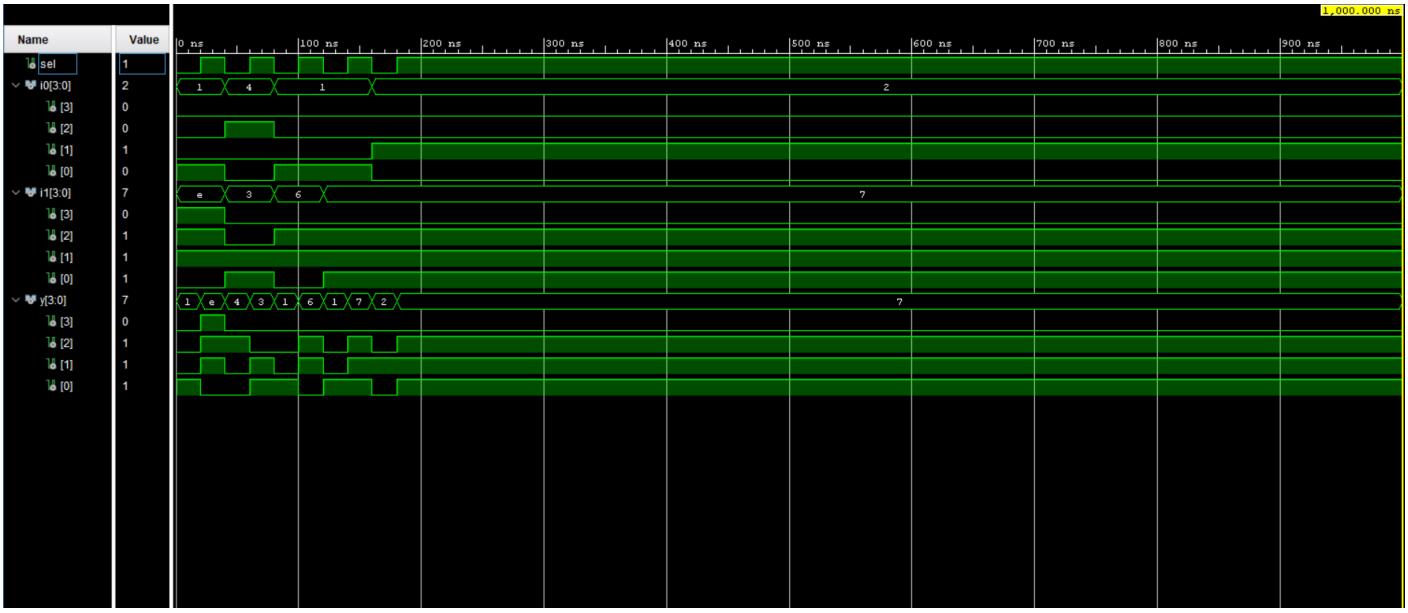
```
        -- 240497: a=bits[3:0]="0001", b=bits[7:4]="0111"
        a <= "0001"; b <= "0111"; sub <= '0'; wait for 20 ns; -- 1+7=8 overflow
        a <= "0001"; b <= "0111"; sub <= '1'; wait for 20 ns; -- 1-7=-6

        -- 240498: a=bits[3:0]="0010", b=bits[7:4]="0111"
        a <= "0010"; b <= "0111"; sub <= '0'; wait for 20 ns; -- 2+7=9 overflow
        a <= "0010"; b <= "0111"; sub <= '1'; wait for 20 ns; -- 2-7=-5

        wait;
    end process;
end sim;
```




## Address_Selector_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Address_Selector_TB is
end Address_Selector_TB;

architecture sim of Address_Selector_TB is
    component mux_2way_4bit
        port (sel    : in  std_logic;
              i0, i1 : in  std_logic_vector(3 downto 0);
              y      : out std_logic_vector(3 downto 0));
    end component;

    signal sel     : std_logic := '0';
    signal i0, i1, y : std_logic_vector(3 downto 0) := (others => '0');
begin
    uut : mux_2way_4bit port map (sel, i0, i1, y);

    stim : process
    begin
        i0 <= "0001"; i1 <= "1110"; sel <= '0'; wait for 20 ns; -- y=PC+1
        sel <= '1'; wait for 20 ns;                              -- y=jump addr

        -- 240180: i0=bits[3:0]="0100", i1=bits[7:4]="0011"
        i0 <= "0100"; i1 <= "0011"; sel <= '0'; wait for 20 ns; -- 240180
y="0100"
        sel <= '1'; wait for 20 ns;                              -- 240180
y="0011"

        -- 240225: i0=bits[3:0]="0001", i1=bits[7:4]="0110"
        i0 <= "0001"; i1 <= "0110"; sel <= '0'; wait for 20 ns; -- 240225
```

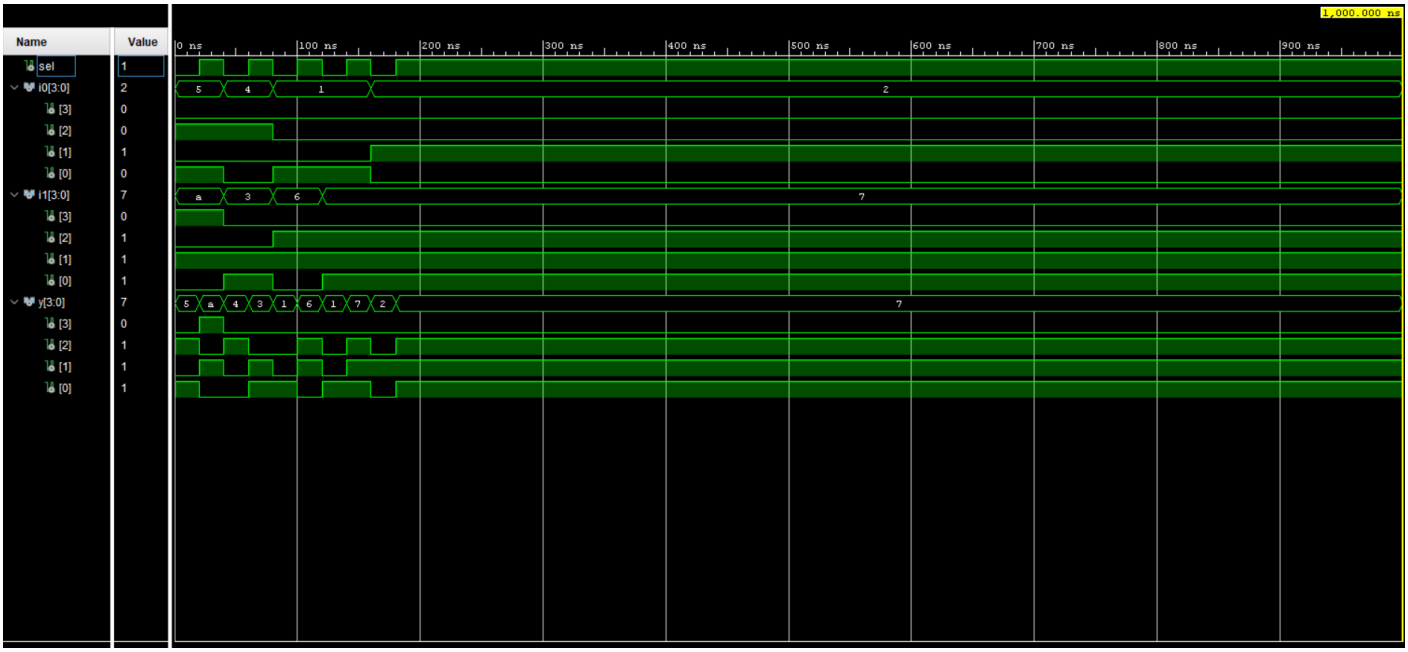
```
        sel <= '1'; wait for 20 ns;

        -- 240497: i0=bits[3:0]="0001", i1=bits[7:4]="0111"
        i0 <= "0001"; i1 <= "0111"; sel <= '0'; wait for 20 ns; -- 240497
        sel <= '1'; wait for 20 ns;

        -- 240498: i0=bits[3:0]="0010", i1=bits[7:4]="0111"
        i0 <= "0010"; i1 <= "0111"; sel <= '0'; wait for 20 ns; -- 240498
        sel <= '1'; wait for 20 ns;

        wait;
    end process;
end sim;
```




## Load_Selector_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Load_Selector_TB is
end Load_Selector_TB;

architecture sim of Load_Selector_TB is
    component mux_2way_4bit
        port (sel    : in  std_logic;
              i0, i1 : in  std_logic_vector(3 downto 0);
              y      : out std_logic_vector(3 downto 0));
    end component;

    signal sel     : std_logic := '0';
    signal i0, i1, y : std_logic_vector(3 downto 0) := (others => '0');
begin
    uut : mux_2way_4bit port map (sel, i0, i1, y);

    stim : process
    begin
        i0 <= "0101"; i1 <= "1010"; sel <= '0'; wait for 20 ns; -- y=ALU result
        sel <= '1'; wait for 20 ns;                              -- y=immediate

        -- 240180: i0=bits[3:0]="0100", i1=bits[7:4]="0011"
        i0 <= "0100"; i1 <= "0011"; sel <= '0'; wait for 20 ns; -- 240180
        sel <= '1'; wait for 20 ns;
```

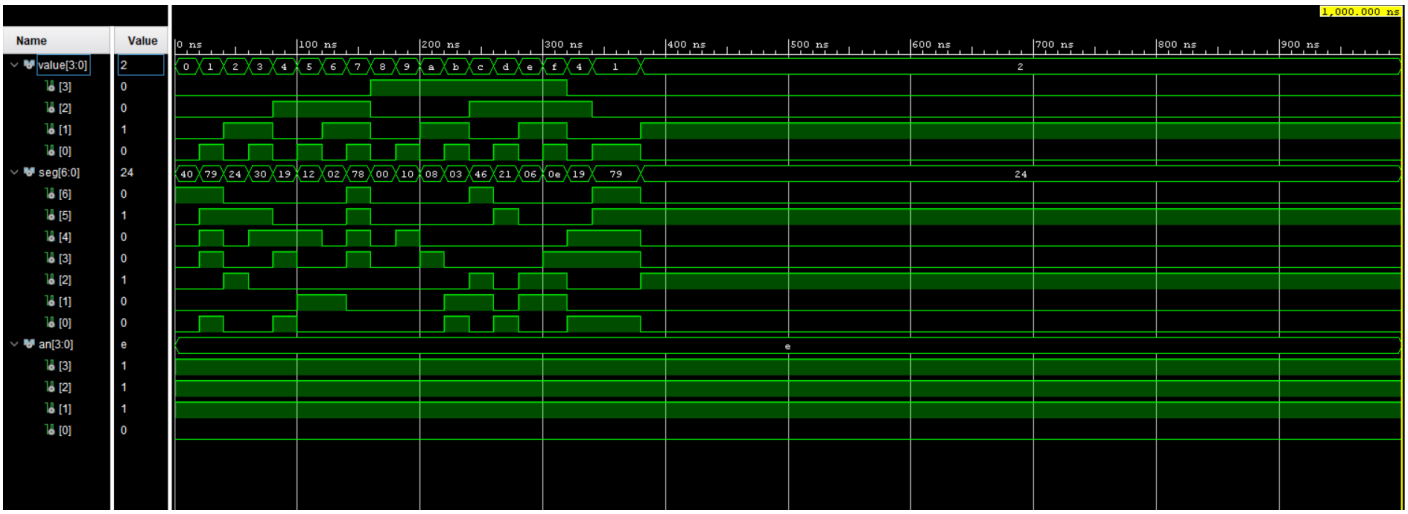
```
        -- 240225: i0=bits[3:0]="0001", i1=bits[7:4]="0110"
        i0 <= "0001"; i1 <= "0110"; sel <= '0'; wait for 20 ns; -- 240225
        sel <= '1'; wait for 20 ns;

        -- 240497: i0=bits[3:0]="0001", i1=bits[7:4]="0111"
        i0 <= "0001"; i1 <= "0111"; sel <= '0'; wait for 20 ns; -- 240497
        sel <= '1'; wait for 20 ns;

        -- 240498: i0=bits[3:0]="0010", i1=bits[7:4]="0111"
        i0 <= "0010"; i1 <= "0111"; sel <= '0'; wait for 20 ns; -- 240498
        sel <= '1'; wait for 20 ns;

        wait;
    end process;
end sim;
```




## LUT_16_7_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity LUT_16_7_TB is
end LUT_16_7_TB;

architecture sim of LUT_16_7_TB is
    component seven_seg_display
        port (value : in  std_logic_vector(3 downto 0);
              seg   : out std_logic_vector(6 downto 0);
              an    : out std_logic_vector(3 downto 0));
    end component;

    signal value : std_logic_vector(3 downto 0) := (others => '0');
    signal seg   : std_logic_vector(6 downto 0);
    signal an    : std_logic_vector(3 downto 0);
begin
    uut : seven_seg_display port map (value, seg, an);

    stim : process
    begin
        value <= "0000"; wait for 20 ns;
        value <= "0001"; wait for 20 ns;
        value <= "0010"; wait for 20 ns;
```

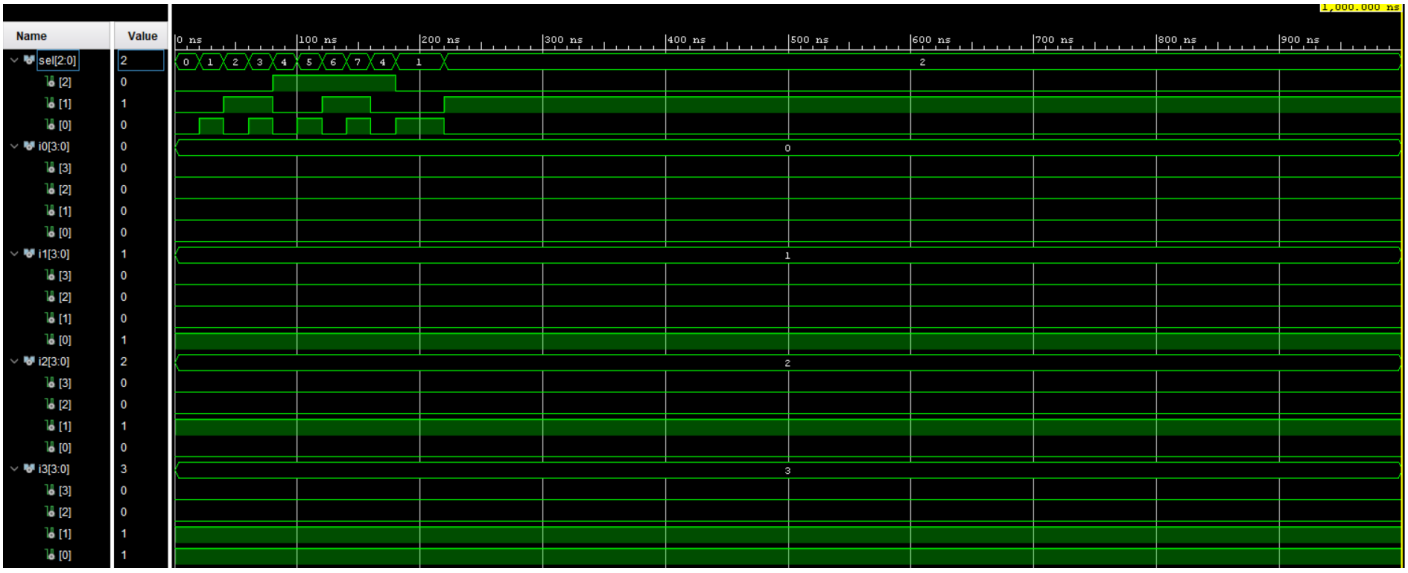
```
        value <= "0011"; wait for 20 ns;
        value <= "0100"; wait for 20 ns;
        value <= "0101"; wait for 20 ns;
        value <= "0110"; wait for 20 ns;
        value <= "0111"; wait for 20 ns;
        value <= "1000"; wait for 20 ns;
        value <= "1001"; wait for 20 ns;
        value <= "1010"; wait for 20 ns;
        value <= "1011"; wait for 20 ns;
        value <= "1100"; wait for 20 ns;
        value <= "1101"; wait for 20 ns;
        value <= "1110"; wait for 20 ns;
        value <= "1111"; wait for 20 ns;

        -- 240180: bits[3:0]="0100"
        value <= "0100"; wait for 20 ns; -- 240180
        -- 240225: bits[3:0]="0001"
        value <= "0001"; wait for 20 ns; -- 240225
        -- 240497: bits[3:0]="0001"
        value <= "0001"; wait for 20 ns; -- 240497
        -- 240498: bits[3:0]="0010"
        value <= "0010"; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```

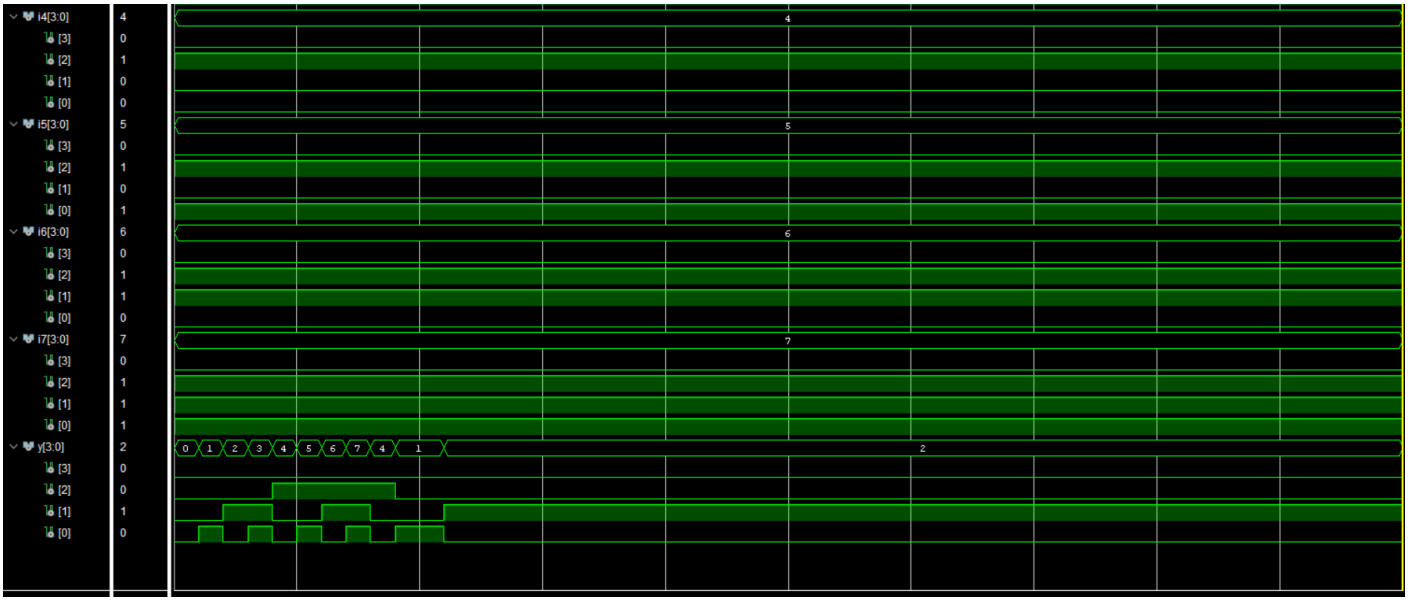



## RegisterData_Multiplexer_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity RegisterData_Multiplexer_TB is
end RegisterData_Multiplexer_TB;

architecture sim of RegisterData_Multiplexer_TB is
    component mux_8way_4bit
        port (sel            : in  std_logic_vector(2 downto 0);
              i0, i1, i2, i3 : in  std_logic_vector(3 downto 0);
              i4, i5, i6, i7 : in  std_logic_vector(3 downto 0);
              y              : out std_logic_vector(3 downto 0));
    end component;

    signal sel             : std_logic_vector(2 downto 0) := "000";
    signal i0, i1, i2, i3 : std_logic_vector(3 downto 0) := (others => '0');
```

```
    signal i4, i5, i6, i7 : std_logic_vector(3 downto 0) := (others => '0');
    signal y              : std_logic_vector(3 downto 0);
begin
    i0 <= "0000"; -- R0 hardwired 0
    i1 <= "0001"; -- 240225 bits[3:0]
    i2 <= "0010"; -- 240498 bits[3:0]
    i3 <= "0011"; -- general
    i4 <= "0100"; -- 240180 bits[3:0]
    i5 <= "0101"; -- general
    i6 <= "0110"; -- 240225 bits[7:4]
    i7 <= "0111"; -- 240497 bits[7:4]

    uut : mux_8way_4bit port map (sel, i0, i1, i2, i3, i4, i5, i6, i7, y);

    stim : process
    begin
        sel <= "000"; wait for 20 ns; -- y="0000"
        sel <= "001"; wait for 20 ns; -- y="0001"
        sel <= "010"; wait for 20 ns; -- y="0010"
        sel <= "011"; wait for 20 ns; -- y="0011"
        sel <= "100"; wait for 20 ns; -- y="0100"
        sel <= "101"; wait for 20 ns; -- y="0101"
        sel <= "110"; wait for 20 ns; -- y="0110"
        sel <= "111"; wait for 20 ns; -- y="0111"

        -- 240180: sel=bits[2:0]="100" -> y=i4="0100"
        sel <= "100"; wait for 20 ns; -- 240180
        -- 240225: sel=bits[2:0]="001" -> y=i1="0001"
        sel <= "001"; wait for 20 ns; -- 240225
        -- 240497: sel=bits[2:0]="001" -> y=i1="0001"
        sel <= "001"; wait for 20 ns; -- 240497
        -- 240498: sel=bits[2:0]="010" -> y=i2="0010"
        sel <= "010"; wait for 20 ns; -- 240498

        wait;
    end process;
end sim;
```

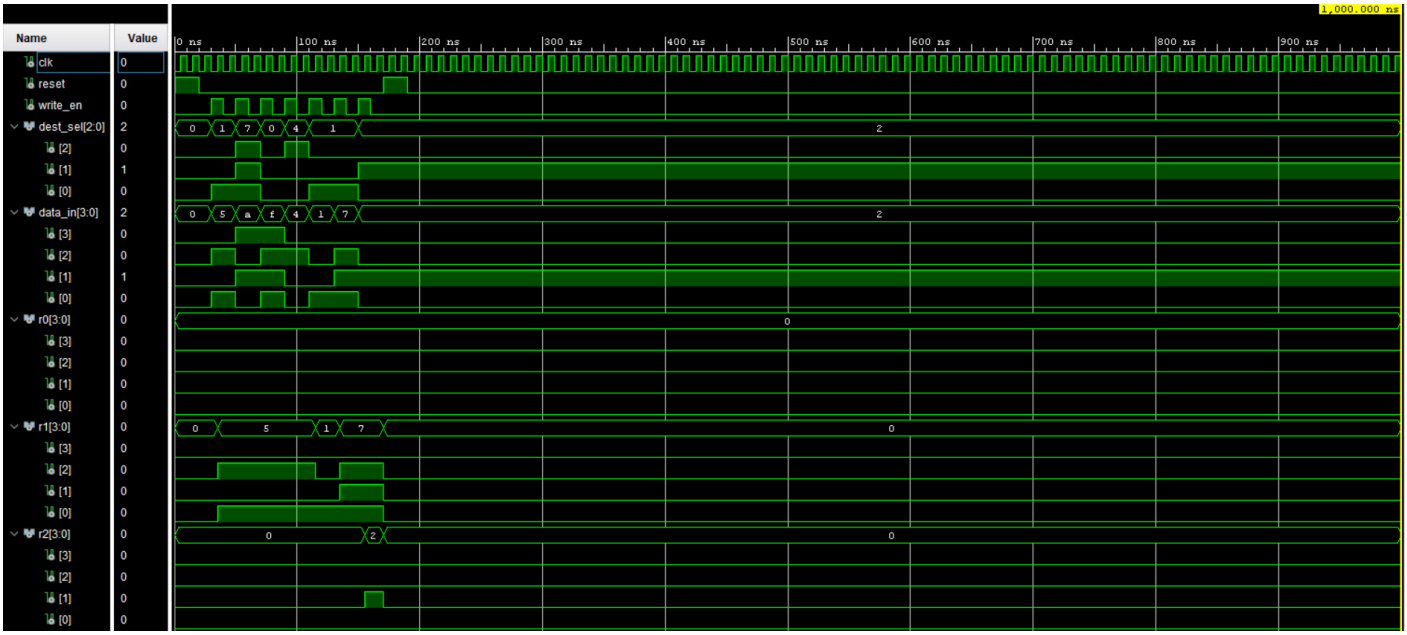


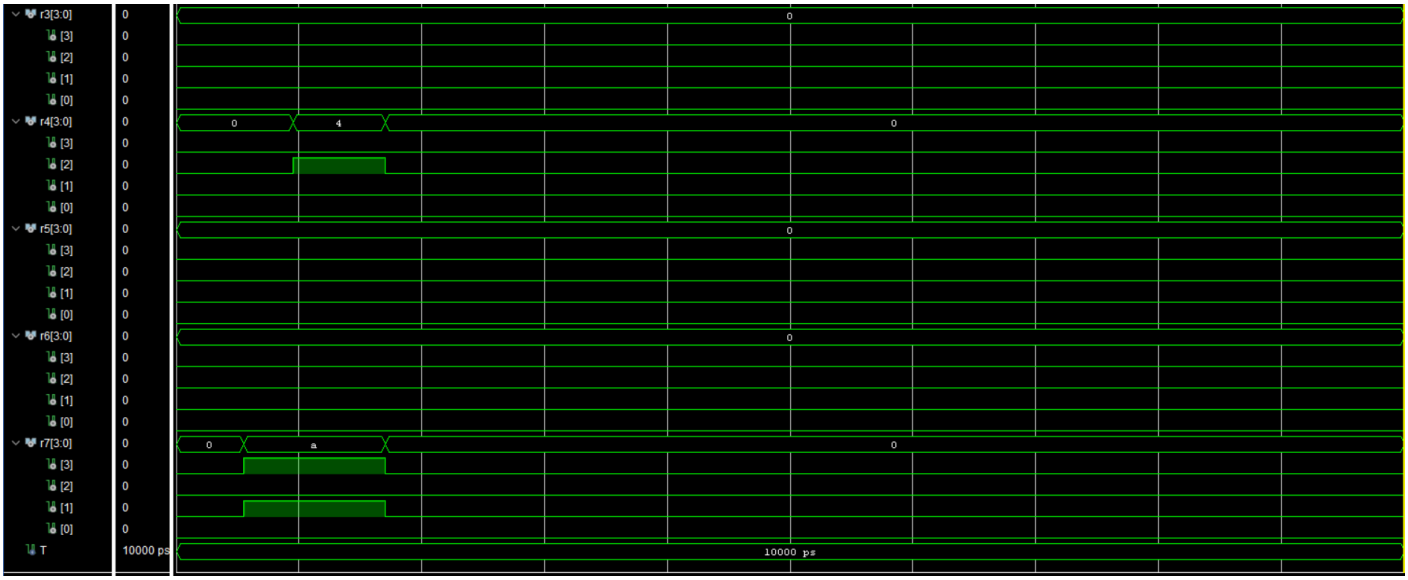



## Register_Bank_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Register_Bank_TB is
end Register_Bank_TB;

architecture sim of Register_Bank_TB is
    component register_bank
        port (clk, reset, write_en : in std_logic;
              dest_sel : in  std_logic_vector(2 downto 0);
              data_in  : in  std_logic_vector(3 downto 0);
              r0, r1, r2, r3 : out std_logic_vector(3 downto 0);
              r4, r5, r6, r7 : out std_logic_vector(3 downto 0));
    end component;

    signal clk, reset, write_en : std_logic := '0';
    signal dest_sel : std_logic_vector(2 downto 0) := "000";
    signal data_in  : std_logic_vector(3 downto 0) := (others => '0');
    signal r0, r1, r2, r3, r4, r5, r6, r7 : std_logic_vector(3 downto 0);
    constant T : time := 10 ns;
begin
    uut : register_bank
        port map (clk, reset, write_en, dest_sel, data_in,
                  r0, r1, r2, r3, r4, r5, r6, r7);

    clk_gen : process
    begin
        clk <= '0'; wait for T/2;
        clk <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 2*T;
        reset <= '0'; wait for T;

        dest_sel <= "001"; data_in <= "0101"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;

        dest_sel <= "111"; data_in <= "1010"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;
```

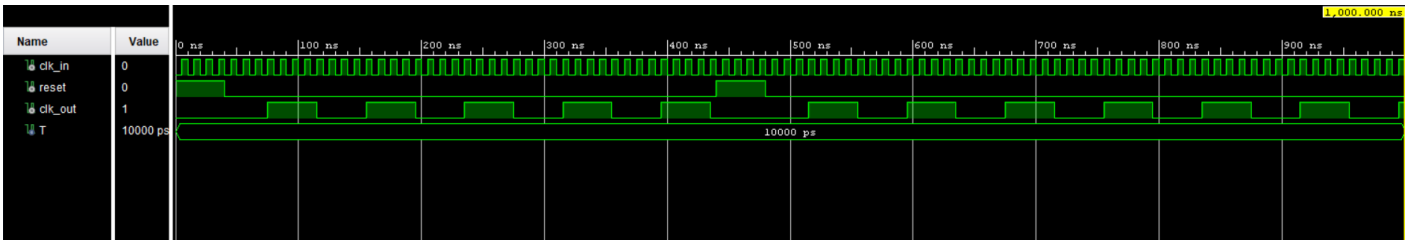
```
        -- attempt write to R0 (hardwired zero, ignored)
        dest_sel <= "000"; data_in <= "1111"; write_en <= '1'; wait for T;
        write_en <= '0'; wait for T;

        -- 240180: data_in=bits[3:0]="0100", dest=bits[2:0]="100"(R4)
        dest_sel <= "100"; data_in <= "0100"; write_en <= '1'; wait for T; --
240180
        write_en <= '0'; wait for T;

        -- 240225: data_in=bits[3:0]="0001", dest=bits[2:0]="001"(R1)
        dest_sel <= "001"; data_in <= "0001"; write_en <= '1'; wait for T; --
240225
        write_en <= '0'; wait for T;

        -- 240497: data_in=bits[7:4]="0111", dest=bits[2:0]="001"(R1)
        dest_sel <= "001"; data_in <= "0111"; write_en <= '1'; wait for T; --
240497
        write_en <= '0'; wait for T;

        -- 240498: data_in=bits[3:0]="0010", dest=bits[2:0]="010"(R2)
        dest_sel <= "010"; data_in <= "0010"; write_en <= '1'; wait for T; --
240498
        write_en <= '0'; wait for T;

        reset <= '1'; wait for 2*T;
        reset <= '0'; wait for T;

        wait;
    end process;
end sim;
```






## Slow_Clk_TB.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity Slow_Clk_TB is
end Slow_Clk_TB;

architecture sim of Slow_Clk_TB is
    component clock_divider
        generic (DIVISOR : integer := 100_000_000);
        port (clk_in  : in  std_logic;
              reset   : in  std_logic;
              clk_out : out std_logic);
    end component;

    signal clk_in, reset, clk_out : std_logic := '0';
    constant T : time := 10 ns;
begin
    uut : clock_divider
        generic map (DIVISOR => 4)
        port map (clk_in, reset, clk_out);

    clk_gen : process
    begin
        clk_in <= '0'; wait for T/2;
        clk_in <= '1'; wait for T/2;
    end process;

    stim : process
    begin
        reset <= '1'; wait for 4*T;
        reset <= '0';

        wait for 40*T;

        reset <= '1'; wait for 4*T;
        reset <= '0';

        wait for 40*T;

        wait;
    end process;
end sim;
```


Name
Value
clk_in
0
reset
0
clk_out
1
T
10000 ps
0 ns
100 ns
200 ns
300 ns
400 ns
500 ns
600 ns
700 ns
800 ns
900 ns
1,000.000 ns
10000 ps

# Appendix E - Constraint Files

## E.1 Basic Constraints File - nanoprocessor.xdc

### nanoprocessor.xdc

```
## --- 100 MHz onboard clock (W5) ---------------------------------------------
-
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

## --- Reset (BTNC, U18) ------------------------------------------------------
-
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

## --- LEDs LD0..LD15 ---------------------------------------------------------
-
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property PACKAGE_PIN V3  [get_ports {led[9]}]
set_property PACKAGE_PIN W3  [get_ports {led[10]}]
set_property PACKAGE_PIN U3  [get_ports {led[11]}]
set_property PACKAGE_PIN P3  [get_ports {led[12]}]
set_property PACKAGE_PIN N3  [get_ports {led[13]}]
set_property PACKAGE_PIN P1  [get_ports {led[14]}]
set_property PACKAGE_PIN L1  [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]

## --- 7-segment display (cathodes a..g, active low) --------------------------
-
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[*]}]

## --- 7-segment digit anodes (active low) ------------------------------------
-
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[*]}]

## --- Configuration ----------------------------------------------------------
-
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

## E.2 Extended Constraints File - nanoprocessor_v2.xdc

### nanoprocessor_v2.xdc

```
## Clock signal (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -name sys_clk_pin -period 10.000 -waveform {0 5} [get_ports clk]

## Reset - centre pushbutton BTNC
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

## LEDs (LD0 .. LD15)
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property PACKAGE_PIN V3  [get_ports {led[9]}]
set_property PACKAGE_PIN W3  [get_ports {led[10]}]
set_property PACKAGE_PIN U3  [get_ports {led[11]}]
set_property PACKAGE_PIN P3  [get_ports {led[12]}]
set_property PACKAGE_PIN N3  [get_ports {led[13]}]
set_property PACKAGE_PIN P1  [get_ports {led[14]}]
set_property PACKAGE_PIN L1  [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]

## 7-segment display cathodes (segments a..g, active low)
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]    ;# a
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]    ;# b
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]    ;# c
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]    ;# d
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]    ;# e
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]    ;# f
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]    ;# g
set_property IOSTANDARD LVCMOS33 [get_ports {seg[*]}]

## 7-segment digit anodes (active low). Only AN0 used; others driven to '1'
inside.
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[*]}]

## Configuration: prevent unused-pin warnings
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO        [current_design]
```